

Trilithium: Efficient and Universally Composable Distributed ML-DSA Signing

Antonín Dufka¹[0009–0003–5058–2571], Semjon Kravtšenko¹[0009–0007–8204–9519],
Peeter Laud¹[0000–0002–9030–8142], and Nikita Snetkov^{1,2}[0000–0002–1414–2080]

¹ Cybernetica AS, Mäealuse 2/1, 12618 Tallinn, Estonia

{antonin.dufka, semjon.kravtsenko, peeter.laud, nikita.snetkov}@cyber.ee

² Tallinn University of Technology, Akadeemia tee 15a, 12618 Tallinn

Abstract. In this paper, we present Trilithium: a protocol for distributed key generation and signing compliant with FIPS 204 (ML-DSA). Our protocol allows two parties, “server” and “phone” with assistance of correlated randomness provider (CRP) to produce a standard ML-DSA signature. We prove our protocol to be secure against a malicious server or phone in the universal composability model, introducing some novel techniques to argue the security of two-party secure computation protocols with active security against one party, but only active privacy against the other. We provide an implementation of our protocol in Rust and benchmark it, showing the practicality of the protocol. As additional contribution, we propose an actively secure generalization of Attrapadung et al. (ASIA CCS ’22) comparison protocol and a new rounding protocol for fractions with private numerator.

Keywords: ML-DSA · Crystals-Dilithium · distributed signing · MPC · Universal Composability · threshold signatures

1 Introduction

Two-party protocols are an important use-case for threshold signing. They have established a firm foothold in ecosystems with formalized frameworks of certification and assurance. An example of a deployed two-party protocol is Smart-ID³ is used by millions of residents of several European countries, and certified as a Qualified Signature Creation Device (QSCD)⁴. Smart-ID allows a smartphone user to produce RSA signatures distributively: the protocol is deployed as a central server, keeping one share of every user’s signing key, while the second share is managed by that user’s smartphone. It is interoperable with the rest of the ecosystem — the signatures created with the Smart-ID service are indistinguishable from other RSA signatures and can be verified by anyone.

With National Institute of Standards and Technology announcing new post-quantum standards⁵ and the inevitable transition to quantum-safe primitives

³ <https://www.smart-id.com/>

⁴ <https://www.smart-id.com/e-service-providers/smart-id-as-a-qscd/>

⁵ <https://www.nist.gov/news-events/news/2024/08/>

nist-releases-first-3-finalized-post-quantum-encryption-standards

[90,62,94,85], currently deployed threshold schemes have to be replaced. While there are several pathways towards interoperability, certification-reliant ecosystems demand the use of standardized cryptosystems. As ML-DSA (formerly known as Crystals-Dilithium) [48,80] appears to be on its way to becoming the *default* quantum-safe signature scheme, we ask the question:

Can an efficient and actively secure distributed protocol for ML-DSA be constructed? In this paper, we answer this question affirmatively.

Related work

Over the past few years, substantial number of works have studied the topic of distributed signatures, both t -out-of- n and n -out-of- n scenarios. For quantum-vulnerable schemes there exist many efficient and practical distributed signing protocols: for RSA [58,36,19,17,29], ECDSA [54,73,43,25,42,26,23,96,71,44,69], probabilistic [83,47,81,67,72] and deterministic [82,53,51,68] Schnorr signature schemes. There is also a fair amount of post-quantum distributed signature schemes [65,27,8,34,46], especially in the area of lattice-based protocols [32,38,16] [52,1,57,92,63]. While many of them can be considered efficient, none of them are interoperable with new standards, except Cozzo and Smart [32] protocols. Still, their approach does not provide active security and cannot be directly applied in real-life applications without substantial modifications.

Our protocol is an instance of classical secret-sharing [70] based secure two-party / multi-party computation [97,28,10,39], with proofs in the universal composability model [20,24]. While the execution of the ML-DSA key generation algorithm on top of a 2PC / MPC protocol set is relatively straightforward, if one can create *small* private values (which may be reduced to the conversion of private values between different moduli of sharing [84,33]), the signature creation is trickier. Besides the generation of small private values (same as key generation), it has two main parts: high bits computation, and rejection sampling. The latter is mostly inequality checking, for which a number of MPC protocols have been proposed [76,4,99]; we are adapting Attrapadung et al. [4] for our purposes.

The high bits computation mostly consists of division (with public divisor) together with truncation / rounding of the result. Protocols for secure division have been proposed [14,95,93], but they are not much related to our techniques, and neither is Coron et al.’s *DecomposeMod* [31]. Our division protocol is highly specific to the used divisor.

Additive secret-sharing based techniques are used to secure hardware implementations of cryptographic algorithms against an adversary probing a limited number of wires [60]. The state of the art for Dilithium is presented by Coron et al. [31,30], with implementation by Raj et al. [89]. Coron et al. [31] propose the *DecomposeMod* gadget for the private computation of high bits. An implementation of this gadget in software gives a communication-efficient MPC protocol, but with round complexity higher than our techniques.

We separately want to outline two very recent reports. Bienstock et al. [11] propose an MPC protocol for ML-DSA using relatively standard techniques,

including `DecomposeMod` [31] for high bits computation and Rabbit [76] for rejection checking. They propose a solution for releasing the commitment message of the Dilithium identification protocol in aborted sessions. Borin et al. [15] give the construction of a Fiat-Shamir-with-Aborts (FSwA) based threshold signature scheme with the distributions of secrets and the related hardness assumptions rather different from ML-DSA. However, the verification algorithm of their scheme is identical to ML-DSA, i.e. they achieve *interchangeability* [86] with ML-DSA.

Our actively secure two-party protocols have subprotocols that provide active security against one of the parties, but only active *privacy* against the other party. This consideration of privacy as a separate goal is inspired by [13,87]. Similar considerations are present in certain offline precomputation protocols of MPC protocol sets [49], but our methods of composition appear to be novel.

Our Contributions

1. We propose *Trilithium*⁶, a protocol for distributed ML-DSA key generation and signing, and prove it secure in the universal composability (UC) model.
 - We give a framework for actively *private* multiparty computation protocols in the UC model.
2. We provide an implementation and benchmarks of *Trilithium*.
3. As part of Trilithium, but with independent interest, we add security against active attacks by one of the parties to the two-party passively secure comparison protocol of Attrapadung et al. [4]. Moreover we generalize it from binary rings to arbitrary fields $\mathbb{Z}_p$.
4. Also as part of Trilithium, but perhaps with independent interest, we give a MPC protocol to compute $\lfloor w/\alpha \rfloor$ for private w and certain public α -s; our protocol has less communication rounds than `DecomposeMod`, the minimization of which is more important than reductions in the number of communicated bits.
5. We contribute to the discussion of the zero-knowledge properties of the Dilithium identification protocol, increasing the rigor by stating the hardness assumptions that imply the necessary properties for secure signing.

2 Preliminaries

2.1 Notation

Let $a \xleftarrow{\$} A$ denote a uniformly random sampling of an element a from the set A . Symbol $\perp$ is used to indicate a failure or rejection. $\mathbb{Z}$ denotes the set of integers, $\mathbb{N}$ the set of non-negative integers, $\mathbb{Z}_q$ the set of integers modulo q . The set $\{0, 1, \dots, n-1\}$ is denoted by $[n]$. Let R_q denote the ring of polynomials

⁶ Since Crystals-Dilithium (ML-DSA submission name to NIST PQC competition) was inspired by Star Trek series, we decided to call our protocol Trilithium, a compound from Star Trek Generations

$\mathbb{Z}_q[X]/(X^n + 1)$, where q is a prime number and $n \in \mathbb{N}$. We denote scalar values (including polynomials) by italic letters, vectors by bold lowercase letters, and matrices by bold uppercase letters.

For $\alpha \in \mathbb{N}$ and $x \in \mathbb{Z}$, let $x' =: x \bmod^\pm \alpha$ denote centered modulo reduction, where $x' \equiv x \pmod{\alpha}$ and either $-\frac{\alpha}{2} < x' \leq \frac{\alpha}{2}$ (if α is even) or $-\frac{\alpha-1}{2} \leq x' \leq \frac{\alpha-1}{2}$ (if α is odd).

For an element $x \in \mathbb{Z}_q$, its infinity norm is defined as $\|x\|_\infty = |x \bmod^\pm q|$, where $|x|$ denotes the absolute value of the element. For an element $p \in R_q$, its infinity norm is defined as $\|p\|_\infty = \max_i \|p_i\|_\infty$. For a vector $\mathbf{v}$, it is $\|\mathbf{v}\|_\infty = \max_i \|v_i\|_\infty$.

Let $v \in [B]$. The *characteristic vector* of v is a vector $\mathbf{cv}(v) = (t_0, t_1, \dots, t_{B-1})$, where $t_v = 1$ and $t_i = 0$ for all $i \neq v$. The length of $\mathbf{cv}(v)$, i.e. the upper bound B will be clear from the context.

2.2 Secure Multiparty Computation

Our protocol, Trilithium, is an instance of secret-sharing based secure multiparty computation (MPC) protocols, where the intermediate values have been additively shared (often modulo “the ML-DSA modulus” q) between two parties: the *Phone* and the *Server*. In these protocols, a private value $v \in \mathbb{Z}_q$ is stored as $\llbracket v \rrbracket = (\llbracket v \rrbracket_P, \llbracket v \rrbracket_S)$, where $v = \llbracket v \rrbracket_P + \llbracket v \rrbracket_S$, Phone knows the random value $\llbracket v \rrbracket_P \in \mathbb{Z}_q$ and Server knows the value (also random) $\llbracket v \rrbracket_S \in \mathbb{Z}_q$. We may write $\llbracket v \rrbracket^{(q)}$ to emphasize that sharing is done modulo q .

There exist protocols for arithmetic operations with additively shared values. While linear (including affine) operations can be performed locally by parties, others require communication between them. Such non-linear operations may benefit from some *correlated random values (CR)* distributed before the protocol start. For certain operations, there exist protocols for generating CR [64]. Alternatively, some deployments may warrant the use of a (somewhat) trusted party, the *Correlated Randomness Provider (CRP)* to generate and distribute CR. In this paper, we have chosen the CRP approach.

Storing v as $\llbracket v \rrbracket$ does not prevent a malicious party from tampering with its value by changing its share. It can be avoided with the help of (homomorphic) “*message authentication codes*” (MACs) [10,39]. In this paper, we use so-called *BeDOZa-style MACs* [10], where the Phone and the Server have random private MAC keys Δ_P, Δ_S , respectively, and the value v is stored as $\langle\langle v \rangle\rangle = (\llbracket v \rrbracket, \llbracket M_S \rrbracket, \llbracket M_P \rrbracket)$, where $M_S = \llbracket v \rrbracket_P \cdot \Delta_S$ and $M_P = \llbracket v \rrbracket_S \cdot \Delta_P$. We call M_S “*Server’s MAC on $\llbracket v \rrbracket$* ” and M_P “*Phone’s MAC on $\llbracket v \rrbracket$* ”. Affine operations with MAC-ed values $\langle\langle v \rangle\rangle$ need no communication.

Whenever the Phone P sends its share $\llbracket v \rrbracket_P$ to a Server S , it must accompany it with $\llbracket M_S \rrbracket_P$. S is able to verify the correctness of the MAC by checking whether $\llbracket v \rrbracket_P \cdot \Delta_S = \llbracket M_S \rrbracket_S + \llbracket M_S \rrbracket_P$. If P tries to change $\llbracket v \rrbracket_P$, then its MAC share has to be correspondingly changed, and this task is as hard as guessing the random Δ_S . The probability of guessing is $1/q$; and if this value is too large for given q , then several independent MAC keys Δ and MACs can be used. Independent MAC keys $\Delta^{(q_i)}$ are also used if the protocol makes use of several different moduli q_i .

Instead of shares of the MACs, their digests can be communicated, hence MACs do not noticeably contribute to the online complexity of protocols.

While Trilithium two-party protocol (if CRP is honest) is secure against a malicious P or S, it includes subprotocols that by their own provide security only against a malicious P and a semi-honest S, while still providing *privacy* against a malicious S, i.e. S learns nothing from the *messages* exchanged by the protocol. A malicious S can still change the outputs of such protocols, and learn something from them. In the following, we say that such (sub-)protocols are “AP-secure”, while we call protocols with active security against both parties “AA-secure” (See Sec. 5). In our AP-secure protocols, we often use the representation $\langle v \rangle = (\llbracket v \rrbracket, \llbracket M_S \rrbracket)$ for private values.

2.3 Universal Composability

We prove the security of our protocols in the *Universal Composability (UC)* framework [20,21], using the notation, the modeling of corruptions, and ideal functionality specification of [21, Secs. 2, 7.1, 7.2]. A *protocol* Π is a collection of *machines*. Machines can send *inputs* to and receive *outputs* from other machines; these input-output channels form a directed acyclic graph. A protocol runs with an *environment* $\mathcal{Z}$ and an adversary $\mathcal{A}$, where $\mathcal{Z}$ can send inputs to and receive outputs from certain *main* machines of Π , while $\mathcal{A}$ communicates with all machines in $\Pi \cup \{\mathcal{Z}\}$ via *backdoor* channels. Let $\text{EXEC}_{\mathcal{Z} \parallel \Pi \parallel \mathcal{A}}$ be the random variable of the output of $\mathcal{Z}$ in the run. A protocol Π is *at least as secure as* Π' , if for all adversaries $\mathcal{A}$ exists an adversary $\mathcal{S}$, such that for all environments $\mathcal{Z}$ we have $\text{EXEC}_{\mathcal{Z} \parallel \Pi \parallel \mathcal{A}} \approx \text{EXEC}_{\mathcal{Z} \parallel \Pi' \parallel \mathcal{A}'}$. Typically, one constructs $\mathcal{A}'$ as the composition of $\mathcal{A}$ with a simulator *Sim*.

An *ideal functionality* for n participants is a machine $\mathcal{F}$ that can take inputs from n machines. The *ideal protocol* $\text{IDEAL}_{\mathcal{F}}$ consists of $\mathcal{F}$ and machines $D_1^{\mathcal{F}}, \dots, D_n^{\mathcal{F}}$ called the “*dummy parties*”, where the latter forward their inputs to $\mathcal{F}$ and send outputs from $\mathcal{F}$ back to their callers. The “dummy parties” are the main machines of $\text{IDEAL}_{\mathcal{F}}$. A protocol Π is a *secure implementation of* $\mathcal{F}$ if it is at least as secure as $\text{IDEAL}_{\mathcal{F}}$.

Our protocols have two participants — P(hone) and S(erver). A protocol Π consists of two machines, $\mathcal{M}_P$ and $\mathcal{M}_S$ executing the protocol logic, as well as subprotocols $\text{IDEAL}_{\mathcal{F}_i}$ for various ideal components $\mathcal{F}_i$ corresponding either to abstractions of subprotocols, or the generation of CR. In our proofs, we make heavy use of composability: if a subprotocol $\text{IDEAL}_{\mathcal{F}'}$ is part of Π that securely implements the ideal functionality $\mathcal{F}$, and if Π' is a secure implementation of $\mathcal{F}'$, then $(\Pi \setminus \text{IDEAL}_{\mathcal{F}'}) \cup \Pi'$, also securely implements $\mathcal{F}$ (assume no name clashes). We say that Π securely implements $\mathcal{F}$ in $\mathcal{F}'$ -*hybrid model*. In particular, the CRP will be modeled as a set of ideal functionalities.

The adversary $\mathcal{A}$ may (statically) *corrupt* one of P and S. To corrupt X, $\mathcal{A}$ notifies $\mathcal{Z}$, any ideal functionalities and $\mathcal{M}_X$ about it at the start of the execution. The machine $\mathcal{M}_{\bar{X}}$ (where $\{X, \bar{X}\} = \{P, S\}$) is not notified. During the run, the control of $\mathcal{M}_X$ and any dummy $D_{\mathcal{F}}^X$ is given to $\mathcal{A}$. The functionality $\mathcal{F}$ no longer talks to $D_{\mathcal{F}}^X$, but to $\mathcal{A}$.

2.4 ML-DSA

Module-Lattice-Based Digital Signature Algorithm (ML-DSA) is a quantum-safe digital signature scheme. ML-DSA follows the Fiat-Shamir with Aborts (FSwA) [75] paradigm and its security relies on hardness of *MLWE*, *MSIS* and *SelfTargetMSIS* problems.

Ducas et al. [48, Fig. 1] present a “template” signature scheme, which was turned to Crystals-Dilithium and ML-DSA by shortening the public key and adding some *hints* to signatures. The integrity properties of the “template” scheme are no worse than those of ML-DSA. The public key is shortened by removing some of its bits. Given a public key and a signature of the “template” scheme, one can compute the *hints* for ML-DSA, resulting in a valid ML-DSA signature. Hence it is sufficient to give a two-party protocol for key generation and signing in the “template” scheme; both parties locally store the public key of the “template” scheme, publish the shortened key, and locally add the hints to the signatures produced by the signing protocol. In the following, let us describe the “template” scheme; see [48] and FIPS 204 [80] for the description of the full scheme.

The parameters of the “template” scheme are a prime number q , data sizes k, ℓ, n, τ , and value sizes γ_1, γ_2, η . Denote $\alpha = 2\gamma_2$ and $\beta = \eta \cdot \tau$. The scheme needs $\alpha \mid (q - 1)$. Let $S_m \subseteq R_q$ [resp. $\tilde{S}_m \subseteq R_q$] for $m \in \mathbb{N}$ denote the set of polynomials with all coefficients between $-m$ [resp. $-m + 1$] and m . Let $B_\tau \subseteq S_1$ denote the set of polynomials with exactly τ non-zero coefficients.

Given a value $w \in \mathbb{Z}_q = \{0, \dots, q - 1\}$, we define $\text{HighBits}(w) \in \mathbb{Z}_q$ as 0 if $w' = q - 1$, and $\lfloor w'/\alpha \rfloor$ otherwise, where $w' = (w + \alpha/2 - 1) \bmod q$. This definition is equivalent to [48, Fig. 3]. Also equivalent is defining $\text{LowBits}(w) := (w - \alpha \cdot \text{HighBits}(w)) \bmod q$. One commonly uses superscript $(\cdot)^H$ and $(\cdot)^L$ for high and low bits. The definitions of high and low bits are extended from $\mathbb{Z}_q$ to R_q coefficient-wise, and to vectors of polynomials component-wise.

The signature scheme employs “hashing into a ball”. It defines a hash function H , based on SHAKE [79], whose output is uniformly distributed in B_τ . See [48, 80] for its precise description.

The key generation algorithm of the “simplified” scheme [48, Fig. 1] is given in Alg. 1. The algorithm for a single signing *attempt* is given in Alg. 2 (white background only; rest is discussed in Sec. 3). If Alg. 2 returns $\perp$, then the signer is expected to rerun it, with fresh $\mathbf{y}$. This does not reduce the security of our protocols — the frequency of retries is independent of the private key. The ML-DSA specification has picked the parameters so, that the average number of reruns is 4 to 5. In this paper, we give a two-party protocol $\Pi_{\text{ML-DSA}}$ for Alg. 1 and 2, securely realizing $\mathcal{F}_{\text{ML-DSA}}$ given as Func. 1 (with adversary $\mathcal{S}$). Recall that if participant $\mathbf{X}$ is corrupted then its input actually comes from $\mathcal{S}$. Also, $\mathcal{Z}$ may still receive output from $\mathbf{X}$ because $\mathcal{S}$ controls the dummy party $D^{\mathbf{X}}$.

Our protocols also make use of the verification algorithm of the template scheme. The verification consists of making sure that $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, computing $\mathbf{x}_0 \leftarrow \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$, and checking that $c = H(\mu, \mathbf{x}_0^H)$. Note that $\mathbf{x}_0 = \mathbf{w} - c \cdot \mathbf{s}_2$.

Output: Keypair pk, sk
 $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$; // Generated from a seed
 $(\mathbf{s}_1, \mathbf{s}_2) \xleftarrow{\$} S_\eta^\ell \times S_\eta^k$
 $\mathbf{t} \leftarrow \mathbf{A} \cdot \mathbf{s}_1 + \mathbf{s}_2$
return $pk = (\mathbf{A}, \mathbf{t}), sk = (\mathbf{s}_1, \mathbf{s}_2)$
Algorithm 1: Key generation in “template” scheme

Input: Keypair $(\mathbf{A}, \mathbf{t}; \mathbf{s}_1, \mathbf{s}_2)$, message digest μ
Output: Signature $\sigma = (\mathbf{z}, c)$, or $\perp$
 $(\mathbf{y}, \mathbf{e}_w) \xleftarrow{\$} \tilde{S}_{\gamma_1}^\ell \times S_\eta^k$; // Follows [80], not [48]
 $\mathbf{w} \leftarrow \mathbf{A} \cdot \mathbf{y} + \mathbf{e}_w$
 $c \in B_\tau \leftarrow H(\mu, \mathbf{w}^H)$
 $\mathbf{z} \leftarrow \mathbf{y} + c \cdot \mathbf{s}_1$
 $\mathbf{r}_0 \leftarrow \text{LowBits}(\mathbf{w} - c \cdot \mathbf{s}_2)$
if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ **and** $\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$ **then**
| **return** $(\mathbf{z}, c)$
else return $\perp$;
Algorithm 2: Signing attempt in the “template” scheme
and modification of [11] : SignAtt

- On input $(\text{keygen}, \text{sid})$ from both $\mathcal{P}$ and $\mathcal{S}$, use Alg. 1 to generate a new keypair $(\mathbf{A}, \mathbf{t}; \mathbf{s}_1, \mathbf{s}_2)$. Send $(\text{pkey}, \text{sid}, \mathbf{A}, \mathbf{t})$ to $\mathcal{S}$. Receive $(\text{proceed}, \text{sid})$ from $\mathcal{S}$ and send $(\text{pkey}, \text{sid}, \mathbf{A}, \mathbf{t})$ to the honest party(s). Accept no further (keygen) -commands.
- On input $(\text{sign}, \text{sid}, \mu)$ from both $\mathcal{P}$ and $\mathcal{S}$, compute $\sigma \leftarrow \text{SignAtt}(\mathbf{A}, \mathbf{t}; \mathbf{s}_1, \mathbf{s}_2; \mu)$ (include steps in grey background iff $\mathbf{m} = 1$). Let $\text{com} \leftarrow \mathbf{c} \cdot \mathbf{w}^H$, where $\mathbf{w}$ was computed inside SignAtt. Send $(\text{signature}, \text{sid}, \mu, \sigma, \text{com})$ to $\mathcal{S}$. Receive $(\text{proceed}, \text{sid})$ from the adversary. Send $(\text{signature}, \text{sid}, \sigma)$ to the honest party(s).

Functionality 1: $\mathcal{F}_{\text{ML-DSA}}^{\mathbf{c}, \mathbf{m}}$ where $\mathbf{c}, \mathbf{m} \in \{0, 1\}$: ideal functionality for 2-party ML-DSA ($\mathbf{c} = \mathbf{m} = 0$) and modifications

Let us recall the values of the most important parameters in the three parameter sets of ML-DSA [80]: $q = 2^{23} - 2^{13} + 1$, $\alpha \in \{190464, 523776\}$, $\eta \in \{2, 4\}$, $\gamma_1 \in \{2^{17}, 2^{19}\}$. Certain properties of these values make our protocols efficient.

2.5 Lattice-based hardness assumptions

While the unforgeability of ML-DSA, and hence also the property of our protocols to constitute a secure threshold signature scheme is based on certain lattice-based hardness assumptions, we also “independently” use one of these assumptions to show that an adversarial party cannot cheat:

Definition 1 (Decisional Module-LWE, D-MLWE). Let χ be a probability distribution and $n, m, q, \eta \in \mathbb{N}$. We define the advantage of adversary $\mathcal{A}$ in breaking decisional Module-LWE for the set of parameters (q, n, m, η, χ) as $\text{Adv}_{(q, n, m, \eta, \chi)}^{\text{D-MLWE}}(\mathcal{A}) := |P_0^{\text{D-MLWE}} - P_1^{\text{D-MLWE}}|$, where:

Input: $pk = (\mathbf{A}, \mathbf{t})$, $sk = (\mathbf{s}_1, \mathbf{s}_2)$
Output: A sample from the distribution of transcripts
 $\mathbf{y} \xleftarrow{\$} \bar{S}_{\gamma_1}^\ell$
 $\mathbf{w} \leftarrow \mathbf{A} \cdot \mathbf{y}$
 $c \xleftarrow{\$} B_\tau$
 $\mathbf{z} \leftarrow \mathbf{y} + c \cdot \mathbf{s}_1$
 $\mathbf{r}_0 \leftarrow \text{LowBits}(\mathbf{w} - c \cdot \mathbf{s}_2)$
if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ **then return** $(\mathbf{w}^H, c, \perp)$;
if $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$ **then return** $(\mathbf{w}^H, c, \perp)$;
return $(\mathbf{w}^H, c, \mathbf{z})$
Algorithm 3: Generating transcripts of Dilithium identification protocol:
Trans

$P_0^{D\text{-MLWE}} = \Pr[b = 1 : \mathbf{A} \leftarrow R_q^{n \times m}, (\mathbf{s}_1, \mathbf{s}_2) \leftarrow \chi(S_\eta^m \times S_\eta^n), \mathbf{t} := \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2, b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t})$; $P_1^{D\text{-MLWE}} = \Pr[b = 1 : \mathbf{A} \leftarrow R_q^{n \times m}, \mathbf{t} \leftarrow R_q^n, b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t})$.

Definition 2 (Computational Module-LWE, C-MLWE). Let χ be a probability distribution and $n, m, q, \eta \in \mathbb{N}$. We define the advantage of adversary $\mathcal{A}$ in breaking computational Module-LWE for the set of parameters (q, n, m, η, χ) as follows: $\text{Adv}_{(q, n, m, \eta, \chi)}^{\text{C-MLWE}}(\mathcal{A}) := \Pr[\mathbf{s}_1 = \mathbf{s}'_1 : \mathbf{A} \leftarrow R_q^{n \times m}, (\mathbf{s}_1, \mathbf{s}_2) \leftarrow \chi(S_\eta^m \times S_\eta^n), \mathbf{t} := \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2, \mathbf{s}'_1 \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t})]$.

Definition 3 (Module-SIS, MSIS). Let $n, m, q, \eta \in \mathbb{N}$. We define the advantage of adversary $\mathcal{A}$ in breaking Module-SIS for the set of parameters (q, n, m, η) as follows: $\text{Adv}_{(q, n, m, \eta)}^{\text{MSIS}}(\mathcal{A}) := \Pr[\mathbf{A} \mid \mathbf{I}] \cdot \mathbf{x} = \mathbf{0} \pmod{q} \text{ and } 0 < \|\mathbf{x}\|_\infty \leq \eta : \mathbf{A} \leftarrow R_q^{n \times m}, \mathbf{x} \leftarrow \mathcal{A}(\mathbf{A})]$.

3 Zero-knowledge of Dilithium identification protocol

Dilithium signature scheme is built from an underlying three-move identification protocol using the FSWA paradigm. In this protocol, the prover first sends the *commitment*: the value $\mathbf{w}^H$ in Fig. 2. The verifier sends back a challenge c randomly sampled from B_τ . Prover computes $\mathbf{z}$, makes the rejection checks (also computing $\mathbf{r}_0$ for that) and sends $\mathbf{z}$ only if the checks pass. Kiltz et al. show [66] that a FSWA-based signature scheme is secure if the identification protocol is sound, and has *no-abort Honest Verifier Zero-Knowledge* (naHVZK). The latter means that the passing transcripts can be simulated. Alg. 3 gives the transcript generation algorithm for Dilithium's identification protocol, its naHVZK means that its output distribution (conditioned on finishing in line 3) can be sampled without knowing the private key.

Would the signature scheme still be secure if the adversary had the ability to learn the commitments $\mathbf{w}^H$ also from unsuccessful signing attempts? Previous literature discusses this question, that we review in App. D. But formally, we would need the identification protocol to be HVZK. In particular, Devevey et al. [41, Thm. 4] and Grilo et al. [55, Thm. 3] show that a strong form of HVZK

Input: $pk = (\mathbf{A}, \mathbf{t})$, $sk = (\mathbf{s}_1, \mathbf{s}_2)$
Output: A sample from the distribution of transcripts
 $c \xleftarrow{\$} B_\tau$
if Bernoulli($p_{\mathbf{z}}$) **then**
 $\bar{\mathbf{z}} \xleftarrow{\$} S_{\gamma_1 - \beta - 1}^\ell$
 if $\|\text{LowBits}(\mathbf{A} \cdot \bar{\mathbf{z}} - c \cdot \mathbf{t})\|_\infty < \gamma_2 - \beta$ **then**
 $\mathbf{z} \xleftarrow{\$} \{\mathbf{z}' \in S_{\gamma_1 - \beta - 1}^\ell \mid \|\text{LowBits}(\mathbf{A} \cdot \mathbf{z}' - c \cdot \mathbf{t})\|_\infty < \gamma_2 - \beta\}$
 return $(\text{HighBits}(\mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}), c, \mathbf{z})$
 else
 $\mathbf{y} \xleftarrow{\$} \{\mathbf{y}' \in S_{\gamma_1 - \beta - 1}^\ell - c\mathbf{s}_1 \mid \|\text{LowBits}(\mathbf{A} \cdot \mathbf{y}' - c\mathbf{s}_2)\|_\infty \geq \gamma_2 - \beta\}$
 return $(\text{HighBits}(\mathbf{A} \cdot \mathbf{y}), c, \perp)$
else
 $\mathbf{y} \xleftarrow{\$} \{\mathbf{y}' \in \tilde{S}_{\gamma_1}^\ell \mid \|c \cdot \mathbf{s}_1 + \mathbf{y}'\|_\infty \geq \gamma_1 - \beta\}$
 return $(\text{HighBits}(\mathbf{A} \cdot \mathbf{y}), c, \perp)$

Algorithm 4: Generating transcripts of Dilithium identification protocol; reordered steps

(sHVZK), where the distinguisher of transcripts is also given the private key, is sufficient for the signature scheme to still provide UF-CMA security against quantum adversaries [11]. The sHVZK property requires the existence of an efficient algorithm Sim , such that the pair $(sk, \text{Sim}(pk))$ is indistinguishable from the pair $(sk, \text{Trans}(pk, sk))$, where (pk, sk) is an honestly generated keypair.

In order to show sHVZK of the identification protocol, Bienstock et al. [11] slightly modify the “template” scheme, as shown in Alg. 2. They add a small error to $\mathbf{w}$, relying on the hinting in full Dilithium to make the verification work. This added error allows them to reduce the sHVZK property of the corresponding identification protocol to the MLWE assumption.

It turns out that their proof strategy is adaptable to unmodified Dilithium, using a kind of MLWR (Module Learning with Rounding) assumption [6] instead. The operations in Alg. 3 can be reordered (see App. D) as in Alg. 4, where $p_{\mathbf{z}} = |S_{\gamma_1 - \beta - 1}^\ell| / |\tilde{S}_{\gamma_1}^\ell|$ is the probability of $\mathbf{z}$ passing the rejection check. Lines 4–4 make the random choice of $\mathbf{r}_0$ passing or not passing the rejection check. The three branches of Alg. 4 correspond to the three **return**-statements in Alg. 3, in opposite order. The set $S_{\gamma_1 - \beta - 1}^\ell - c\mathbf{s}_1$ denotes the set of all such vectors $\mathbf{y}'$, where $\mathbf{y}' + c\mathbf{s}_1 \in S_{\gamma_1 - \beta - 1}^\ell$.

The choice of the branches, and the computations in the first branch of Alg. 4 can be performed without sk . For the other two branches, we need an indistinguishability assumption. If $q, n, k, \ell, \alpha \in \mathbb{N}$ and $S_{\mathbf{A}} \subseteq R_q^\ell$ for each $\mathbf{A} \in R_q^{k \times \ell}$ then we let the $\text{MLWR}_{q,n,k,\ell,\alpha,S}$ -assumption state that an efficient algorithm cannot distinguish a pair $(\mathbf{A}, \mathbf{b}^H)$ with uniformly sampled $\mathbf{A} \times \mathbf{b} \leftarrow R_q^{k \times \ell} \times R_q^k$ from a pair where $\mathbf{A}$ is still uniformly sampled, but $\mathbf{b} \leftarrow \mathbf{A} \cdot \mathbf{y}$, where $\mathbf{y}$ is uniformly sampled from $S_{\mathbf{A}}$. We use this assumption for the Dilithium parameters q, n, k, ℓ, α and the sets $S_{\mathbf{A}}$ in lines 4 and 4 of Alg. 4. In this way, we can return the high bits of a random vector in the following lines.

Is the hardness assumption reasonable for the given parameters? There exist reductions from (M)LWR to (M)LWE [3,2,12,74,61], but they do not really apply to the parameters of ML-DSA. But our choices of $S_{\mathbf{A}}$ are in line with the techniques of [3,61] that mainly rely on the conditional entropy of $\mathbf{y}$ being high enough, given $\mathbf{A}$ and $\text{HighBits}(\mathbf{A} \cdot \mathbf{y})$. Indeed, the low bits and high bits of a value can be considered to be rather independent. Other reductions are onto MLWE assumptions with similarly distributed $\mathbf{y}$; these distributions can be linearly shifted, or combined with / subtracted from each other as in [88, Lemma 5].

MLWR is the basis of security for a number of proposed public-key encryption (PKE) schemes [40,77,59]. They typically use the same (or smaller) values of n, k, ℓ, q , (much) smaller α and sets $S_{\mathbf{A}}$. Hence all our parameters, except q , are more conservative than in these PKE schemes.

The sHVZK property of the Dilithium identification protocol (and its modification [11]) gives us a secure implementation of $\mathcal{F}_{\text{ML-DSA}}^{0,m}$ in $\mathcal{F}_{\text{ML-DSA}}^{1,m}$ -hybrid model, where a *global random oracle* functionality $\mathcal{G}_{\text{PRO}}$ [18,22] is available for all machines to use in place of the hash function H . In the protocol $\Pi_{\text{ML-DSA}}^{0,m}$, the machines $\mathcal{M}_{\text{P}}$ and $\mathcal{M}_{\text{S}}$ simply relay the messages between $\mathcal{Z}$ and $\mathcal{F}_{\text{ML-DSA}}^{1,m}$. A simulator, when it receives (signature, $\text{sid}, \mu, \perp, 0$) from $\mathcal{F}_{\text{ML-DSA}}^{0,m}$ and has to come up with $\mathbf{w}^H$ in order to send it to $\mathcal{A}$, will run the simulated transcript generation algorithm until obtaining $(\mathbf{w}^H, c, \perp)$, and set $H(\mu, \mathbf{w}^H) \leftarrow c$. By our arguments above, such simulation is indistinguishable from the real protocol.

In the rest of this paper, we present protocols securely implementing $\mathcal{F}_{\text{ML-DSA}}^{1,m}$. We consider both the original ML-DSA and its modified variant [11], due to the more standard zero-knowledge arguments for the latter.

4 Trilithium: Universally Composable Distributed ML-DSA Signature Scheme

In this section, we define Trilithium and describe our subprotocols, using the notation typical for MPC protocols. Our descriptions are bottom-up, starting with small operations and finishing with the protocols for key generation and signing. The arguments and descriptions in this section are informal; formal security arguments are given in Sec. 5.

In our protocols, we use the protocol `randbit()` with no inputs, returning $\langle\langle b \rangle\rangle^{(q)}$ for a random $b \in \{0, 1\}$ and a modulus q determined by the context. We use the bit conversion protocol `convbit`($\langle\langle b \rangle\rangle^{(2)}$) that takes a bit b shared modulo 2, and returns $\langle\langle b \rangle\rangle^{(q)}$ — the same bit shared modulo q . Obviously, we also make use of protocols for adding and multiplying shared values, and for generating public random values modulo some prime number Q (called `pubrand()`). All these protocols are standard, and described e.g. in [98]. We use the CRP for any correlated randomness that these protocols require.

Input: $\langle\langle v \rangle\rangle = (\llbracket v \rrbracket, \llbracket M_S \rrbracket, \llbracket M_P \rrbracket)$,
 where $v \in \mathbb{Z}_q$, $M_S = \llbracket v \rrbracket_P \cdot \Delta_S$ and $M_P = \llbracket v \rrbracket_S \cdot \Delta_P$
Output: v
 $S \rightarrow P$: $\llbracket v \rrbracket_S$ and $\llbracket M_P \rrbracket_S$
 P : **stop** if $\llbracket v \rrbracket_S \cdot \Delta_P - \llbracket M_P \rrbracket_P \neq \llbracket M_P \rrbracket_S$
 $P \rightarrow S$: $\llbracket v \rrbracket_P$ and $\llbracket M_S \rrbracket_P$
 S : **stop** if $\llbracket v \rrbracket_P \cdot \Delta_S - \llbracket M_S \rrbracket_S \neq \llbracket M_S \rrbracket_P$
 P and S : **return** $\llbracket v \rrbracket_P + \llbracket v \rrbracket_S$

Algorithm 5: Declassification protocol: declassify

- On input (init, sid , Δ_P) from P and (init, sid , Δ_S) from S : Store Δ_P and Δ_S .
- If P is honest: on input (declassify, sid , $\langle\langle v \rangle\rangle_P$) from P and (declassify, sid , $\langle\langle v \rangle\rangle_S$) from S , where $\langle\langle v \rangle\rangle_X = (\llbracket v \rrbracket_X, \llbracket M_S \rrbracket_X, \llbracket M_P \rrbracket_X)$, proceed as follows:
 1. Compute $v \leftarrow \llbracket v \rrbracket_P + \llbracket v \rrbracket_S$. Let the boolean b_X indicate whether M_X verifies.
 Let $v_X \leftarrow b_X ? v : \perp$.
 2. Send (result, sid , v_P) to P .
 3. If b_P and S is corrupt, send (result, sid , v , $\llbracket M_S \rrbracket_P$) to S .
 4. If b_P and S is honest, send (result, sid , v_S) to S .
 5. If $\neg b_P$, send (result, sid , $\perp$) to either S or S .
- If P is corrupt: on input (declassify, sid) from P (i.e. S) and (declassify, sid , $\langle\langle v \rangle\rangle_S$) from S , proceed as follows:
 1. Send (input, sid , $\llbracket v \rrbracket_S$, $\llbracket M_P \rrbracket_S$) to S . Get back (data, sid , $\langle\langle v \rangle\rangle_P$) or (data, sid , $\perp$).
 2. Compute v , b_X , v_X as in previous case.
 3. Send (result, sid , v_S) to S .

Functionality 2: $\mathcal{F}_{\text{declassify}}^{\text{AA}}$: ideal functionality for declassification

4.1 Declassification

Let us start with the *declassification* protocol **declassify** given in Alg. 5. As discussed in Sec. 2.2, it involves the checking of MACs, using the MAC keys $\Delta_P^{(q)}$ and $\Delta_S^{(q)}$ for the modulus q . The inputs to the declassification protocol are the shares $\llbracket v \rrbracket$ of the value v , and the shares of Server's MAC M_S and Phone's MAC M_P for this sharing $\llbracket v \rrbracket$. Recall that shares of MAC's are actually not sent, but only their digests are sent and compared (Sec. 2.2).

We see that Alg. 5 is asymmetric: P learns the value v first. This is reflected in the functionality $\mathcal{F}_{\text{declassify}}^{\text{AA}}$, given in Func. 2, which we introduce in order to handle declassification in UC fashion. In Func. 2 and below, we use X to denote either S or P ; the functionality allows one of the parties to be corrupted. Recall that the input of a corrupt participant comes from $\mathcal{S}$ (implicitly), and the output to corrupt participant goes to $\mathcal{S}$ (explicitly). In App. B.1 we show that Alg. 5 is indeed a secure implementation of Func. 2.

Let us emphasize once more that the goal of this paper is to securely implement Func. 1. The main design goal of all other ideal functionalities is to enable the security proofs.

Par: Q — a small prime number
Output: $\langle\langle v \rangle\rangle^{(q)}$, $v \xleftarrow{\$} \{0, \dots, Q-1\}$
CR: $\langle\langle \mathbf{t} \rangle\rangle^{(q)}$, s.t. $|\mathbf{t}| = Q$, $\mathbf{t}$ is a one-hot 0-1 vector
 $r \leftarrow \text{pubrand}() \in \mathbb{Z}_Q$
return $\sum_{i=0}^{Q-1} i \cdot \langle\langle t_{(i+r) \bmod Q} \rangle\rangle$
Algorithm 6: Generating a small random value: $\Pi_{\text{gen_small}}$

It turns out that Alg. 5 and Func. 2 are also good descriptions of the declassification of $\langle\langle v \rangle\rangle$, if we leave out checks related to P’s MAC. In particular:

- Remove M_P and line 5 from Alg. 5.
- Let $b_P \equiv \text{true}$ in Func. 2.

In App. B.1 we also discuss how this algorithm securely implements this functionality, denoted $\mathcal{F}_{\text{declassify}}^{\text{AP}}$.

4.2 Randomness generation

The protocol `gen_small`, parametrized by a small prime number Q and given in Alg. 6, takes no inputs and returns a uniform random value $\langle\langle v \rangle\rangle^{(q)}$, such that $0 \leq v < Q$. The correlated randomness used by this algorithm is a private 0-1 vector, where the position of the only value “1” is randomly selected by the CRP, and unknown to P and S.

A simpler protocol, where the CRP just generates the value $v \in \{0, \dots, Q-1\}$, secret-shares it modulo q and hands the shares over to P and S (i.e. the CR is the output) would also be UC-secure according to our definitions in Sec. 5 (Func. 3). We have chosen to use a more complex protocol, such that all parties affect the value of v . This allows us to discuss security also against a malicious CRP (App. F). Similarly, we expect the output of `randbit()` (used in Alg. 11) to be affected by all parties.

In the rest of Sec. 4 we distinguish between parameters (fixed at compile-time) and inputs (known only at run-time) to a protocol. We write $f[c](a)$ to denote the invocation of a protocol f with parameter c and input a . We may omit the parameters if they’re clear from the context. The correlated randomness **CR** used by the protocols is assumed to be uniformly distributed, subject to the listed constraints.

4.3 Private key generation

Generation of private small random values is all that is needed for ML-DSA key generation. The protocol `keygen` is given in Alg. 7. It generates the necessary number of random values and arranges them as coefficients of polynomials (with X denoting the formal variable). Here and in the following, we do not make explicit whether polynomials are represented coefficient-wise or in the Number Theoretic Transform (NTT) representation. Note that the conversion between

Par: ML-DSA parameters k, ℓ, η
Input: The matrix $\mathbf{A}$ of the public key
Output: The private key $\langle\langle \mathbf{s}_1 \rangle\rangle, \langle\langle \mathbf{s}_2 \rangle\rangle$
foreach $i \in [k + \ell], t \in [256]$ **do**
 if $\eta = 2$ **then**
 $\langle\langle r_{i,t} \rangle\rangle \leftarrow \text{gen_small}[5]() - 2$
 else $\eta = 4$
 $\langle\langle r_{i,t} \rangle\rangle \leftarrow 3 \cdot \text{gen_small}[3]() + \text{gen_small}[3]() - 4$
 $\langle\langle \mathbf{s}_1 \rangle\rangle \leftarrow (\sum_{i=0}^{255} r_{i-1,t} X^t)_{i=1}^\ell$
 $\langle\langle \mathbf{s}_2 \rangle\rangle \leftarrow (\sum_{t=0}^{255} r_{i+\ell-1,t} X^t)_{i=1}^k$
return $\langle\langle \mathbf{s}_1 \rangle\rangle, \langle\langle \mathbf{s}_2 \rangle\rangle$
Algorithm 7: ML-DSA private key generation: keygen

Par: $B \in \mathbb{N}$. Denote $a = \lfloor q/B \rfloor$
Input: $\langle\langle v \rangle\rangle^{(q)}$, where $v < B$
Output: $\langle\langle b \rangle\rangle^{(q)}$, $b \in \{0, 1\}$, $b = 1 \Leftrightarrow v = 0$
CR: $\langle\langle m \rangle\rangle^{(q)}$, $\langle\langle \mathbf{t} \rangle\rangle^{(q)}$, s.t. $|\mathbf{t}| = B + 1$, $\mathbf{t} = \text{cv}(\lfloor m/a \rfloor)$
 $d \leftarrow \text{declassify}(\langle\langle m \rangle\rangle + a \cdot \langle\langle v \rangle\rangle)$
return $\langle\langle t_{\lfloor d/a \rfloor} \rangle\rangle$
Algorithm 8: Checking whether a value is 0: zero_check

the two is a linear operation. Our implementation converts wherever reasonable. In particular, the multiplications of polynomials are done using NTT.

It would perhaps be natural to have Alg. 7 also contain public key generation, starting from the generation of a random matrix $\mathbf{A}$, and finishing with the (linear) computation of $\langle\langle \mathbf{t} \rangle\rangle \leftarrow \mathbf{A} \cdot \langle\langle \mathbf{s}_1 \rangle\rangle + \langle\langle \mathbf{s}_2 \rangle\rangle$ and its declassification. For the uniformity of our security arguments (Sec. 5), these steps are not included here. But they are part of $\Pi_{\text{ML-DSA}}$.

4.4 Zero equality checking protocol

A necessary sub-protocol in distributed signing is the equality check for values shared modulo q . This is equivalent to checking whether a value is 0: the input to `zero_check`, given in Alg. 8 is a value $\langle\langle v \rangle\rangle$, and the output is $\langle\langle b \rangle\rangle$, where $b = 1$ iff $v = 0$. While protocols for this task have been proposed before [35], our protocol is simplified by the guarantee $v < B$ for a small upper bound B parametrizing the `zero_check` protocol. With this guarantee, we can keep the size of the correlated randomness proportional to B , not to the much larger modulus q . We discuss the correctness of Alg. 8 in App. C.1.

4.5 Symmetric reshare to parts

In order to compute the *high bits* of a value $w \in \mathbb{Z}_q$, we have to decompose it into high and low bits. Consider a vector $\mathbf{r} = (r_0, r_1, \dots, r_{n-1}) \in \mathbb{N}^n$, such that $R_n \geq 2q$, where we denote $R_0 = 1$ and $R_i = R_{i-1} \cdot r_{i-1}$. Given a value $v \in [2q]$, there exist unique $v_0, v_1, \dots, v_{n-1}$, such that $0 \leq v_i < r_i$ and $v = \sum_{i=0}^{n-1} R_i \cdot v_i$.

Par: $\mathbf{r} = (r_0, \dots, r_{n-1})$, where $\prod r_i \geq 2q$
Input: $\langle\langle w \rangle\rangle^{(q)}$
Output: $\langle\langle \mathbf{b}_0 \rangle\rangle^{(q)}, \dots, \langle\langle \mathbf{b}_{n-1} \rangle\rangle^{(q)}$, CV-s of digits of either w or $w + q$ (but never of $2q - 1$)
CR: $\langle\langle s \rangle\rangle^{(q)}, \langle\langle \mathbf{d}_0 \rangle\rangle^{(2)}, \dots, \langle\langle \mathbf{d}_{n-1} \rangle\rangle^{(2)}$: CV-s of digits of s
 $\forall i$: append $\langle\langle \mathbf{d}_i \rangle\rangle$ with $\langle\langle 0 \rangle\rangle$ -s, s.t. its length becomes $2r_i$
 $y \leftarrow \text{declassify}(\langle\langle w \rangle\rangle - \langle\langle s \rangle\rangle)$
 $(z_0, \dots, z_{n-1}) \leftarrow \text{the digits of } y$
for $i = 0$ **to** $n - 1$ **do**
 $\langle\langle \mathbf{p}'_i \rangle\rangle \leftarrow \text{rotate_right}(\langle\langle \mathbf{d}_i \rangle\rangle, z_i)$
 $\langle\langle \mathbf{p}''_i \rangle\rangle \leftarrow \text{rotate_right}(\langle\langle \mathbf{d}_i \rangle\rangle, z_i + 1)$
 $\langle\langle \mathbf{p}_i \rangle\rangle \leftarrow \text{if } i = 0 \text{ then } \langle\langle \mathbf{p}_0 \rangle\rangle \text{ else } \langle\langle c_i \rangle\rangle ? \langle\langle \mathbf{p}'_i \rangle\rangle : \langle\langle \mathbf{p}''_i \rangle\rangle;$
 $\langle\langle c_{i+1} \rangle\rangle \leftarrow \sum_{j=r_i}^{2r_i-1} \langle\langle p_{i,j} \rangle\rangle$
foreach $i \in [n]$, $j \in [r_i]$ **do**
 $\langle\langle b_{i,j} \rangle\rangle^{(q)} \leftarrow \text{convbit}(\langle\langle p_{i,j} \rangle\rangle^{(2)} + \langle\langle p_{i,j+r_i} \rangle\rangle^{(2)})$
return $\langle\langle \mathbf{b}_0 \rangle\rangle, \dots, \langle\langle \mathbf{b}_{n-1} \rangle\rangle$
Algorithm 9: Symmetric reshare to parts: `symm_rp`

We call $(v_0, \dots, v_{n-1})$ the *representation of v in the mixed-radix base $\mathbf{r}$* , and call the individual values v_i the *digits of v (wrt. $\mathbf{r}$)*.

Our protocol `symm_rp`, called “*symmetric reshare to parts*” and parametrized by a mixed-radix basis $\mathbf{r}$, takes as input a shared value $\langle\langle w \rangle\rangle$ and returns the secret-shared digits of w in the representation of their characteristic vectors (CV) (where the length of the CV of the i -th digit is r_i). Actually, the digits may correspond either to w or to $w + q$; this is chosen randomly by `symm_rp`, given in Alg. 9.

We see that Alg. 9 begins by making public the value $y = (w - s) \bmod q$. The loop that follows basically adds the values y and s , thus explaining why the result randomly corresponds to either w or $w + q$ (but never to $2q - 1$). This addition is digit-wise. To add a digit with the CV $\mathbf{d}_i$, and the digit z_i , we rotate the vector to the right by z_i positions, thereby increasing the index of the position with “1” on it by z_i . As the length of $\mathbf{d}_i$ is $2r_i$, this rotation (also with $z_i + 1$) cannot move the element 1 of the vector over its right end. The rotation with z_i corresponds to no carry coming into the i -th digit, while the rotation with $z_i + 1$ corresponds to the incoming carry. The actual carry bit, c_i , is used to *obviously choose* between the two possibilities. The oblivious choice operation, $b ? x : y$ is defined to be x if $b = 1$, and y if $b = 0$. It can be computed as $y + b \cdot (x - y)$. In Alg. 9 we use it for vector-valued x and y ; in our implementation, the CR we use is for scalar-vector multiplication.

During this addition, we have an outgoing carry at the i -th digit if that digit is at least r_i , hence we find that carry by summing up the indicator bits of the upper half of the vector $\mathbf{p}_i$. The final value of that digit comes either from the lower (if there was no carry) or the upper (if there was carry) half of the vector $\mathbf{p}_i$.

Par: ML-DSA parameter $\alpha = 2 \cdot \gamma_2$. Let $s = (q - 1)/\alpha$
Input: $\langle\langle w \rangle\rangle$
Output: $\langle\langle v \rangle\rangle$, s.t. $v = w^H$
 $\langle\langle w' \rangle\rangle \leftarrow \langle\langle w \rangle\rangle + (\alpha/2 - 1)$
 $\mathbf{r} \leftarrow (r_0, \dots, r_4) = \begin{cases} (31, 24, 16, 16, 2s + 1), & \text{if } \alpha = 190464 \\ (31, 33, 32, 16, 2s + 1), & \text{if } \alpha = 523776 \end{cases}$
 $(\langle\langle \mathbf{b}_0 \rangle\rangle, \dots, \langle\langle \mathbf{b}_4 \rangle\rangle) \leftarrow \text{symm_rp}[\mathbf{r}](\langle\langle w' \rangle\rangle)$
 $\langle\langle f \rangle\rangle \leftarrow \sum_{i=0}^3 \langle\langle b_{i,0} \rangle\rangle + \sum_{i=s+1}^{2s} \langle\langle b_{4,i} \rangle\rangle$
 $\langle\langle c \rangle\rangle \leftarrow \text{zero_check}[6](5 - \langle\langle f \rangle\rangle)$
return $(\sum_{i=0}^{s-1} i \cdot \langle\langle b_{4,i} \rangle\rangle) + (\sum_{i=s}^{2s} (i - s) \cdot \langle\langle b_{4,i} \rangle\rangle) - \langle\langle c \rangle\rangle$
Algorithm 10: Finding the high bits of a value: `high_bits`

In our implementation, we first propagate all carries as in carry-lookahead adders [45], and only afterwards compute the vectors $\mathbf{p}_i$. We use binomial tuples [91] to compute the carries in a single round of communication. In this way, the main loop only requires two communication rounds, instead of $n - 1$. The binomial tuples will not become too large, because we use $n = 5$.

4.6 Distributed HighBits decomposition

The computation of the high bits of a secret-shared value is performed by the `high_bits` protocol, given in Alg. 10. The algorithm begins by shifting w by $\alpha/2 - 1$. Indeed, according to the definition of high bits, we have $w^H = \lfloor w'/\alpha \rfloor$, unless $w' = q - 1$, in which case $w^H = 0$. We continue by splitting w' (or $w' + q$) into five digits, let us call these digits $d_0, \dots, d_4$, their characteristic vectors are $\mathbf{b}_0, \dots, \mathbf{b}_4$.

Note that $r_0 \cdots r_3 = \alpha$. Hence, if the digits correspond to w' , then $\lfloor w'/\alpha \rfloor$ is “basically” d_4 . If the digits correspond to $w' + q$, then we have added the digits of q to the digits of w' to obtain $d_0, \dots, d_4$; the digits of q are $(1, 0, 0, 0, s)$. To handle both cases, we should return d_4 if $d_4 < s$, and return $d_4 - s$ otherwise. This can be computed from $\mathbf{b}_4$ by finding its inner product with the constant vector $(0, 1, \dots, s - 1, 0, 1, 2, \dots)$.

There are exceptions, though. One of them is $w' = q - 1$, in which case the digits are $(0, 0, 0, 0, s)$. We see that this is actually not an exception; we would return $d_4 - s = 0$ as we should. A true exception is that if the digits happen to be $(0, 0, 0, 0, s')$, where $s' > s$, then we should not return $s' - s$, but $s' - s - 1$. We check this by adding up the bits indicating this situation into f , and checking that all these bits are 1 (i.e. $5 - f$ is zero).

We point out that the feasibility of Alg. 10 very much relies on the *smoothness* of α : we have managed to represent α as the product of a few numbers, none of which are too big to have a vector of private values of this length. This smoothness is not implied by the requirements of ML-DSA: while $q - 1$ is required to be divisible by a large power of 2 to enable NTT, the construction of ML-DSA puts no further requirements on the odd prime factors of $q - 1$.

Par: ML-DSA parameters $k, \ell, \alpha, \gamma_1$
Input: $\mathbf{A}$
Output: commitment $\langle\langle \mathbf{w}^H \rangle\rangle$, ephemeral secret $\langle\langle \mathbf{y} \rangle\rangle$
CR: Small error $\langle\langle \mathbf{e}_w \rangle\rangle$, where $\mathbf{e}_w \in S_\eta^k$

```

1 foreach  $i \in [\ell], t \in [256], u \in [(\log \gamma_1) + 1]$  do
2   |  $\langle\langle b_{i,t,u} \rangle\rangle \leftarrow \text{randbit}()$ 
3  $\langle\langle \mathbf{y} \rangle\rangle \leftarrow \left( \sum_{t=0}^{255} (-\gamma_1 + \sum_{u=0}^{\log \gamma_1} 2^u \cdot \langle\langle b_{i-1,t,u} \rangle\rangle) \cdot X^t \right)_{i=1}^\ell$ 
4  $\langle\langle \mathbf{w} \rangle\rangle = \left( \sum_{t=0}^{255} \langle\langle w_{i-1,t} \rangle\rangle \cdot X^t \right)_{i=1}^k \leftarrow \mathbf{A} \cdot \langle\langle \mathbf{y} \rangle\rangle + \langle\langle \mathbf{e}_w \rangle\rangle$ 
5 foreach  $i \in [k], t \in [256]$  do
6   |  $\langle\langle w_{i,t}^H \rangle\rangle \leftarrow \text{high\_bits}(\langle\langle w_{i,t} \rangle\rangle)$ 
7 return  $\langle\langle \mathbf{y} \rangle\rangle, \langle\langle \mathbf{w}^H \rangle\rangle$ 

```

Algorithm 11: Computing the FSwA commitment: gcom

With the introduced algorithms, the computation of the commitment $\mathbf{w}^H$ and the secrets corresponding to it is straightforward. We record it in Alg. 11. NTT is used for the multiplication of polynomials. The vector of polynomials $\langle\langle \mathbf{w}^H \rangle\rangle$ will be declassified outside Alg. 11, the declassification is not included here for the same reason as in Alg. 7.

In Alg. 11, the parts in grey background are used to implement the modifications by Bienstock et al. [11]. We see that in contrast to the generation of $\mathbf{y}$, the small error vector $\mathbf{e}_w$ is generated by the CRP alone. This is secure even if we consider a malicious CRP (and honest P and S), because learning $\mathbf{e}_w$ for successful signing attempts does not break privacy [11]. See further discussion in App. F.

4.7 Rejection checks

With $\mathbf{w}^H$, we can compute the challenge c and use it to compute the response $\langle\langle \mathbf{z} \rangle\rangle = \langle\langle \mathbf{y} \rangle\rangle + c \cdot \langle\langle \mathbf{s}_1 \rangle\rangle$. Before releasing $\langle\langle \mathbf{z} \rangle\rangle$, we have to perform the rejection check on both $\mathbf{z}$ and $\mathbf{r}_0 = \text{LowBits}(\mathbf{w} - c \cdot \mathbf{s}_2)$. The latter passes rejection check [11, Lemma 5] iff $\mathbf{x} = \mathbf{w}^L - c \cdot \mathbf{s}_2 = \mathbf{w} - \alpha \cdot \mathbf{w}^H - c \cdot \mathbf{s}_2$ passes it.

A rejection check makes sure that the value belongs to a segment. The value and the segment can be shifted, such that the segment starts from 0, hence the rejection check is an inequality check against a constant, for which we have chosen to adapt the efficient two-party protocol (with CRP) by Attrapadung et al. [4]. Their protocol works for rings $\mathbb{Z}_{2^n}$, and has only passive security. We have managed to adapt it to rings $\mathbb{Z}_q$, and to AP-security. This turns out to be sufficient for an AA-secure ML-DSA protocol, as we explain below.

An important step in [4] is locally splitting a share $\llbracket v \rrbracket_P$ or $\llbracket v \rrbracket_S$ into smaller digits, and then thinking of these smaller digits as shares of (smaller) values and working with them. For passive security and rings $\mathbb{Z}_{2^n}$, this splitting is basically a non-operation, taking certain bits from the shares. With active security and MACs on the shares, we need the digits of the shares to obtain MACs, too. We cannot do the splitting so, that the digits of both Phone's and Server's share get the MACs, but we can give MACs to digits of one party's share. This is

Par: output modulus Q , radix $r \leq \frac{Q+1}{2}$, output length d
Input: $\langle v \rangle^{(q)} = (\llbracket v \rrbracket^{(q)}, \llbracket V \rrbracket^{(q)})$
Output: $\langle \mathbf{p} \rangle^{(Q)}$, s.t. $\text{split}^{-1}(\llbracket \mathbf{p} \rrbracket_{\mathbf{P}}^{(Q)} + \llbracket \mathbf{p} \rrbracket_{\mathbf{S}}^{(Q)}) = v \pmod{q}$
CR: $m \in \mathbb{Z}_q$ to $\mathbf{P}$. $\llbracket M \rrbracket^{(q)} = \llbracket m \cdot \Delta_{\mathbf{S}}^{(q)} \rrbracket^{(q)}$ and $\llbracket \mathbf{r} \rrbracket^{(Q)} = \llbracket \text{split}(m) \cdot \Delta_{\mathbf{S}}^{(Q)} \rrbracket^{(Q)}$
1 $\mathbf{P}$: $w \leftarrow \llbracket v \rrbracket_{\mathbf{P}}^{(q)} - m$ and $M_w \leftarrow \llbracket V \rrbracket_{\mathbf{P}}^{(q)} - \llbracket M \rrbracket_{\mathbf{P}}^{(q)}$
2 $\mathbf{P} \rightarrow \mathbf{S}$: w, M_w
3 $\mathbf{S}$: **stop** if $M_w \neq w \cdot \Delta_{\mathbf{S}}^{(q)} - (\llbracket V \rrbracket_{\mathbf{S}}^{(q)} - \llbracket M \rrbracket_{\mathbf{S}}^{(q)})$
4 $\mathbf{S}$: $\langle \mathbf{p} \rangle_{\mathbf{S}}^{(Q)} \leftarrow (\text{split}(\llbracket v \rrbracket_{\mathbf{S}}^{(q)} + w) \bmod q), \llbracket \mathbf{r} \rrbracket_{\mathbf{S}}^{(Q)}$
5 $\mathbf{P}$: $\langle \mathbf{p} \rangle_{\mathbf{P}}^{(Q)} \leftarrow (\text{split}(m), \llbracket \mathbf{r} \rrbracket_{\mathbf{P}}^{(Q)})$
6 **return** $\langle \mathbf{p} \rangle^{(Q)}$

Algorithm 12: Asymmetric reshare to parts: `asymm_rp`

done by the protocol `asymm_rp`, given in Alg. 12. Similarly to Alg. 9, it takes a radix basis as a parameter. To simplify this, we let all radices be equal to r . Let $\text{split}[r, d](v)$ return the length- d vector of digits of the value v . Its inverse is $\text{split}^{-1}[r, d](\mathbf{v}) = \sum_{i=0}^{d-1} r^i \cdot v_i$.

The input of the Alg. 12 is a value $\langle v \rangle = (\llbracket v \rrbracket, \llbracket \llbracket v \rrbracket_{\mathbf{P}} \cdot \Delta_{\mathbf{S}} \rrbracket)$, where we have denoted the second component with $\llbracket V \rrbracket$. The correlated randomness contains a random m known only to $\mathbf{P}$, and Server's MACs on both m and $\text{split}(m)$, denoted M and $\mathbf{r}$, respectively. In the end result, $\text{split}(m)$ will be Phone's share, while Server adjusts its share (before splitting it) by the difference of $\llbracket v \rrbracket_{\mathbf{P}}$ and m .

The input and output modulus of Alg. 12 do not have to be the same. In fact, we need the outputs to be shared over a small modulus, in order to input them to the protocols that are analogues to [4]. In Alg. 12, we make explicit, over which modulus each value has been shared. Recall that for different moduli, the Server has different MAC keys $\Delta_{\mathbf{S}}$. Also, the radix r has to be small enough for the additions in $\llbracket \mathbf{p} \rrbracket_{\mathbf{P}} + \llbracket \mathbf{p} \rrbracket_{\mathbf{S}}$ to not overflow. In our implementation, we use $Q = 29$, $r = 15$ and $d = 6$.

The inequality protocol of Attrapadung et al. [4] computes the *overflows* of the to-be-compared private values. Given a vector $\mathbf{p} = (p_0, \dots, p_{d-1})$, where $p_i \leq 2(r-1)$ (such vector could privately be the output of Alg 12), we say that $\mathbf{p}$ *overflows* if $\sum_{i=0}^{d-1} r^i \cdot p_i \geq r^d$. The overflowing of $\langle \mathbf{p} \rangle$ is computed in two steps, first for a single digit and then for the entire vector. In order to compare $\langle v \rangle$ with C , they require the overflow bits of $\langle v \rangle$ and $\langle v \rangle - C$ (which may roll over modulo q); the comparison result is a linear combination of these bits.

After Alg. 12, the conversion of the protocol of [4] to AP-security and different moduli is relatively straightforward, see App. A for details of the algorithm `ineq`($\langle v \rangle, C$), outputting whether $v < C$ or not. In our efficient implementation of `ineq`, we use several different moduli: $Q = 29$ for digits, $N = 67$ for the overflows of single digits, and $M = 71$ for the inequality checks.

Algorithm `ineq` allows us to perform rejection checks on $\langle \mathbf{z} \rangle$ and $\langle \mathbf{x} \rangle$, as shown in Alg. 13. Line 6 invokes the *characteristic vector* protocol (Alg. 15 in App. A), converting the value $\langle s \rangle$ shared over $\mathbb{Z}_M$ into its characteristic vector $\langle \mathbf{b} \rangle$ shared over $\mathbb{Z}_2$. Alg. 13 provides only privacy, but no correctness against a

Par: ML-DSA parameters, and d, Q, N, M, r
Input: $(\sum_{t=0}^{255} \langle z_{i-1,t} \rangle \cdot X^t)_{i=1}^\ell, (\sum_{t=0}^{255} \langle x_{i-1,t} \rangle \cdot X^t)_{i=1}^k$
Output: A bit indicating whether rejection check is passed

```

1 foreach  $i \in [\ell], t \in [256]$  do
2    $\langle v_{256 \cdot i + t} \rangle^{(M)} \leftarrow \text{ineq}(\langle z_{i,t} \rangle + \gamma_1 - \beta - 1, 2(\gamma_1 - \beta) - 1)$ 
3 foreach  $i \in [k], t \in [256]$  do
4    $\langle v_{256 \cdot (i+\ell) + t} \rangle^{(M)} \leftarrow \text{ineq}(\langle x_{i,t} \rangle + \gamma_2 - \beta - 1, 2(\gamma_2 - \beta) - 1)$ 
5  $\langle s \rangle^{(M)} \leftarrow \sum_{i=0}^{\text{length}(\mathbf{v})-1} (1 - \langle v_i \rangle^{(M)})$ 
6  $\langle \mathbf{b} \rangle \leftarrow \text{ch\_vec}[M, 2](\langle s \rangle^{(M)})$ 
7 return  $\langle b_0 \rangle$ 

```

Algorithm 13: Rejection check: `rej_check`

malicious Server. Moreover, when the result of rejection check is opened, it may give Server some information on $\mathbf{z}$ and $\mathbf{x}$. Rejection check with AP-security is still useful: if the declassified result of rejection check is “OK”, then we first declassify $\mathbf{z}$ only to the Phone. The Phone verifies the signature $(\mathbf{z}, c)$ and only on success sends it to the Server. If the Server interfered with the computations and caused the rejection check result to be “OK”, but $(\mathbf{z}, c)$ does not verify, then the Phone recognizes that the Server is malicious. If the Server caused the rejection check result to be “not OK”, even though it should have been “OK”, then the Server only reduced the amount of information it would have learned: if the Server hadn’t tampered with the computations, it would have learned the entire $\mathbf{z}$ (and also $\mathbf{x}$; it can be computed from the signature and the public key) from the signature. If the Server tampered with the computations of the rejection check in a manner that didn’t change its result, then it would have received at least as much information about $\mathbf{z}$ and $\mathbf{x}$ also by not tampering. Hence the Server’s best tactic is to not tamper.

Alg. 13 invokes the `ineq` protocol (with parameters d, Q, N, M, r), and then computes the long conjunction over the results. We see that it adds up the negations of the bits, and then returns whether the result is 0. Actually, the vector $\mathbf{v}$ is much longer than the modulus M , hence the sum may wrap around. The Phone will consider this wraparound as Server’s malicious behaviour. We have chosen the modulus $M = 71$ so, that probability of the sum being at least M is less than 2^{-256} if both parties behave honestly. Alg. 13 does not declassify its output; the declassification is part of $\Pi_{\text{ML-DSA}}$.

The entire signing attempt is given in Alg. 14.

5 Security

Each of the protocols given above corresponds to a real system Π ; the system $\Pi_{\text{ML-DSA}}$ is the composition of the systems corresponding to Alg. 7 and Alg. 14. The security analysis becomes more modular when we state the ideal functionalities securely implemented by these systems.

The ideal functionalities for all AA-secure protocols above are instances of the generic functionality given in Func. 3. The functionality is parametrized by

Par: ML-DSA parameters, and d, Q, N, M, r
Input: $\mathbf{A}, \mathbf{t}, \langle \mathbf{s}_1 \rangle, \langle \mathbf{s}_2 \rangle, \mu$
Output: Signature $(\mathbf{z}, c)$ on μ , or $\perp$

```

1  $\langle \mathbf{y} \rangle, \langle \mathbf{w}^H \rangle \leftarrow \text{gcom}[k, \ell, \alpha, \gamma_1](\mathbf{A})$ 
2  $\langle \mathbf{w}^L \rangle \leftarrow \mathbf{A} \cdot \langle \mathbf{y} \rangle - \alpha \cdot \langle \mathbf{w}^H \rangle$ 
3  $c \leftarrow H(\mu, \text{declassify}(\langle \mathbf{w}^H \rangle))$ 
4  $\langle \mathbf{z} \rangle = (\sum_{t=0}^{255} \langle z_{i-1,t} \rangle \cdot X^t)_{i=1}^\ell \leftarrow \langle \mathbf{y} \rangle + c \cdot \langle \mathbf{s}_1 \rangle$ 
5  $\langle \mathbf{x} \rangle = (\sum_{t=0}^{255} \langle x_{i-1,t} \rangle \cdot X^t)_{i=1}^k \leftarrow \langle \mathbf{w}^L \rangle - c \cdot \langle \mathbf{s}_2 \rangle$ 
6 if  $\text{declassify}(\text{rej\_check}(\langle \mathbf{z} \rangle, \langle \mathbf{x} \rangle))$  then
7    $\mathbf{S} \rightarrow \mathbf{P}: \llbracket \mathbf{z} \rrbracket_{\mathbf{S}}$ 
8    $\mathbf{P}: \mathbf{z} \leftarrow \llbracket \mathbf{z} \rrbracket_{\mathbf{S}} + \llbracket \mathbf{z} \rrbracket_{\mathbf{P}}$ 
9    $\mathbf{P}: \text{stop}$  if  $\neg \text{Verify}((\mathbf{A}, \mathbf{t}), \mu, (\mathbf{z}, c))$ 
10   $\mathbf{P} \rightarrow \mathbf{S}: \mathbf{z}$ 
11  return  $(\mathbf{z}, c)$ 
12 else return  $\perp$  ;
```

Algorithm 14: A signing attempt: `sign_trilithium`

- On input $(\text{init}, \text{sid}, \Delta_{\mathbf{P}})$ from $\mathbf{P}$ and $(\text{init}, \text{sid}, \Delta_{\mathbf{S}})$ from $\mathbf{S}$: Store $\Delta_{\mathbf{P}}$ and $\Delta_{\mathbf{S}}$.
- On input $(\text{compute}, \text{sid}, \langle v^{\text{priv}} \rangle_{\mathbf{P}}, v^{\text{pub}})$ from $\mathbf{P}$ and $(\text{compute}, \text{sid}, \langle v^{\text{priv}} \rangle_{\mathbf{S}}, v^{\text{pub}})$ from $\mathbf{S}$, do the following:
 - Recover v^{priv} from the shares; check the MACs. If an honest party's MAC does not verify, then stop.
 - Generate random r of the type expected by f .
 - If $\mathbf{B}(v^{\text{priv}}, v^{\text{pub}}) = 0$, then send $(v^{\text{priv}}, v^{\text{pub}})$ to $\mathbf{S}$ and get back w . Otherwise compute $w = f(v^{\text{priv}}, v^{\text{pub}}, r)$.
 - Randomly secret-share w (using $\Delta_{\mathbf{P}}$ and $\Delta_{\mathbf{S}}$), resulting in $\langle w \rangle$.
 - If $\mathbf{X}$ is corrupted, then send $(\text{proceed?}, \text{sid}, \langle w \rangle_{\mathbf{X}})$ to $\mathbf{S}$, otherwise send $(\text{proceed?}, \text{sid})$ to $\mathbf{S}$.
 - Receive $(\text{proceed!}, \text{sid})$ from $\mathbf{S}$. Send $(\text{result}, \text{sid}, \langle w \rangle_{\bar{\mathbf{X}}})$ to all honest parties $\bar{\mathbf{X}}$.

Functionality 3: Generic AA-secure functionality for the function f

the function f it is computing, which may be probabilistic. The functionality is also parametrized by the privacy levels of its inputs: parts of the inputs are secret-shared, while others are available in the open. The outputs are always private; this is the reason we did not declassify the outputs of Alg. 7 or Alg. 11. There is another constraint on outputs, less visible in Func. 3: the outputs may not be linearly dependent on other outputs or inputs. For this reason, $\mathbf{A} \cdot \langle \mathbf{y} \rangle$ was computed in both Alg. 11 and Alg. 14, and $\langle \mathbf{t} \rangle$ was not computed in Alg. 7. Finally, a predicate $\mathbf{B}$ limiting the domain of f is a parameter of the functionality.

Not all AA-secure protocols in Sec. 4 use all features of f in Func. 3. Clearly, if a protocol is generating random values, then the corresponding f is randomized. The function f for “symmetric reshare to parts” is also randomized: it is resharing either v or $v + q$ depending on whether $r \leq v$ or not, for a random $r \in \mathbb{Z}_q$. The predicate $\mathbf{B}$ is materially used only in the functionality describing the “zero check” (Alg. 8); for all others, $\mathbf{B} \equiv 1$.

- On input $(\text{init}, \text{sid})$ from $\mathbf{P}$ and $(\text{init}, \text{sid}, \Delta_{\mathbf{S}})$ from $\mathbf{S}$: Store $\Delta_{\mathbf{S}}$.
- On input $(\text{compute}, \text{sid}, \langle v^{\text{priv}} \rangle_{\mathbf{P}}, v^{\text{pub}})$ from $\mathbf{P}$ and $(\text{compute}, \text{sid}, \langle v^{\text{priv}} \rangle_{\mathbf{S}}, v^{\text{pub}})$ from $\mathbf{S}$, do the following:
 - Recover v^{priv} from the shares; check the MAC. If $\mathbf{S}$ is honest, but the MAC does not verify, then stop.
 - Generate random r of the type expected by f .
 - If $\mathbf{S}$ is honest, then compute $w = f(v^{\text{priv}}, v^{\text{pub}}, r)$.
 - If $\mathbf{S}$ is corrupted, then randomly generate the output share $\langle w \rangle_{\mathbf{S}}$ and send it to the adversary.
 - If $\mathbf{S}$ is corrupted, then receive the description of an algorithm $\mathfrak{A}$ from the adversary, compute $w = \mathfrak{A}(\langle v^{\text{priv}} \rangle_{\mathbf{P}}, r)$; it must have the correct format.
 - Compute a sharing of w , giving $\langle w \rangle$. If $\langle w \rangle_{\mathbf{S}}$ already exists, then compute $\langle w \rangle_{\mathbf{P}}$ according to it and w and $\Delta_{\mathbf{S}}$.
 - If $\mathbf{P}$ is corrupted then send $(\text{proceed?}, \text{sid}, \langle w \rangle_{\mathbf{P}})$ to $\mathcal{S}$ and receive $(\text{proceed!}, \text{sid})$ back. Send $(\text{result}, \text{sid}, \langle w \rangle_{\mathbf{S}})$ to $\mathcal{S}$.
 - If $\mathbf{P}$ is honest then send $(\text{proceed?}, \text{sid})$ to $\mathcal{S}$ and receive $(\text{proceed!}, \text{sid})$ back. Send $(\text{result}, \text{sid}, \langle w \rangle_{\mathbf{X}})$ to any honest party $\mathbf{X}$.

Functionality 4: Generic AP-secure functionality for the function f

- On input $(\text{generate}, \text{sid})$ from both $\mathbf{P}$ and $\mathbf{S}$: sample a value v according to the distribution $\mathcal{D}$. Send $(\text{proceed?}, \text{sid}, v)$ to $\mathcal{S}$ and receive $(\text{proceed!}, \text{sid})$ back. Send $(\text{result}, \text{sid}, v)$ to all honest parties $\mathbf{X}$.

Functionality 5: Public randomness generation: $\mathcal{F}_{\text{pubrand}}^{\mathcal{D}}$

The ideal functionalities for all AP-secure protocols, *except* for Alg. 12, are instances of the generic functionality given in Func. 4. Alg. 12 is not an instance, because the shares of the outputs of that protocol are not uniformly distributed over $\mathbb{Z}_Q$, and thus cannot be described in the manner allowed by Func. 4. Its corresponding ideal functionality will be given separately.

We see that Func. 4 is parametrized similarly to Func. 3, except we have no use for the predicate $\mathbf{B}$. Note how we model the passive security (but active privacy) against $\mathbf{S}$: if $\mathbf{S}$ is corrupted then it chooses the output of the functionality on the basis of inputs, but cannot learn anything. The input to $\mathfrak{A}$ includes everything that $\mathcal{S}$ does not yet know: the values $\langle v^{\text{priv}} \rangle_{\mathbf{S}}$ and v^{pub} may be included in the description of $\mathfrak{A}$.

Protocol 1 is $\Pi_{\text{ML-DSA}}$, given in the hybrid model with ideal components $\mathcal{F}_{\text{keygen}}$, $\mathcal{F}_{\text{gcom}}^{\text{m}}$, $\mathcal{F}_{\text{rej_check}}$, $\mathcal{F}_{\text{declassify}}^{\text{AA}}$, $\mathcal{F}_{\text{declassify}}^{\text{AP}}$, and $\mathcal{F}_{\text{pubrand}}^{\mathcal{U}}$ (given in Func. 5), where $\mathcal{U}$ is the uniform distribution over matrices $\mathbf{A}$ in the public key of ML-DSA. Here $\mathcal{F}_{\text{gcom}}^{\text{m}}$ is a specialization of Func. 3, where the function f receives public $\mathbf{A}$ as input, and returns $(\mathbf{y}, \mathbf{w}^H)$, similarly to Alg. 11 (either without or with modifications of [11]). The functionality $\mathcal{F}_{\text{rej_check}}$ is a specialization of Func. 4, where the function f receives $\mathbf{z}$ and $\mathbf{x}$ as private inputs, and outputs a

- On input $(\text{keygen}, \text{sid})$ from $\mathbf{X}$ to $\mathcal{M}_\mathbf{X}$, machine $\mathcal{M}_\mathbf{X}$ generates MAC keys $\Delta_\mathbf{X}$. Sends $(\text{init}, \text{sid}, \Delta_\mathbf{X})$ to all ideal components that have to be initialized (Messages from $\mathcal{M}_\mathbf{P}$ to $\mathcal{F}_{\text{rej_check}}$ and $\mathcal{F}_{\text{declassify}}^{\text{AP}}$ do not include $\Delta_\mathbf{P}$). Machine $\mathcal{M}_\mathbf{X}$ will then
 1. send $(\text{generate}, \text{sid})$ to $\mathcal{F}_{\text{pubrand}}^{\mathcal{U}}$ and get back $(\text{result}, \text{sid}, \mathbf{A})$;
 2. send $(\text{compute}, \text{sid})$ to $\mathcal{F}_{\text{keygen}}$ and get back $(\text{result}, \text{sid}, \langle\langle \mathbf{s}_1 \rangle\rangle_\mathbf{X}, \langle\langle \mathbf{s}_2 \rangle\rangle_\mathbf{X})$;
 3. Compute $\langle\langle \mathbf{t} \rangle\rangle_\mathbf{X} \leftarrow \mathbf{A} \cdot \langle\langle \mathbf{s}_1 \rangle\rangle_\mathbf{X} + \langle\langle \mathbf{s}_2 \rangle\rangle_\mathbf{X}$;
 4. send $(\text{declassify}, \text{sid}, \langle\langle \mathbf{t} \rangle\rangle_\mathbf{X})$ to $\mathcal{F}_{\text{declassify}}^{\text{AA}}$ and get back $(\text{result}, \text{sid}, \mathbf{t})$.
 It stores the received values and sends $(\text{pkey}, \text{sid}, \mathbf{A}, \mathbf{t})$ back to $\mathbf{X}$.
- On input $(\text{sign}, \text{sid}, \mu)$ from $\mathbf{P}$ to $\mathcal{M}_\mathbf{P}$, the machine $\mathcal{M}_\mathbf{P}$ works as follows, where $\mathbf{X} \equiv \mathbf{P}$:
 1. Send $(\text{compute}, \text{sid}, \mathbf{A})$ to $\mathcal{F}_{\text{gcom}}^{\text{m}}$ and get back $(\text{result}, \text{sid}, \langle\langle \mathbf{y} \rangle\rangle_\mathbf{X}, \langle\langle \mathbf{w}^H \rangle\rangle_\mathbf{X})$
 2. Compute $\langle\langle \mathbf{w}^L \rangle\rangle_\mathbf{X} \leftarrow \mathbf{A} \cdot \langle\langle \mathbf{y} \rangle\rangle_\mathbf{X} - \alpha \cdot \langle\langle \mathbf{w}^H \rangle\rangle_\mathbf{X}$
 3. Send $(\text{declassify}, \text{sid}, \langle\langle \mathbf{w}^H \rangle\rangle_\mathbf{X})$ to $\mathcal{F}_{\text{declassify}}^{\text{AA}}$ and get back $(\text{result}, \text{sid}, \mathbf{w}^H)$
 4. Compute $c \leftarrow H(\mu, \mathbf{w}^H)$, $\langle\langle \mathbf{z} \rangle\rangle_\mathbf{X} \leftarrow \langle\langle \mathbf{y} \rangle\rangle_\mathbf{X} + c \cdot \langle\langle \mathbf{s}_1 \rangle\rangle_\mathbf{X}$,
 $\langle\langle \mathbf{x} \rangle\rangle_\mathbf{X} \leftarrow \langle\langle \mathbf{w}^L \rangle\rangle_\mathbf{X} - c \cdot \langle\langle \mathbf{s}_2 \rangle\rangle_\mathbf{X}$
 5. Obtain $\langle\langle \mathbf{z} \rangle\rangle_\mathbf{X}$ and $\langle\langle \mathbf{x} \rangle\rangle_\mathbf{X}$ from $\langle\langle \mathbf{z} \rangle\rangle_\mathbf{X}$ and $\langle\langle \mathbf{x} \rangle\rangle_\mathbf{X}$ by forgetting the shares of $\mathbf{P}$'s MACs
 6. Send $(\text{compute}, \text{sid}, \langle\langle \mathbf{z} \rangle\rangle_\mathbf{X}, \langle\langle \mathbf{x} \rangle\rangle_\mathbf{X})$ to $\mathcal{F}_{\text{rej_check}}$ and get back $(\text{result}, \text{sid}, \langle\langle b \rangle\rangle_\mathbf{X})$
 7. Send $(\text{declassify}, \text{sid}, \langle\langle b \rangle\rangle_\mathbf{X})$ to $\mathcal{F}_{\text{declassify}}^{\text{AP}}$ and get back $(\text{result}, \text{sid}, b)$
 8. If $b = 0$, send $(\text{result}, \text{sid}, \perp)$ to $\mathbf{X}$
 9. If $b = 1$, then receive $(\text{ss}, \text{sid}, \llbracket \mathbf{z} \rrbracket_\mathbf{S})$ from $\mathcal{M}_\mathbf{S}$. Recover $\mathbf{z}$. Verify the signature $(\mathbf{z}, c)$ on the message μ . If it verifies, send $(\text{sig}, \text{sid}, \mathbf{z})$ to $\mathcal{M}_\mathbf{S}$ and $(\text{signature}, \text{sid}, (\mathbf{z}, c))$ to $\mathbf{P}$. If it does not verify, then stop working.
- On input $(\text{sign}, \text{sid}, \mu)$ from $\mathbf{S}$ to $\mathcal{M}_\mathbf{S}$, the machine $\mathcal{M}_\mathbf{S}$ works as follows:
 1. Perform the steps (1)–(8) above, with $\mathbf{X} \equiv \mathbf{S}$
 9. If $b = 1$, then send $(\text{ss}, \text{sid}, \llbracket \mathbf{z} \rrbracket_\mathbf{S})$ to $\mathcal{M}_\mathbf{P}$. Receive $(\text{sig}, \text{sid}, \mathbf{z})$ from $\mathcal{M}_\mathbf{P}$. Verify the signature $(\mathbf{z}, c)$ on the message μ . If it verifies, send $(\text{signature}, \text{sid}, (\mathbf{z}, c))$ to $\mathbf{S}$. If it does not verify, then stop working.

Protocol 1: ML-DSA protocol, $\Pi_{\text{ML-DSA}}$

bit showing whether they all passed the rejection check. The functionality $\mathcal{F}_{\text{keygen}}$ is also a specialization of Func. 3 according to Alg. 7.

In the main body of the paper, let us show that $\Pi_{\text{ML-DSA}}$ is at least as secure as $\mathcal{F}_{\text{ML-DSA}}^{1, \text{m}}$. App. B contains the security proofs for its subprotocols, corresponding to other algorithms.

The system $\Pi_{\text{ML-DSA}}$ consists of two protocol machines, $\mathcal{M}_\mathbf{P}$ and $\mathcal{M}_\mathbf{S}$, and the subprotocols for the six ideal functionalities that work as described in Sec. 2.3.

To show that $\Pi_{\text{ML-DSA}}$ is at least as secure as $\mathcal{F}_{\text{ML-DSA}}$, we need to construct a simulator Sim . As the corruptions of parties are static, our simulator is basically a choice between three simulators: one for the case with not corrupted parties, one for corrupted Phone, and one for corrupted Server. The first case is trivial; the simulator internally executes both $\mathcal{M}_\mathbf{P}$ and $\mathcal{M}_\mathbf{S}$ and relays the scheduling information between $\mathcal{F}_{\text{ML-DSA}}$ and $\mathcal{A}$. Let us describe the other two simulators.

5.1 Simulator for corrupted phone

In the following, $\mathcal{A}$ expects that the machine $\mathcal{M}_P$ has been corrupted, and all six ideal functionalities have been informed of the corruption of the participant P . In the actual execution with $\mathcal{F}_{\text{ML-DSA}}$ and Sim , the backdoor messages that $\mathcal{A}$ expects to be between itself and $\mathcal{M}_P$ (to control its communication with $\mathcal{Z}$) or with the ideal functionalities (to command them on behalf of $\mathcal{M}_P$) will terminate with the simulator.

Key generation. On input $(\text{pkey}, \text{sid}, \mathbf{A}, \mathbf{t})$ from $\mathcal{F}_{\text{ML-DSA}}$, proceed as follows. Generate the MAC keys Δ_S . Notify $\mathcal{A}$ of the start of key generation; receive MAC keys Δ_P from it (as part of the initialization queries from $\mathcal{A}$ to ideal components).

Receive $\mathcal{A}$'s command $(\text{generate}, \text{sid})$ to $\mathcal{F}_{\text{pubrand}}$. Send $(\text{result}, \text{sid}, \mathbf{A})$ to $\mathcal{A}$.

Receive $\mathcal{A}$'s command $(\text{compute}, \text{sid}, \mathbf{A})$ to $\mathcal{F}_{\text{keygen}}$. Generate random $\langle\langle \mathbf{s}_1 \rangle\rangle_P$ and $\langle\langle \mathbf{s}_2 \rangle\rangle_P$. Compute $\langle\langle \mathbf{t} \rangle\rangle_P \leftarrow \mathbf{A} \cdot \langle\langle \mathbf{s}_1 \rangle\rangle_P + \langle\langle \mathbf{s}_2 \rangle\rangle_P$. Send $(\text{result}, \text{sid}, \langle\langle \mathbf{s}_1 \rangle\rangle_P, \langle\langle \mathbf{s}_2 \rangle\rangle_P)$ to $\mathcal{A}$.

Receive $\mathcal{A}$'s command $(\text{declassify}, \text{sid})$ to $\mathcal{F}_{\text{declassify}}^{\text{AA}}$. Compute $\llbracket \mathbf{t} \rrbracket_S \leftarrow \mathbf{t} - \llbracket \mathbf{t} \rrbracket_P$ and send (as if coming from $\mathcal{F}_{\text{declassify}}^{\text{AA}}$) the message $(\text{input}, \text{sid}, \llbracket \mathbf{t} \rrbracket_S, \llbracket \llbracket \mathbf{t} \rrbracket_S \cdot \Delta_P \rrbracket_S)$ to $\mathcal{A}$. Receive $(\text{data}, \text{sid}, \langle\langle \mathbf{t} \rangle\rangle_P)$ from $\mathcal{A}$. If $\mathcal{A}$ has changed $\llbracket \mathbf{t} \rrbracket_P$ or Server's MAC on it, or if $\perp$ was received, then tell $\mathcal{F}_{\text{ML-DSA}}$ to stop. Otherwise send $(\text{proceed}, \text{sid})$ to $\mathcal{F}_{\text{ML-DSA}}$.

Signing. On input $(\text{signature}, \text{sid}, \mu, \sigma, \mathbf{w}_H)$ from $\mathcal{F}_{\text{ML-DSA}}$, where the signature σ is $(\mathbf{z}, c)$, proceed as follows. The simulator finds $\mathbf{x}_0 \leftarrow \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$, and $\mathbf{x} \leftarrow \mathbf{x}_0 - \alpha \cdot \mathbf{w}^H$. The simulator randomly generates $\langle\langle \mathbf{z} \rangle\rangle_P$ and $\langle\langle \mathbf{w}^L \rangle\rangle_P$. The simulator computes

$$\begin{aligned} \bullet \langle\langle \mathbf{y} \rangle\rangle_P &\leftarrow \langle\langle \mathbf{z} \rangle\rangle_P - c \cdot \langle\langle \mathbf{s}_1 \rangle\rangle_P & \bullet \langle\langle \mathbf{w} \rangle\rangle_P &\leftarrow \mathbf{A} \cdot \langle\langle \mathbf{y} \rangle\rangle_P \\ \bullet \langle\langle \mathbf{x} \rangle\rangle_P &\leftarrow \langle\langle \mathbf{w}^L \rangle\rangle_P - c \cdot \langle\langle \mathbf{s}_2 \rangle\rangle_P & \bullet \llbracket \mathbf{z} \rrbracket_S &\leftarrow \mathbf{z} - \llbracket \mathbf{z} \rrbracket_P \\ \bullet \langle\langle \mathbf{w}^H \rangle\rangle_P &\leftarrow \alpha^{-1} \cdot (\langle\langle \mathbf{w} \rangle\rangle_P - \langle\langle \mathbf{w}^L \rangle\rangle_P) \end{aligned}$$

The simulator computes $\langle\langle \mathbf{w}^H \rangle\rangle_S = (\llbracket \mathbf{w}^H \rrbracket_S, \llbracket \mathbf{m}_S \rrbracket_S, \llbracket \mathbf{m}_P \rrbracket_S)$ from $\mathbf{w}^H$, $\langle\langle \mathbf{w}^H \rangle\rangle_P$, Δ_P and Δ_S .

Receive $\mathcal{A}$'s command $(\text{compute}, \text{sid}, \mathbf{A})$ to $\mathcal{F}_{\text{gcom}}$. Send $(\text{proceed?}, \text{sid}, \langle\langle \mathbf{y} \rangle\rangle_P, \langle\langle \mathbf{w}^H \rangle\rangle_P)$ to $\mathcal{A}$, as if coming from $\mathcal{F}_{\text{gcom}}$. (And receive $(\text{proceed!}, \text{sid})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{gcom}}$; we're not going to mention this any more).

Receive $\mathcal{A}$'s command $(\text{declassify}, \text{sid})$ to $\mathcal{F}_{\text{declassify}}^{\text{AA}}$, send $(\text{input}, \text{sid}, \llbracket \mathbf{w}^H \rrbracket_S, \llbracket \mathbf{m}_P \rrbracket_S)$ to $\mathcal{A}$ and get back $(\text{data}, \text{sid}, \langle\langle \mathbf{w}^H \rangle\rangle_P)$. Continue as in the simulation of declassification during key generation; $\mathcal{A}$ will learn $\mathbf{w}^H$.

Receive $\mathcal{A}$'s command $(\text{compute}, \text{sid}, \langle\langle \mathbf{z} \rangle\rangle_P, \langle\langle \mathbf{x} \rangle\rangle_P)$ to $\mathcal{F}_{\text{rej_check}}$. The simulator already knows the expected values of $\langle\langle \mathbf{z} \rangle\rangle_P$ and $\langle\langle \mathbf{x} \rangle\rangle_P$; if the adversary submits something else, then the simulator treats it as a failure in verifying Server's MACs and tells the ideal functionality to not output the signature. Simulator generates a random $\langle\langle b \rangle\rangle_P$ and sends it to $\mathcal{A}$, as if coming from $\mathcal{F}_{\text{rej_check}}$. Following that, the simulator expects $\mathcal{A}$'s command $(\text{declassify}, \text{sid})$ to $\mathcal{F}_{\text{declassify}}^{\text{AP}}$, followed by $\langle\langle b \rangle\rangle_P$. Perform the declassification as above; $\mathcal{A}$ learns that $b = 1$.

The simulator follows this up with $(ss, sid, \llbracket \mathbf{z} \rrbracket_S)$, presumably from $\mathcal{M}_S$ to $\mathcal{M}_P$. The simulator expects $(sig, sid, \mathbf{z})$ from $\mathcal{A}$ (as if $\mathcal{M}_P$ sent it to $\mathcal{M}_S$), and, having received this, instructs the ideal functionality to output the signature.

But if the simulator received the query $(signature, sid, M, \perp, \mathbf{w}^H)$ from $\mathcal{F}_{ML-DSA}$, then the simulator computes $c = H(M, \mathbf{w}^H)$ and creates random $\langle \mathbf{y} \rangle_P$ and $\langle \mathbf{w}^L \rangle_P$. It computes $\langle \mathbf{z} \rangle_P$, $\langle \mathbf{w} \rangle_P$, $\langle \mathbf{y} \rangle_P$, and $\langle \mathbf{w}^H \rangle_P$ as before, and again sends $\langle \mathbf{y} \rangle_P$, $\langle \mathbf{w}^L \rangle_P$, $\langle \mathbf{w}^H \rangle_P$, and $\mathbf{w}^H$ to the adversary through the same interactions as before. It again expects to get back $\langle \mathbf{z} \rangle_P$ and $\langle \mathbf{x} \rangle_P$ from the adversary, sent as inputs to $\mathcal{F}_{rej_check}$. The simulator then tells the adversary that the result of rejection checking is 0.

5.2 Simulator for corrupted server

Key generation. The key generation protocol and ideal functionality are symmetric between the Phone and the Server, except for the asymmetry of $\mathcal{F}_{declassify}^{AA}$ that forces $\mathcal{A}$ to release information sooner. Similarly, the simulator learns $(\mathbf{A}, \mathbf{t})$ from $\mathcal{F}_{ML-DSA}$, generates Δ_P , gets Δ_S from the adversary, generates $\langle \mathbf{s}_1 \rangle_S$ and $\langle \mathbf{s}_2 \rangle_S$ and sends them to $\mathcal{A}$. The simulator then expects $\mathcal{A}$ to send the command $(declassify, sid, \langle \mathbf{t} \rangle'_S)$ to $\mathcal{F}_{declassify}^{AA}$, where the share of the value and P's MAC in $\langle \mathbf{t} \rangle'_S$ are supposed to be equal to $\langle \mathbf{t} \rangle_S = \mathbf{A} \cdot \langle \mathbf{s}_1 \rangle_S + \langle \mathbf{s}_2 \rangle_S$. If not, the simulator tells $\mathcal{F}_{ML-DSA}$ to stop. Otherwise, the simulator sends $(result, sid, \mathbf{t}, (\mathbf{t} - \llbracket \mathbf{t} \rrbracket_S) \cdot \Delta_S - \mathbf{m})$ to $\mathcal{A}$, where $\mathbf{m}$ is the share of S's MAC in $\langle \mathbf{t} \rangle_S$. The simulator then tells $\mathcal{F}_{ML-DSA}$ to return the public key to P.

Successful signing. If $\mathcal{F}_{ML-DSA}$ sends $(signature, sid, \mu, (\mathbf{z}, c), \mathbf{w}^H)$, proceed as follows. The simulator finds $\mathbf{x}_0 \leftarrow \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$ and $\mathbf{x} \leftarrow \mathbf{x}_0 - \alpha \cdot \mathbf{w}^H$. It secret-shares $\mathbf{z}$ into $\langle \mathbf{z} \rangle$, using Δ_S and Δ_P to compute the MACs. The simulator randomly generates $\langle \mathbf{w}^L \rangle_S$. The simulator computes

$$\begin{aligned} \bullet \langle \mathbf{y} \rangle_S &\leftarrow \langle \mathbf{z} \rangle_S - c \cdot \langle \mathbf{s}_1 \rangle_S & \bullet \langle \mathbf{w} \rangle_S &\leftarrow \mathbf{A} \cdot \langle \mathbf{y} \rangle_S \\ \bullet \langle \mathbf{x} \rangle_S &\leftarrow \langle \mathbf{w}^L \rangle_S - c \cdot \langle \mathbf{s}_2 \rangle_S \\ \bullet \langle \mathbf{w}^H \rangle_S &\leftarrow \alpha^{-1} \cdot (\langle \mathbf{w} \rangle_S - \langle \mathbf{w}^L \rangle_S) \end{aligned}$$

The simulator also computes $\langle \mathbf{w}^H \rangle_P$ and $\langle \mathbf{x} \rangle_P$ on the basis of $\langle \mathbf{w}^H \rangle_S$, $\mathbf{w}^H$, $\langle \mathbf{x} \rangle_S$, $\mathbf{x}$, Δ_P and Δ_S .

Receive $\mathcal{A}$'s command $(compute, sid, \mathbf{A})$ to $\mathcal{F}_{gcom}$. Sends $\langle \mathbf{y} \rangle_S$ and $\langle \mathbf{w}^H \rangle_S$ to $\mathcal{A}$, as if coming from $\mathcal{F}_{gcom}$ (same steps as in Sec. 5.1).

Receive $\mathcal{A}$'s command $(declassify, sid, \langle \mathbf{w}^H \rangle_S)$ to $\mathcal{F}_{declassify}^{AA}$. Continue as in the simulation of declassification during key generation; $\mathcal{A}$ will learn $\mathbf{w}^H$.

Receive $\mathcal{A}$'s command $(compute, sid, \langle \mathbf{z} \rangle_S, \langle \mathbf{x} \rangle_S)$ to $\mathcal{F}_{rej_check}$. The simulator generates a random $\langle b \rangle_S$ and sends it back to $\mathcal{A}$. The simulator then receives the description of the algorithm $\mathfrak{A}$ from $\mathcal{A}$ and applies it to $\langle \mathbf{z} \rangle_P$ and $\langle \mathbf{x} \rangle_P$ that it generated while secret-sharing $\mathbf{z}$ and $\mathbf{x}$. The algorithm returns a boolean b . The simulator creates the share $\langle b \rangle_P$, such that together with $\langle b \rangle_S$ it would declassify to b .

Receive $\mathcal{A}$'s command $(\text{declassify}, \text{sid}, \langle b \rangle_S)$ to $\mathcal{F}_{\text{declassify}}^{\text{AP}}$. Simulates the interaction with $\mathcal{F}_{\text{declassify}}^{\text{AP}}$, making $\mathcal{A}$ learn b .

If b happened to be 0, then the run stops here, i.e. the simulator does not send $(\text{proceed}, \text{sid})$ to $\mathcal{F}_{\text{ML-DSA}}$. If $b = 1$, then the simulator expects to receive $(\text{ss}, \text{sid}, \llbracket \mathbf{z}' \rrbracket_S)$ from the adversary. If $\llbracket \mathbf{z}' \rrbracket_S$ is equal to $\llbracket \mathbf{z} \rrbracket_S$, then the simulator sends $(\text{sig}, \text{sid}, \mathbf{z})$ back to the adversary and allows $\mathcal{F}_{\text{ML-DSA}}$ to proceed. If $\llbracket \mathbf{z}' \rrbracket_S$ is different from $\llbracket \mathbf{z} \rrbracket_S$, then the run stops.

Indeed, sending a different $\llbracket \mathbf{z}' \rrbracket_S$ corresponds to changing the signature $(\mathbf{z}, c)$ into $(\mathbf{z} + \mathbf{z}^\delta, c)$, where $\mathbf{z}^\delta = \llbracket \mathbf{z}' \rrbracket_S - \llbracket \mathbf{z} \rrbracket_S$ can be computed by the simulator. The size of the individual elements of $\mathbf{z}^\delta$ is bounded by $2(\gamma_1 - \beta)$. For the changed signature to verify, the equality $(\mathbf{A} \cdot \mathbf{z})^H = (\mathbf{A} \cdot (\mathbf{z} + \mathbf{z}^\delta))^H$ must hold, or we have a collision of the hash function H . This equality implies that $\mathbf{A} \cdot \mathbf{z}^\delta \approx \mathbf{0}$, i.e. we have a short solution $\mathbf{x}$ for the task $(\mathbf{A} \mid \mathbf{I}) \cdot \mathbf{x} = 0$.

Unsuccessful signing. Input $(\text{signature}, \text{sid}, \mu, \perp, \mathbf{w}^H)$ from $\mathcal{F}_{\text{ML-DSA}}$ makes the simulator Sim to proceed as follows. It generates random $\langle \mathbf{z} \rangle_S$ and $\langle \mathbf{w}^L \rangle_S$, and computes the values $\langle \mathbf{y} \rangle_S, \langle \mathbf{w} \rangle_S, \langle \mathbf{x} \rangle_S, \langle \mathbf{w}^H \rangle_S, \langle \mathbf{w}^H \rangle_P$ as before. With these values in hand, the simulation proceeds identically to Sec. 5.2) up to the point where the adversary $\mathcal{A}$ has sent the description of the algorithm $\mathfrak{A}$ to $\mathcal{F}_{\text{rej_check}}$. The simulator now applies $\mathfrak{A}$ to such $\langle \mathbf{z} \rangle_P$ and $\langle \mathbf{x} \rangle_P$ that (1) wouldn't pass the rejection check together with $\langle \mathbf{z} \rangle_S$ and $\langle \mathbf{x} \rangle_S$, and (2) spell the message "You're in a simulation".

At this point, the algorithm $\mathfrak{A}$ "knows" that it is in an execution with $\mathcal{F}_{\text{ML-DSA}}$ and Sim , not with real system consisting of $\mathcal{M}_P$, (corrupted) $\mathcal{M}_S$, and all the smaller ideal components. But $\mathfrak{A}$ does not have the ability to communicate this finding back to $\mathcal{A}$. It may return the boolean $b = 0$, in which case the unsuccessful signing would "successfully finish". The same "successful finish" would also take place for $b = 0$ in an execution with the real system. Or, $\mathfrak{A}$ may return $b = 1$, leading to the Phone determining that the Server has cheated in both systems. A way for $\mathcal{A}$ to distinguish between the two systems is to get $b = 1$ from $\mathfrak{A}$ and then try to change $\llbracket \mathbf{z} \rrbracket_S$ so that the signature would be valid. In this case the real system would return the signature to the environment, while the ideal system would return $\perp$. But if the value of b may be wrong, then $\mathcal{A}$ does not know yet whether it is in the "successful signing" or "unsuccessful signing" scenario. Hence $\mathcal{A}$ would be able to change the signature also when $\mathcal{F}_{\text{ML-DSA}}$ has created an actual signature, the impossibility of which we have discussed above.

This discussion has already explained how the simulation for "unsuccessful signing" concludes: $\mathfrak{A}$ creates a bit b , this gets secret-shared as $\langle b \rangle$ and declassified. If $b = 0$, then the simulator tells $\mathcal{F}_{\text{ML-DSA}}$ to proceed with outputting $\perp$. If $b = 1$ then the simulator waits for the message $(\text{ss}, \text{sid}, \llbracket \mathbf{z} \rrbracket_S)$, ignores its content and tells $\mathcal{F}_{\text{ML-DSA}}$ to stop.

6 Evaluation

Category	P → S	S → P	CRP → S
ML-DSA-44	460.26 KiB	460.26 KiB	1.42 MiB
ML-DSA-65	880.11 KiB	880.11 KiB	2.66 MiB
ML-DSA-87	1852.12 KiB	1852.11 KiB	4.44 MiB

Table 1. The volume of transmitted data between the parties during ML-DSA key pair generation. The correlated randomness is transmitted to the server in a single packet stream before the protocol is run.

Category	P → S or S → P (single direction)			CR
	Commitment	RejCheck	Total	
ML-DSA-44	56.85 KiB	36.50 KiB	93.36 KiB	55.05 MiB
ML-DSA-65	73.09 KiB	49.35 KiB	122.43 KiB	71.23 MiB
ML-DSA-87	97.45 KiB	66.81 KiB	164.25 KiB	95.95 MiB

Table 2. Volume of transmitted data between client and server during a signing attempt. The presented values are the average of P → S and S → P communication.

We have implemented Trilithium in Rust for both the original and modified [11] ML-DSA, and measured its performance. All measurements were performed on Lenovo laptop with AMD Ryzen 5 PRO 5650U. Traffic control (tc) was used to add latency between the phone and the server. Our set-up of the three parties during the performance measurements imitates the intended deployment of our protocol: The Server and CRP will be located close-by (but under independent control), with a high-bandwidth and low-latency connection between them. The Phone is located elsewhere, and the latency of the connection between the Phone and the Server will be significant. As usual, the parameters of the connection between the Phone and CRP are not significant: the CR share for the Phone is generated from a single random seed. The CRP knows that seed and generates Server’s CR share so, that it matches Phone’s share.

While the communication between CRP and Phone is negligible (and the same seed can be used over many signings), the size of randomness sent by CRP to the Server is very significant. That randomness is dominated by the MAC shares, and depends on the number of MACs we have given to each value. We have chosen the numbers of MACs so, that the probability of guessing the MAC keys for a single modulus is at most 2^{-128} per try. This means 6 MACs for values shared over q , and more for smaller moduli.

The number of communicated bytes in key generation and signing is reported in Tables 1 and 2. These numbers were collected in the networking layer of our implementation. We see that they are very much in the realm of practicality. Table 3 gives the bandwidth requirements for sustained computation of signatures.

Our key generation protocol has 3 rounds, while the signing protocol has 14 rounds. We have benchmarked their execution, and report the running times in Tables 4–7. We have varied only the latency of the network, not the bandwidth, because the number of communicated bytes is small. Each reported number is the average of 100 runs.

Category	P → S	S → P	CRP → S
ML-DSA-44	404 KiB/s	387 KiB/s	234 MiB/s
ML-DSA-65	638 KiB/s	607 KiB/s	362 MiB/s
ML-DSA-87	647 KiB/s	616 KiB/s	370 MiB/s

Table 3. The bandwidth required to produce one ML-DSA signature per second. The value is computed as the bandwidth required for a single signing attempt multiplied by the expected number of repetitions.

Category	No latency + CR	30 ms + CR	100 ms + CR
ML-DSA-44	90 + 10	408 + 40	1342 + 110
ML-DSA-65	114 + 36	650 + 66	1658 + 135
ML-DSA-87	99 + 18	549 + 49	1677 + 119

Table 4. Mean time (in milliseconds) for key generation. Mean time for creating CR is given separately.

Category	CR	Commitment	RejCheck	Total w/o CR
ML-DSA-44	288	141	104	245
ML-DSA-65	430	155	155	310
ML-DSA-87	587	181	183	364

Table 5. Mean time in milliseconds required for a signing iteration with no latency.

Category	CR	Commitment	RejCheck	Total w/o CR
ML-DSA-44	479	523	418	941
ML-DSA-65	976	544	431	975
ML-DSA-87	974	531	475	1006

Table 6. Mean time in milliseconds required for 3 parallel signing iterations with 30 ms latency.

Category	CR	Commitment	RejCheck	Total w/o CR
ML-DSA-44	551	1436	1010	2446
ML-DSA-65	1042	1500	1069	2569
ML-DSA-87	1370	1562	1098	2660

Table 7. Mean time in milliseconds required for 3 parallel signing iterations with 100 ms latency.

Category	1	2	3	4	6	8
ML-DSA-44	2271	1984	1585	1631	1677	2109
ML-DSA-65	3965	2742	2837	3035	3806	4099
ML-DSA-87	3752	3249	3497	3716	4251	5077

Table 8. Mean time in milliseconds of signature creation in dependence on the number of parallel iterations with no latency.

Category	1	2	3	4	6	8
ML-DSA-44	4078	2821	2516	2580	2692	2910
ML-DSA-65	6662	5143	3886	4065	5090	4797
ML-DSA-87	6605	4262	4975	4763	4671	6188

Table 9. Mean time in milliseconds of signature creation in dependence on the number of parallel iterations with 30 ms latency.

Category	1	2	3	4	6	8
ML-DSA-44	9763	6704	5753	4830	4950	5080
ML-DSA-65	15667	9422	7558	6582	6837	6956
ML-DSA-87	11741	8211	7248	8065	7409	7656

Table 10. Mean time in milliseconds of signature creation in dependence on the number of parallel iterations with 100 ms latency.

As we saw, for higher latencies, we have considered running several signing sessions in parallel, in order to reduce the effect of rejections on latency. Based on experiments, the best trade-off was achieved when executing 3–4 sessions in parallel, during which more than 50% of signatures are produced in the first run. The averages from these experiments are reported in Tables 8–10.

For ML-DSA-44, executions with 3 parallel sessions and latency of 30 ms managed to create a signature within 1.5 seconds in more than 50% of cases. A major part of the time is spent on the computation and transmission of CR from the CRP to the server. The CR can be precomputed, reducing the signing time to around 0.75 seconds.

All the numbers above pertain to the original ML-DSA. As the protocol for the modification of [11] differs only in a single local addition (Alg. 11), the bandwidths and execution times reported above remain the same, except that the CR size (Table 2) increases by 69 KiB for ML-DSA-44 and proportionally more for other sizes.

The addition of $\mathbf{e}_w$ to $\mathbf{w}$ slightly decreases the probability of a signing attempt succeeding. We have tried to empirically estimate this change, and believe it to be ca. 0.15 percentage points. Hence the implications of Tables 8–10 for the optimal number of parallel signing attempts remain valid, too.

7 Concluding remarks

In this paper, we presented Trilithium, a new two-party protocol (with CRP assistance) which allows Phone and Server collaboratively and efficiently produce ML-DSA signatures. We believe it to be a suitable replacement for the current Smart-ID protocols. An important property of Smart-ID is its certification as a Qualified Signature Creation Device (QSCD).

It should be possible to implement and deploy Trilithium in a manner that is both certifiable as QSCD, and with reasonable (compared to Smart-ID) performance requirements on the underlying hardware. Indeed, the only communication ever between the CRP and the Phone is the agreement on a random seed. The communication between the CRP and the Server is also one-sided, although the sizes of the messages are large. E.g. Smart-ID recorded a billion transactions in Baltic states in 2023⁷; if all these had used Trilithium for ML-DSA-44 signatures, the average CRP-to-Server communication rate would have been ≈ 7.5 GB/s. While the CRP-Server-pairs could be replicated, the necessary bandwidth is still significant.

We foresee that in Trilithium deployments, the Server and the CRP will be placed physically close but organizationally separated; the service providers in this highly regulated industry should be in a good position to implement the necessary measures. For certification, both the Server and the CRP should run inside a Hardware Security Module (HSM). A HSM in the form factor of a PCI Express expansion card may be a suitable platform for the CRP; such platform would also easily support the required bandwidth towards the Server.

Trilithium can be considered as a three-party (Phone, Server, and CRP) protocol, providing security against a single malicious party (with certain cheap additional checks that can run in parallel to the main signing protocol). Indeed, according to the usual definitions of MPC functionalities, the CRP cannot profit from malicious behaviour, as it receives no outputs from the protocol. But even if the CRP learned the produced signature, the attack surface is small. The CRP may cause the signing to fail, and the failure may depend on the values held by the Phone and the Server. The protocol is thus vulnerable to a *selective failure* attack by the CRP, which we consider insignificant in practice — this failure manifests in either a failed MAC check or an invalid signature learned by the Phone, both of which make the Phone and the Server stop any further runs of the protocol. The CRP would hence learn at most a single bit of information about the private key. See App. F for further discussion.

Trilithium could be generalized to a larger number of parties and to different access structures, as long as these are supported by the underlying MPC protocol set. Indeed, the key generation and the high bits computation readily generalize; they can be re-stated to run on top of an *Arithmetic Black Box (ABB)* [37], as long as this ABB supports computations modulo q and modulo 2, and conversions between the two moduli. Our rejection checks use a two-party comparison protocol; it may need replacement in other settings.

⁷ <https://www.skidsolutions.eu/news/smart-id-expands-to-belgium/>

8 Acknowledgments

This work was funded by the Estonian Research Council under the grant number PRG1780.

References

1. Alkadri, N.A., Döttling, N., Pu, S.: Practical Lattice-Based Distributed Signatures for a Small Number of Signers. In: Pöpper, C., Batina, L. (eds.) Applied Cryptography and Network Security. pp. 376–402. Springer Nature Switzerland, Cham (2024)
2. Alperin-Sheriff, J., Apon, D.: Dimension-Preserving Reductions from LWE to LWR. Cryptology ePrint Archive, Paper 2016/589 (2016), <https://eprint.iacr.org/2016/589>
3. Alwen, J., Krenn, S., Pietrzak, K., Wichs, D.: Learning with Rounding, Revisited - New Reduction, Properties and Applications. In: Canetti, R., Garay, J.A. (eds.) Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2013. Proceedings, Part I. Lecture Notes in Computer Science, vol. 8042, pp. 57–74. Springer (2013). https://doi.org/10.1007/978-3-642-40041-4_4, https://doi.org/10.1007/978-3-642-40041-4_4
4. Attrapadung, N., Morita, H., Ohara, K., Schuldt, J.C.N., Tozawa, K.: Memory and Round-Efficient MPC Primitives in the Pre-Processing Model from Unit Vectorization. In: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security. p. 858–872. ASIA CCS ’22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3488932.3517407>, <https://doi.org/10.1145/3488932.3517407> (2025-01-02)
5. Azouaoui, M., Bronchain, O., Cassiers, G., Hoffmann, C., Kuzovkova, Y., Renes, J., Schneider, T., Schönauer, M., Standaert, F., van Vredendaal, C.: Protecting Dilithium against Leakage Revisited Sensitivity Analysis and Improved Implementations. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2023**(4), 58–79 (2023). <https://doi.org/10.46586/TCHES.V2023.I4.58-79>, <https://doi.org/10.46586/tches.v2023.i4.58-79>
6. Banerjee, A., Peikert, C., Rosen, A.: Pseudorandom Functions and Lattices. In: Pointcheval, D., Johansson, T. (eds.) Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15–19, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7237, pp. 719–737. Springer (2012). https://doi.org/10.1007/978-3-642-29011-4_42, https://doi.org/10.1007/978-3-642-29011-4_42
7. Barthe, G., Belaïd, S., Espitau, T., Fouque, P., Grégoire, B., Rossi, M., Tibouchi, M.: Masking the GLP Lattice-Based Signature Scheme at Any Order. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II. Lecture Notes in Computer Science, vol. 10821, pp. 354–384. Springer (2018). https://doi.org/10.1007/978-3-319-78375-8_12, https://doi.org/10.1007/978-3-319-78375-8_12
8. Battagliola, M., Borin, G., Meneghetti, A., Persichetti, E.: Cutting the grass: Threshold group action signature schemes. In: Oswald, E. (ed.) Topics in Cryptology – CT-RSA 2024. pp. 460–489. Springer Nature Switzerland, Cham (2024)

9. Ben-Sasson, E., Fehr, S., Ostrovsky, R.: Near-Linear Unconditionally-Secure Multiparty Computation with a Dishonest Minority. In: Safavi-Naini, R., Canetti, R. (eds.) *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference*, Santa Barbara, CA, USA, August 19-23, 2012. *Proceedings. Lecture Notes in Computer Science*, vol. 7417, pp. 663–680. Springer (2012). https://doi.org/10.1007/978-3-642-32009-5_39, https://doi.org/10.1007/978-3-642-32009-5_39
10. Bendlin, R., Damgård, I., Orlandi, C., Zakarias, S.: Semi-homomorphic encryption and multiparty computation. In: Paterson, K.G. (ed.) *Advances in Cryptology – EUROCRYPT 2011*. pp. 169–188. Springer Berlin Heidelberg, Berlin, Heidelberg (2011)
11. Bienstock, A., de Castro, L., Escudero, D., Polychroniadou, A., Takahashi, A.: Efficient, Scalable Threshold ML-DSA Signatures: An MPC Approach. *Cryptology ePrint Archive*, Paper 2025/1163 (2025), <https://eprint.iacr.org/2025/1163>
12. Bogdanov, A., Guo, S., Masny, D., Richelson, S., Rosen, A.: On the Hardness of Learning with Rounding over Small Modulus. In: Kushilevitz, E., Malkin, T. (eds.) *Theory of Cryptography - 13th International Conference, TCC 2016-A*, Tel Aviv, Israel, January 10-13, 2016, *Proceedings, Part I. Lecture Notes in Computer Science*, vol. 9562, pp. 209–224. Springer (2016). https://doi.org/10.1007/978-3-662-49096-9_9, https://doi.org/10.1007/978-3-662-49096-9_9
13. Bogdanov, D., Laud, P., Laur, S., Pullonen, P.: From input private to universally composable secure multi-party computation primitives. In: *IEEE 27th Computer Security Foundations Symposium, CSF 2014*, Vienna, Austria, 19-22 July, 2014. pp. 184–198. IEEE Computer Society (2014). <https://doi.org/10.1109/CSF.2014.21>, <https://doi.org/10.1109/CSF.2014.21>
14. Bogdanov, D., Niitsoo, M., Toft, T., Willemson, J.: High-performance secure multiparty computation for data mining applications. *Int. J. Inf. Sec.* **11**(6), 403–418 (2012). <https://doi.org/10.1007/S10207-012-0177-2>, <https://doi.org/10.1007/s10207-012-0177-2>
15. Borin, G., Celi, S., del Pino, R., Espitau, T., Niot, G., Prest, T.: Threshold Signatures Reloaded: ML-DSA and Enhanced Raccoon with Identifiable Aborts. *Cryptology ePrint Archive*, Paper 2025/1166 (2025), <https://eprint.iacr.org/2025/1166>
16. Boschini, C., Takahashi, A., Tibouchi, M.: MuSig-L: Lattice-Based Multi-signature with Single-Round Online Phase. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022*. pp. 276–305. Springer Nature Switzerland, Cham (2022)
17. Buldas, A., Kalu, A., Laud, P., Oruaas, M.: Server-supported RSA signatures for mobile devices. In: Foley, S.N., Gollmann, D., Snekenes, E. (eds.) *Computer Security – ESORICS 2017*. pp. 315–333. Springer International Publishing, Cham (2017)
18. Camenisch, J., Drijvers, M., Gagliardoni, T., Lehmann, A., Neven, G.: The Wonderful World of Global Random Oracles. In: Nielsen, J.B., Rijmen, V. (eds.) *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Tel Aviv, Israel, April 29 - May 3, 2018 *Proceedings, Part I. Lecture Notes in Computer Science*, vol. 10820, pp. 280–312. Springer (2018). https://doi.org/10.1007/978-3-319-78381-9_11, https://doi.org/10.1007/978-3-319-78381-9_11
19. Camenisch, J., Lehmann, A., Neven, G., Samelin, K.: Virtual smart cards: How to sign with a password and a server. In: Zikas, V., De Prisco, R. (eds.) *Security*

- and Cryptography for Networks. pp. 353–371. Springer International Publishing, Cham (2016)
20. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: 42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14–17 October 2001, Las Vegas, Nevada, USA. pp. 136–145. IEEE Computer Society (2001). <https://doi.org/10.1109/SFCS.2001.959888>
 21. Canetti, R.: Universally Composable Security. *J. ACM* **67**(5), 28:1–28:94 (2020). <https://doi.org/10.1145/3402457>, <https://doi.org/10.1145/3402457>
 22. Canetti, R., Dodis, Y., Pass, R., Walfish, S.: Universally Composable Security with Global Setup. In: Vadhan, S.P. (ed.) *Theory of Cryptography, 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21–24, 2007, Proceedings. Lecture Notes in Computer Science*, vol. 4392, pp. 61–85. Springer (2007). https://doi.org/10.1007/978-3-540-70936-7_4, https://doi.org/10.1007/978-3-540-70936-7_4
 23. Canetti, R., Gennaro, R., Goldfeder, S., Makriyannis, N., Peled, U.: Uc non-interactive, proactive, threshold ecdsa with identifiable aborts. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. p. 1769–1787. CCS ’20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3372297.3423367>, <https://doi.org/10.1145/3372297.3423367>
 24. Canetti, R., Lindell, Y., Ostrovsky, R., Sahai, A.: Universally composable two-party and multi-party secure computation. In: Reif, J.H. (ed.) *Proceedings on 34th Annual ACM Symposium on Theory of Computing*, May 19–21, 2002, Montréal, Québec, Canada. pp. 494–503. ACM (2002). <https://doi.org/10.1145/509907.509980>, <https://doi.org/10.1145/509907.509980>
 25. Castagnos, G., Catalano, D., Laguillaumie, F., Savasta, F., Tucker, I.: Two-party ecdsa from hash proof systems and efficient instantiations. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019*. pp. 191–221. Springer International Publishing, Cham (2019)
 26. Castagnos, G., Catalano, D., Laguillaumie, F., Savasta, F., Tucker, I.: Bandwidth-efficient threshold ec-dsa. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *Public-Key Cryptography – PKC 2020*. pp. 266–296. Springer International Publishing, Cham (2020)
 27. Celi, S., Escudero, D., Niot, G.: Share the mayo: Thresholdizing mayo. In: Niederhagen, R., Saarinen, M.J.O. (eds.) *Post-Quantum Cryptography*. pp. 165–198. Springer Nature Switzerland, Cham (2025)
 28. Chaum, D., Crépeau, C., Damgård, I.: Multiparty unconditionally secure protocols (extended abstract). In: Simon, J. (ed.) *Proceedings of the 20th Annual ACM Symposium on Theory of Computing*, May 2–4, 1988, Chicago, Illinois, USA. pp. 11–19. ACM (1988). <https://doi.org/10.1145/62212.62214>, <https://doi.org/10.1145/62212.62214>
 29. Chen, M., Doerner, J., Kondi, Y., Lee, E., Rosefield, S., Shelat, A., Cohen, R.: Multiparty generation of an rsa modulus. *Journal of Cryptology* **35**(2), 12 (Mar 2022). <https://doi.org/10.1007/s00145-021-09395-y>, <https://doi.org/10.1007/s00145-021-09395-y>
 30. Coron, J., Gérard, F., Lepoint, T., Trannoy, M., Zeitoun, R.: Improved High-Order Masked Generation of Masking Vector and Rejection Sampling in Dilithium. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2024**(4), 335–354 (2024). <https://doi.org/10.46586/TCHES.V2024.I4.335-354>, <https://doi.org/10.46586/tches.v2024.i4.335-354>

31. Coron, J., Gérard, F., Trannoy, M., Zeitoun, R.: Improved Gadgets for the High-Order Masking of Dilithium. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2023**(4), 110–145 (2023). <https://doi.org/10.46586/TCHES.V2023.I4.110-145>, <https://doi.org/10.46586/tches.v2023.i4.110-145>
32. Cozzo, D., Smart, N.P.: Sharing the LUOV: Threshold Post-quantum Signatures. In: Albrecht, M. (ed.) *Cryptography and Coding – 17th IMA International Conference, IMACC 2019, Oxford, UK, December 16–18, 2019, Proceedings*. *Lecture Notes in Computer Science*, vol. 11929, pp. 128–153. Springer (2019). https://doi.org/10.1007/978-3-030-35199-1_7
33. Cramer, R., Damgård, I., Escudero, D., Scholl, P., Xing, C.: SpdF_{2^k} : Efficient MPC mod 2^k for dishonest majority. In: Shacham, H., Boldyreva, A. (eds.) *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II*. *Lecture Notes in Computer Science*, vol. 10992, pp. 769–798. Springer (2018). https://doi.org/10.1007/978-3-319-96881-0_26, https://doi.org/10.1007/978-3-319-96881-0_26
34. D’Alconzo, G., Flamini, A., Meneghetti, A., Signorini, E.: A framework for group action-based multi-signatures and applications to LESS, MEDS, and ALTEQ. *Cryptology ePrint Archive*, Paper 2024/1691 (2024), <https://eprint.iacr.org/2024/1691>
35. Damgård, I., Fitzi, M., Kiltz, E., Nielsen, J.B., Toft, T.: Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In: Halevi, S., Rabin, T. (eds.) *Theory of Cryptography, Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4–7, 2006, Proceedings*. *Lecture Notes in Computer Science*, vol. 3876, pp. 285–304. Springer (2006). https://doi.org/10.1007/11681878_15, https://doi.org/10.1007/11681878_15
36. Damgård, I., Mikkelsen, G.L., Skeltved, T.: On the security of distributed multi-prime rsa. In: Lee, J., Kim, J. (eds.) *Information Security and Cryptology - ICISC 2014*. pp. 18–33. Springer International Publishing, Cham (2015)
37. Damgård, I., Nielsen, J.B.: Universally composable efficient multiparty computation from threshold homomorphic encryption. In: Boneh, D. (ed.) *Advances in Cryptology - CRYPTO 2003*. pp. 247–264. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
38. Damgård, I., Orlandi, C., Takahashi, A., Tibouchi, M.: Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. *Journal of Cryptology* **35**(2), 14 (Apr 2022). <https://doi.org/10.1007/s00145-022-09425-3>
39. Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) *Advances in Cryptology – CRYPTO 2012*. pp. 643–662. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
40. D’Anvers, J., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Module-LWR Based Key Exchange, CPA-Secure Encryption and CCA-Secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) *Progress in Cryptology - AFRICACRYPT 2018 - 10th International Conference on Cryptology in Africa, Marrakesh, Morocco, May 7–9, 2018, Proceedings*. *Lecture Notes in Computer Science*, vol. 10831, pp. 282–305. Springer (2018). https://doi.org/10.1007/978-3-319-89339-6_16, https://doi.org/10.1007/978-3-319-89339-6_16
41. Devevey, J., Fallahpour, P., Passelègue, A., Stehlé, D.: A detailed analysis of fiat-shamir with aborts. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in*

- Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V. Lecture Notes in Computer Science, vol. 14085, pp. 327–357. Springer (2023). https://doi.org/10.1007/978-3-031-38554-4_11, https://doi.org/10.1007/978-3-031-38554-4_11
42. Doerner, J., Kondi, Y., Lee, E., Shelat, A.: Threshold ecDSA from ecDSA assumptions: The multiparty case. In: 2019 IEEE Symposium on Security and Privacy (SP). pp. 1051–1066 (2019). <https://doi.org/10.1109/SP.2019.00024>
 43. Doerner, J., Kondi, Y., Lee, E., Shelat, A.: Secure two-party threshold ecDSA from ecDSA assumptions. In: 2018 IEEE Symposium on Security and Privacy (SP). pp. 980–997. IEEE (2018)
 44. Doerner, J., Kondi, Y., Lee, E., Shelat, A.: Threshold ecDSA in three rounds. In: 2024 IEEE Symposium on Security and Privacy (SP). pp. 3053–3071 (2024). <https://doi.org/10.1109/SP54263.2024.00178>
 45. Doran, R.W.: Variants of an improved carry look-ahead adder. IEEE Trans. Computers **37**(9), 1110–1113 (1988). <https://doi.org/10.1109/12.2261>, <https://doi.org/10.1109/12.2261>
 46. Drake, J., Khovratovich, D., Kudinov, M., Wagner, B.: Hash-based multi-signatures for post-quantum ethereum. IACR Communications in Cryptology **2**(1) (2025). <https://doi.org/10.62056/ae7qjp10>
 47. Drijvers, M., Edalatnejad, K., Ford, B., Kiltz, E., Loss, J., Neven, G., Stepanovs, I.: On the security of two-round multi-signatures. In: 2019 IEEE Symposium on Security and Privacy (SP). pp. 1084–1101 (2019). <https://doi.org/10.1109/SP.2019.00050>
 48. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2018**(1), 238–268 (2018). <https://doi.org/10.13154/tches.v2018.i1.238-268>
 49. Escudero, D., Ghosh, S., Keller, M., Rachuri, R., Scholl, P.: Improved primitives for MPC over mixed arithmetic-binary circuits. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 823–852. Springer (2020). https://doi.org/10.1007/978-3-030-56880-1_29, https://doi.org/10.1007/978-3-030-56880-1_29
 50. Escudero, D., Ghosh, S., Keller, M., Rachuri, R., Scholl, P.: Improved Primitives for MPC over Mixed Arithmetic-Binary Circuits. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 823–852. Springer (2020). https://doi.org/10.1007/978-3-030-56880-1_29, https://doi.org/10.1007/978-3-030-56880-1_29
 51. Feng, Q., Yang, K., Zhang, K., Wang, X., Yu, Y., Xie, X.: Stateless deterministic multi-party EdDSA signatures with low communication. Cryptology ePrint Archive, Paper 2024/358 (2024), <https://eprint.iacr.org/2024/358>
 52. Fleischhacker, N., Herold, G., Simkin, M., Zhang, Z.: Chipmunk: Better Synchronized Multi-Signatures from Lattices. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. p. 386–400. CCS '23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3576915.3623219>

53. Garillot, F., Kondi, Y., Mohassel, P., Nikolaenko, V.: Threshold schnorr with stateless deterministic signing from standard assumptions. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021*. pp. 127–156. Springer International Publishing, Cham (2021)
54. Gennaro, R., Goldfeder, S., Narayanan, A.: Threshold-optimal dsa/ecdsa signatures and an application to bitcoin wallet security. In: Manulis, M., Sadeghi, A.R., Schneider, S. (eds.) *Applied Cryptography and Network Security*. pp. 156–174. Springer International Publishing, Cham (2016)
55. Grilo, A.B., Hövelmanns, K., Hülsing, A., Majenz, C.: Tight Adaptive Reprogramming in the QROM. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security*, Singapore, December 6-10, 2021, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 13090, pp. 637–667. Springer (2021). https://doi.org/10.1007/978-3-030-92062-3_22, https://doi.org/10.1007/978-3-030-92062-3_22
56. Güneysu, T., Lyubashevsky, V., Pöppelmann, T.: Practical Lattice-Based Cryptography: A Signature Scheme for Embedded Systems. In: Prouff, E., Schautomont, P. (eds.) *Cryptographic Hardware and Embedded Systems - CHES 2012 - 14th International Workshop*, Leuven, Belgium, September 9-12, 2012. Proceedings. *Lecture Notes in Computer Science*, vol. 7428, pp. 530–547. Springer (2012). https://doi.org/10.1007/978-3-642-33027-8_31, https://doi.org/10.1007/978-3-642-33027-8_31
57. Gur, K.D., Katz, J., Silde, T.: Two-Round Threshold Lattice-Based Signatures from Threshold Homomorphic Encryption. In: Saarinen, M.J., Smith-Tone, D. (eds.) *Post-Quantum Cryptography*. Springer Nature Switzerland, Cham (2024)
58. Hazay, C., Mikkelsen, G.L., Rabin, T., Toft, T.: Efficient rsa key generation and threshold paillier in the two-party setting. In: Dunkelman, O. (ed.) *Topics in Cryptology – CT-RSA 2012*. pp. 313–331. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
59. Hee Cheon, J., Choe, H., Seo, J., Seong, H.: SMAUG (-T), Revisited: Timing-Secure, More Compact, Less Failure. *IEEE Access* **12**, 188386–188397 (2024). <https://doi.org/10.1109/ACCESS.2024.3511346>, <https://doi.org/10.1109/ACCESS.2024.3511346>
60. Ishai, Y., Sahai, A., Wagner, D.A.: Private Circuits: Securing Hardware against Probing Attacks. In: Boneh, D. (ed.) *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference*, Santa Barbara, California, USA, August 17-21, 2003, Proceedings. *Lecture Notes in Computer Science*, vol. 2729, pp. 463–481. Springer (2003). https://doi.org/10.1007/978-3-540-45146-4_27, https://doi.org/10.1007/978-3-540-45146-4_27
61. Jin, H., Liu, F.H., Wang, Z., Yu, Y., Gu, D.: Revisiting the Robustness of (R/M)LWR under Polynomial Moduli and its Applications. *Cryptology ePrint Archive*, Paper 2024/1696 (2024), <https://eprint.iacr.org/2024/1696>
62. Joseph, D., Misoczki, R., Manzano, M., Tricot, J., Pinuaga, F.D., Lacombe, O., Leichenauer, S., Hidary, J., Venables, P., Hansen, R.: Transitioning organizations to post-quantum cryptography. *Nature* **605**(7909), 237–243 (May 2022). <https://doi.org/10.1038/s41586-022-04623-2>, <http://dx.doi.org/10.1038/s41586-022-04623-2>
63. Katsumata, S., Reichle, M., Takemure, K.: Adaptively secure 5 round threshold signatures from mlwe/msis and dl with rewinding. In: Reyzin, L., Stebila, D. (eds.)

- Advances in Cryptology – CRYPTO 2024. pp. 459–491. Springer Nature Switzerland, Cham (2024)
64. Keller, M., Orsini, E., Scholl, P.: MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24–28, 2016. pp. 830–842. ACM (2016). <https://doi.org/10.1145/2976749.2978357>, <https://doi.org/10.1145/2976749.2978357>
 65. Khaburzaniya, I., Chalkias, K., Lewi, K., Malvai, H.: Aggregating and thresholdizing hash-based signatures using starks. In: ASIA CCS '22. p. 393–407. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3488932.3524128>, <https://doi.org/10.1145/3488932.3524128>
 66. Kiltz, E., Lyubashevsky, V., Schaffner, C.: A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III. Lecture Notes in Computer Science, vol. 10822, pp. 552–586. Springer (2018). https://doi.org/10.1007/978-3-319-78372-7_18, https://doi.org/10.1007/978-3-319-78372-7_18
 67. Komlo, C., Goldberg, I.: FROST: flexible round-optimized Schnorr threshold signatures. In: Selected Areas in Cryptography: 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21–23, 2020, Revised Selected Papers 27. pp. 34–65. Springer (2021). https://doi.org/10.1007/978-3-030-81652-0_2
 68. Komlo, C., Goldberg, I.: Arctic: Lightweight and stateless threshold schnorr signatures. Cryptology ePrint Archive, Paper 2024/466 (2024), <https://eprint.iacr.org/2024/466>
 69. Koziel, B., Gordon, S.D., Gentry, C.: Fast two-party threshold ecdsa with proactive security. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 1567–1580. CCS '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3658644.3670387>, <https://doi.org/10.1145/3658644.3670387>
 70. Krenn, S., Lorünser, T.: An Introduction to Secret Sharing. SpringerBriefs in Information Security and Cryptography, Springer Cham (2023)
 71. Lindell, Y.: Fast secure two-party ecdsa signing. Journal of Cryptology **34**, 1–38 (2021)
 72. Lindell, Y.: Simple three-round multiparty schnorr signing with full simulatability. IACR Communications in Cryptology **1**(1) (2024). <https://doi.org/10.62056/a36c015vt>
 73. Lindell, Y., Nof, A.: Fast secure multiparty ecdsa with practical distributed key generation and applications to cryptocurrency custody. In: In the 2018 ACM SIGSAC Conference. pp. 1837–1854 (10 2018). <https://doi.org/10.1145/3243734.3243788>
 74. Liu, F., Wang, Z.: Rounding in the Rings. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 296–326. Springer (2020). https://doi.org/10.1007/978-3-030-56880-1_11, https://doi.org/10.1007/978-3-030-56880-1_11
 75. Lyubashevsky, V.: Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In: Matsui, M. (ed.) Advances in Cryptology – ASIACRYPT 2009. pp. 598–616. Springer Berlin Heidelberg, Berlin, Heidelberg (2009)

76. Makri, E., Rotaru, D., Vercauteren, F., Wagh, S.: Rabbit: Efficient comparison for secure multi-party computation. In: Borisov, N., Díaz, C. (eds.) *Financial Cryptography and Data Security - 25th International Conference, FC 2021, Virtual Event, March 1-5, 2021, Revised Selected Papers, Part I. Lecture Notes in Computer Science*, vol. 12674, pp. 249–270. Springer (2021). https://doi.org/10.1007/978-3-662-64322-8_12, https://doi.org/10.1007/978-3-662-64322-8_12
77. Mera, J.M.B., Karmakar, A., Kundu, S., Verbauwhede, I.: Scabbard: a suite of efficient learning with rounding key-encapsulation mechanisms. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2021**(4), 474–509 (2021). <https://doi.org/10.46586/TCHES.V2021.I4.474-509>, <https://doi.org/10.46586/tches.v2021.i4.474-509>
78. Migliore, V., Gérard, B., Tibouchi, M., Fouque, P.: Masking Dilithium - Efficient Implementation and Side-Channel Evaluation. In: Deng, R.H., Gauthier-Umaña, V., Ochoa, M., Yung, M. (eds.) *Applied Cryptography and Network Security - 17th International Conference, ACNS 2019, Bogota, Colombia, June 5-7, 2019, Proceedings. Lecture Notes in Computer Science*, vol. 11464, pp. 344–362. Springer (2019). https://doi.org/10.1007/978-3-030-21568-2_17, https://doi.org/10.1007/978-3-030-21568-2_17
79. National Institute of Standards and Technology: SHA-3 standard:: permutation-based hash and extendable-output functions. National Institute of Standards and Technology (U.S.), Gaithersburg, MD 20899-8900 (2015). <https://doi.org/10.6028/nist.fips.202>, <http://dx.doi.org/10.6028/NIST.FIPS.202>
80. National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard. National Institute of Standards and Technology (U.S.), Gaithersburg, MD 20899-8900 (2024). <https://doi.org/10.6028/nist.fips.204>, <http://dx.doi.org/10.6028/NIST.FIPS.204>
81. Nick, J., Ruffing, T., Seurin, Y.: Musig2: Simple two-round schnorr multi-signatures. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021*. pp. 189–221. Springer International Publishing, Cham (2021)
82. Nick, J., Ruffing, T., Seurin, Y., Wuille, P.: Musig-dn: Schnorr multi-signatures with verifiably deterministic nonces. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. p. 1717–1731. CCS '20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3372297.3417236>, <https://doi.org/10.1145/3372297.3417236>
83. Nicolosi, A., Krohn, M., Dodis, Y., Eres, D.: Proactive two-party signatures for user authentication. In: *NDSS Symposium 2003* (12 2002)
84. Nielsen, J.B., Nordholt, P.S., Orlandi, C., Burra, S.S.: A new approach to practical active-secure two-party computation. In: Safavi-Naini, R., Canetti, R. (eds.) *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings. Lecture Notes in Computer Science*, vol. 7417, pp. 681–700. Springer (2012). https://doi.org/10.1007/978-3-642-32009-5_40, https://doi.org/10.1007/978-3-642-32009-5_40
85. Näther, C., Herzinger, D., Gazdag, S.L., Steghöfer, J.P., Daum, S., Loebenberger, D.: Migrating Software Systems Toward Post-Quantum Cryptography-A Systematic Literature Review. *IEEE Access* **12**, 132107–132126 (2024). <https://doi.org/10.1109/ACCESS.2024.3450306>, <https://doi.org/10.1109/ACCESS.2024.3450306>
86. Peralta, R., Brandão, L.T.: NIST First Call for Multi-Party Threshold Schemes. National Institute of Standards and Technology (U.S.), Gaithersburg, MD 20899-

- 8900 (Jan 2023). <https://doi.org/10.6028/nist.ir.8214c.ipd>, <http://dx.doi.org/10.6028/NIST.IR.8214C.ipd>
87. Pettai, M., Laud, P.: Automatic proofs of privacy of secure multi-party computation protocols against active adversaries. In: Fournet, C., Hicks, M.W., Viganò, L. (eds.) IEEE 28th Computer Security Foundations Symposium, CSF 2015, Verona, Italy, 13-17 July, 2015. pp. 75–89. IEEE Computer Society (2015). <https://doi.org/10.1109/CSF.2015.13>, <https://doi.org/10.1109/CSF.2015.13>
 88. Pino, R.D., Niot, G.: Finally! A Compact Lattice-Based Threshold Signature. In: Jager, T., Pan, J. (eds.) Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part III. Lecture Notes in Computer Science, vol. 15676, pp. 169–199. Springer (2025). https://doi.org/10.1007/978-3-031-91826-1_6, https://doi.org/10.1007/978-3-031-91826-1_6
 89. Raj, K., Ravi, P., Chia, T.K., Chattopadhyay, A.: Improved ML-DSA Hardware Implementation With First Order Masking Countermeasure. Cryptology ePrint Archive, Paper 2024/1817 (2024), <https://eprint.iacr.org/2024/1817>
 90. Regenscheid, A.: Transition to Post-Quantum Cryptography Standards. Tech. rep., National Institute of Standards and Technology (2024). <https://doi.org/10.6028/nist.ir.8547.ipd>, <http://dx.doi.org/10.6028/NIST.IR.8547.ipd>
 91. Reisert, P., Rivinius, M., Krips, T., Hasler, S., Küsters, R.: Actively secure polynomial evaluation from shared polynomial encodings. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VI. Lecture Notes in Computer Science, vol. 15489, pp. 3–35. Springer (2024). https://doi.org/10.1007/978-981-96-0938-3_1, https://doi.org/10.1007/978-981-96-0938-3_1
 92. Snetkov, N., Vakarjuk, J., Laud, P.: TOPCOAT: towards practical two-party Crystals-Dilithium. Discover Computing **27**(1), 18 (Jul 2024). <https://doi.org/10.1007/s10791-024-09449-2>
 93. Toft, T.: Primitives and Applications for Multi-party Computation. Ph.D. thesis, University of Aarhus (2007)
 94. Vakarjuk, J., Snetkov, N., Laud, P.: Identifying Obstacles of PQC Migration in E-Estonia. In: 2024 16th International Conference on Cyber Conflict: Over the Horizon (CyCon). pp. 63–81 (2024). <https://doi.org/10.23919/CyCon62501.2024.10685570>, <https://doi.org/10.23919/CyCon62501.2024.10685570>
 95. Veugen, T.: Encrypted integer division and secure comparison. Int. J. Appl. Cryptogr. **3**(2), 166–180 (2014). <https://doi.org/10.1504/IJACT.2014.062738>, <https://doi.org/10.1504/IJACT.2014.062738>
 96. Xue, H., Au, M.H., Xie, X., Yuen, T.H., Cui, H.: Efficient online-friendly two-party ecdsa signature. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. p. 558–573. CCS '21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3460120.3484803>
 97. Yao, A.C.: Protocols for secure computations (extended abstract). In: 23rd Annual Symposium on Foundations of Computer Science, Chicago, Illinois, USA, 3-5 November 1982. pp. 160–164. IEEE Computer Society (1982). <https://doi.org/10.1109/SFCS.1982.38>, <https://doi.org/10.1109/SFCS.1982.38>
 98. Yuan, B., Yang, S., Zhang, Y., Ding, N., Gu, D., Sun, S.: MD-ML: super fast privacy-preserving machine learning for malicious security with a

Par: Input modulus Q , output modulus N
Input: $\langle v \rangle^{(Q)}$
Output: $\langle \mathbf{b} \rangle^{(N)}$, s.t. $\mathbf{b}$ is the CV of v
CR: $\langle r \rangle^{(Q)}$ and $\langle \mathbf{d} \rangle^{(N)}$, s.t. $\mathbf{d}$ is the CV of r
1 $f \leftarrow \text{declassify}(\langle v \rangle^{(Q)} - \langle r \rangle^{(Q)})$
2 $\forall i \in [Q] : \langle b_i \rangle^{(N)} \leftarrow \langle d_{(i-f) \bmod Q} \rangle^{(N)}$
3 **return** $\langle \mathbf{b} \rangle^{(N)}$
Algorithm 15: Characteristic vector: `ch_vec`

Par: Input modulus Q , radix $r \leq \frac{Q+1}{2}$, output modulus N
Input: $\langle v \rangle^{(Q)}$
Output: $\langle b \rangle^{(N)}$, $\langle c \rangle^{(N)}$, indicating whether $v = r - 1$ and $v \geq r$, respectively
1 $\langle \mathbf{w} \rangle \leftarrow \text{ch_vec}[Q, N](\langle v \rangle)$
2 **return** $\langle w_{r-1} \rangle$ and $\sum_{i=r}^{Q-1} \langle w_i \rangle$
Algorithm 16: Short overflow: `short_of`

- dishonest majority. In: Balzarotti, D., Xu, W. (eds.) 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024. USENIX Association (2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/yuan>
99. Zhou, L., Wang, Z., Cui, H., Song, Q., Yu, Y.: Bicaptor: Two-round secure three-party non-linear computation without preprocessing for privacy-preserving machine learning. In: 44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023. pp. 534–551. IEEE (2023). <https://doi.org/10.1109/SP46215.2023.10179449>, <https://doi.org/10.1109/SP46215.2023.10179449>

A Inequality protocol

Having split a private value into digits $\langle \mathbf{p} \rangle^{(Q)}$ (we use Alg. 12 for this), Attrapadung et al. [4] proceed to convert these digits to characteristic vectors. The protocol for this conversion is straightforward; it is given in Alg 15. The input and output modulus of this protocol are allowed to be different.

The inequality protocol of Attrapadung et al. [4] computes the *overflows* of the to-be-compared private values. Given a vector $\mathbf{p} = (p_0, \dots, p_{d-1})$, where $p_i \leq 2(r-1)$ (such vector could privately be the output of Alg 12), we say that $\mathbf{p}$ *overflows* if $\sum_{i=0}^{d-1} r^i \cdot p_i \geq r^d$. The overflowing of $\langle \mathbf{p} \rangle$ is computed in two steps, first for a single digit (`short_of`, Alg. 16) and then for the entire vector (`long_of`, Alg. 17). See [4] or App. C.2 for explanations, e.g. for the role of m in Alg. 17.

The inequality (wrt. a constant) is given in Alg. 18. Similarly to [4], we compute the overflows of the values $\langle v \rangle$ and $\langle v \rangle - C$, where the latter may roll over modulo q . We are interested in overflows wrt. q , but Alg. 17 computes them wrt. r^d , hence we adjust Server’s share of v by adding $r^d - q$ to it. This addition happens after we have split $\langle v \rangle$ to digits $\langle \mathbf{p} \rangle$; the adjusted vector of digits is $\langle \mathbf{a} \rangle$. We obtain the shares of digits of $\langle v \rangle - C$ similarly, doing the subtraction of

Par: Length d ; input, “middle”, and output moduli Q, N, M satisfying $N > 2^d$;
 radix $r \leq (Q + 1)/2$
Input: $\langle \mathbf{p} \rangle^{(Q)}$ of length d
Output: Bit $\langle z \rangle^{(M)}$, indicating whether $\mathbf{p}$ overflows

```

1 foreach  $i \in [d]$  do
2    $(\langle b_i \rangle^{(N)}, \langle c_i \rangle^{(N)}) \leftarrow \text{short\_of}[Q, r, N](\langle p_i \rangle)$ 
3  $\langle m \rangle^{(N)} \leftarrow \sum_{i=1}^{d-1} 2^{i-1} \cdot \langle b_i \rangle + \sum_{i=0}^{d-1} 2^i \cdot \langle c_i \rangle$ 
4  $\langle \mathbf{k} \rangle^{(M)} \leftarrow \text{ch\_vec}[N, M](\langle m \rangle)$ 
5 return  $\sum_{i=2^{d-1}}^{N-1} \langle k_i \rangle$ 

```

Algorithm 17: Long overflow: long_of

Par: Length d ; “middle” ($\times 2$) and output moduli Q, N, M satisfying $N > 2^d$;
 radix $r \leq (Q + 1)/2$
Input: $\langle v \rangle^{(q)}$, $C \in \mathbb{N}$
Output: Bit $\langle z \rangle^{(M)}$ indicating whether $v < C$

```

1  $\langle \mathbf{p} \rangle^{(Q)} \leftarrow \text{asymm\_rp}[Q, r, d](\langle v \rangle)$ 
2  $S: w_S \leftarrow \text{split}^{-1}(\llbracket \mathbf{p} \rrbracket_S)$ 
3  $\langle \mathbf{a} \rangle^{(Q)} \leftarrow \langle \mathbf{p} \rangle$ ;  $\llbracket \mathbf{a} \rrbracket_S := \text{split}(w_S + r^d - q)$ 
4  $\langle \mathbf{b} \rangle^{(Q)} \leftarrow \langle \mathbf{p} \rangle$ ;  $\llbracket \mathbf{b} \rrbracket_S := \text{split}(((w_S - C) \bmod q) + r^d - q)$ 
5  $\langle c \rangle^{(M)} \leftarrow \langle 0 \rangle$ ;  $\llbracket c \rrbracket_S := w_S \geq C ? 1 : 0$ 
6  $\langle x \rangle^{(M)} \leftarrow \text{long\_of}[d, Q, N, M, r](\langle \mathbf{a} \rangle)$ 
7  $\langle y \rangle^{(M)} \leftarrow \text{long\_of}[d, Q, N, M, r](\langle \mathbf{b} \rangle)$ 
8 return  $1 + \langle x \rangle - \langle y \rangle - \langle c \rangle$ 

```

Algorithm 18: Inequality of a private value and a constant: ineq

C and addition of $(r^d - q)$ at server’s side; the result is the vector $\langle \mathbf{b} \rangle$. Note that $\langle \mathbf{a} \rangle_P = \langle \mathbf{b} \rangle_P$. See [4] or App. C.3 for the discussion of correctness of Alg. 18.

B Security proofs

For each algorithm in Sec. 4, we will now describe an ideal functionality, a real protocol for running this functionality, and possibly another ideal functionality for generating the correlated randomness. We will then show that the real protocol is a secure implementation of the ideal functionality in the hybrid model that includes the correlated randomness generation, and all functionalities for the sub-protocols invoked by the algorithm. We start with the declassification from Sec. 4.1, then give the generic protocols and precomputation functionalities, as much as these can be given, and then go through the rest of the algorithms in Sec. 4.

In the main body of the paper we have used the notation $\langle\langle v \rangle\rangle = (\llbracket v \rrbracket, \llbracket M_S \rrbracket, \llbracket M_P \rrbracket)$, where M_S and M_P are Server’s and Phone’s MACs on $\llbracket v \rrbracket$. For presenting the proofs below, let us introduce additional notation: we denote the three components of $\langle\langle v \rangle\rangle$ by ${}_v\langle\langle v \rangle\rangle$, $_{ms}\langle\langle v \rangle\rangle$, and $_{mp}\langle\langle v \rangle\rangle$ respectively. We may also write e.g. $_{mp}\langle\langle v \rangle\rangle_S$ for $\llbracket M_P \rrbracket_S$. The notation ${}_v\langle v \rangle$ and $_{ms}\langle v \rangle$ has similar meaning.

- On input $(\text{init}, \text{sid}, \Delta_X)$ from X to $\mathcal{M}_X$: store Δ_X .
- On input $(\text{declassify}, \text{sid}, \langle v \rangle_S)$ from S to $\mathcal{M}_S$, the machine $\mathcal{M}_S$ works as follows:
 - Send $(\text{msg1}, \text{sid}, \langle v \rangle_S, \langle v \rangle_S)$ to $\mathcal{M}_P$.
 - Expect back $(\text{msg2}, \text{sid}, \langle v \rangle_P, \langle v \rangle_P)$.
 - * If the message was $(\text{msg2}, \text{sid}, \perp)$, then send $(\text{result}, \text{sid}, \perp)$ to S and do not proceed further.
 - Recover $v \leftarrow \langle v \rangle_P + \langle v \rangle_S$.
 - Verify S 's MAC: check that $\langle v \rangle_P \cdot \Delta_S = \langle v \rangle_P + \langle v \rangle_S$
 - If the MAC verifies, send $(\text{result}, \text{sid}, v)$ to S .
 - If the MAC does not verify, send $(\text{result}, \text{sid}, \perp)$ to S .
- On input $(\text{declassify}, \text{sid}, \langle v \rangle_P)$ from P to $\mathcal{M}_P$, the machine $\mathcal{M}_P$ works as follows:
 - Expect $(\text{msg1}, \text{sid}, \langle v \rangle_S, \langle v \rangle_S)$ from $\mathcal{M}_S$.
 - Recover $v \leftarrow \langle v \rangle_P + \langle v \rangle_S$.
 - Verify P 's MAC: check that $\langle v \rangle_S \cdot \Delta_P = \langle v \rangle_P + \langle v \rangle_S$
 - If the MAC verifies, send $(\text{msg2}, \text{sid}, \langle v \rangle_P, \langle v \rangle_P)$ to $\mathcal{M}_S$ and $(\text{result}, \text{sid}, v)$ to P .
 - If the MAC does not verify, send $(\text{msg2}, \text{sid}, \perp)$ to $\mathcal{M}_S$ and $(\text{result}, \text{sid}, \perp)$ to P .

Protocol 2: AA declassification protocol

B.1 Declassification

Protocol for AA security. The ideal functionality for AA-secure declassification has been given in Func. 2 and the algorithm in Alg. 5. The description of the algorithm can be turned to the description of the real system consisting of machines $\mathcal{M}_P$ and $\mathcal{M}_S$; this description is given in Prot. 2. To show that Prot. 2 securely implements Func. 2, we construct the simulator. Due to our corruptions being static, the simulator executes a choice between the case where P is corrupted, and where S is corrupted (and the third, trivial case where neither party is corrupted).

Simulator for AA-secure declassification with corrupted Phone. During initialization, the simulator receives Δ_P from $\mathcal{A}$. During declassification, it receives $(\text{input}, \text{sid}, \langle v \rangle_S, \langle v \rangle_S)$ from the ideal functionality. The simulator sends $(\text{msg1}, \langle v \rangle_S, \langle v \rangle_S)$ to $\mathcal{A}$ and expects back either $(\text{msg2}, \text{sid}, \perp)$ or $(\text{msg2}, \text{sid}, \langle v \rangle_P, \langle v \rangle_P)$.

If the simulator received $\perp$ as part of the (msg2) -message, then it sends $(\text{data}, \text{sid}, \perp)$ to the ideal functionality. If the simulator received the components of $\langle v \rangle_P$, then it sends $(\text{data}, \text{sid}, \langle v \rangle_P, \langle v \rangle_P)$ to the ideal functionality. In both cases, the ideal functionality will give the same output to S as the machine $\mathcal{M}_S$ would have given in a real execution.

Simulator for AA-secure declassification with corrupted Server. During initialization, the simulator receives Δ_S from $\mathcal{A}$. During declassification,

- On input $(\text{init}, \text{sid})$ from $\mathcal{P}$ and $(\text{init}, \text{sid}, \Delta_S)$ from $\mathcal{S}$: Store Δ_S .
- If $\mathcal{P}$ is honest: on input $(\text{declassify}, \text{sid}, \langle v \rangle_P)$ from $\mathcal{P}$ and $(\text{declassify}, \text{sid}, \langle v \rangle_S)$ from $\mathcal{S}$, where $\langle v \rangle_X = (\llbracket v \rrbracket_X, \llbracket M_S \rrbracket_X)$, proceed as follows:
 1. Compute $v \leftarrow \llbracket v \rrbracket_P + \llbracket v \rrbracket_S$. Let the boolean b_S indicate whether M_S verifies. Let $v_S \leftarrow b_S ? v : \perp$.
 2. Send $(\text{result}, \text{sid}, v)$ to $\mathcal{P}$.
 3. If $\mathcal{S}$ is corrupt, send $(\text{result}, \text{sid}, v, \llbracket M_S \rrbracket_P)$ to $\mathcal{S}$.
 4. If $\mathcal{S}$ is honest, send $(\text{result}, \text{sid}, v_S)$ to $\mathcal{S}$.
- If $\mathcal{P}$ is corrupt: on input $(\text{declassify}, \text{sid})$ from $\mathcal{P}$ (i.e. $\mathcal{S}$) and $(\text{declassify}, \text{sid}, \langle v \rangle_S)$ from $\mathcal{S}$, proceed as follows:
 1. Send $(\text{input}, \text{sid}, \llbracket v \rrbracket_S)$ to $\mathcal{S}$. Get back $(\text{data}, \text{sid}, \langle v \rangle_P)$ or $(\text{data}, \text{sid}, \perp)$.
 2. Compute v, b_S, v_S as in previous case.
 3. Send $(\text{result}, \text{sid}, v_S)$ to $\mathcal{S}$.

Functionality 6: AP-secure declassification

- On input $(\text{init}, \text{sid}, \Delta_S)$ from $\mathcal{S}$ to $\mathcal{M}_S$: store Δ_S .
- On input $(\text{declassify}, \text{sid}, \langle v \rangle_S)$ from $\mathcal{S}$ to $\mathcal{M}_S$, the machine $\mathcal{M}_S$ works as follows:
 - Send $(\text{msg1}, \text{sid}, \sqrt{\langle v \rangle}_S)$ to $\mathcal{M}_P$.
 - Expect back $(\text{msg2}, \text{sid}, \langle v \rangle_P)$.
 - Recover $v \leftarrow \sqrt{\langle v \rangle}_P + \sqrt{\langle v \rangle}_S$.
 - Verify $\mathcal{S}$'s MAC: check that $\sqrt{\langle v \rangle}_P \cdot \Delta_S = \text{ms} \langle v \rangle_P + \text{ms} \langle v \rangle_S$.
 - If the MAC verifies, send $(\text{result}, \text{sid}, v)$ to $\mathcal{S}$.
 - If the MAC does not verify, send $(\text{result}, \text{sid}, \perp)$ to $\mathcal{S}$.
- On input $(\text{declassify}, \text{sid}, \langle v \rangle_P)$ from $\mathcal{P}$ to $\mathcal{M}_P$, the machine $\mathcal{M}_P$ works as follows:
 - Expect $(\text{msg1}, \text{sid}, \sqrt{\langle v \rangle}_S)$ from $\mathcal{M}_S$.
 - Recover $v \leftarrow \sqrt{\langle v \rangle}_P + \sqrt{\langle v \rangle}_S$.
 - Send $(\text{msg2}, \text{sid}, \langle v \rangle_P)$ to $\mathcal{M}_S$ and $(\text{result}, \text{sid}, v)$ to $\mathcal{P}$.

Protocol 3: AP declassification protocol

it receives $(\text{msg1}, \text{sid}, \sqrt{\langle v \rangle}_S, \text{mp} \langle v \rangle_S)$ from $\mathcal{A}$. The simulator makes up an arbitrary $\text{ms} \langle v \rangle_S$ and sends $(\text{declassify}, \text{sid}, \langle v \rangle_S)$ to the ideal functionality. The ideal functionality sends back either $(\text{result}, \text{sid}, \perp)$ or $(\text{result}, \text{sid}, v, \text{ms} \langle v \rangle_P)$.

If the simulator received $\perp$ as part of the (result) -message, then it sends $(\text{msg2}, \text{sid}, \perp)$ to $\mathcal{A}$. Otherwise, the simulator computes $\sqrt{\langle v \rangle}_P \leftarrow v - \sqrt{\langle v \rangle}_S$ and sends $(\text{msg2}, \text{sid}, \sqrt{\langle v \rangle}_P, \text{ms} \langle v \rangle_P)$ to $\mathcal{A}$.

Protocol and ideal functionality for AP security. In Sec. 4.1 we have already discussed how the ideal functionality and the protocol for AP-secure declassification would look like: they would remove some message parts and checks from Func. 2 and Alg. 5. Let us now give the description explicitly in Func. 6 and Prot. 3. We notice that the ideal functionality still ensures that if $\mathcal{S}$ is honest, then it learns either the to-be-declassified value v or nothing at all. But if $\mathcal{S}$ is corrupt, then the adversary $\mathcal{S}$ can give an arbitrary $\llbracket v \rrbracket_S$ as a part of $\mathcal{S}$'s input, and this share will be added to $\mathcal{P}$'s output.

- On input $(\text{init}, \text{sid}, \Delta_P)$ from P and $(\text{init}, \text{sid}, \Delta_S)$ from S : Store Δ_P and Δ_S .
 - For AP-secure functionalities, the message from P does not contain Δ_P .
- On input $(\text{crrreq}, \text{sid})$ from both P and S :
 - Generate the random values according to the desired distribution. Let R_P be the values meant for P , and R_S the values meant for S .
 - If no party is corrupted, then send $(\text{proceed?}, \text{sid})$ to S . If party X is corrupted, then send $(\text{cr}, \text{sid}, R_X)$ to S .
 - Wait for S to send back $(\text{proceed!}, \text{sid})$.
 - Send $(\text{cr}, \text{sid}, R_{\bar{X}})$ to any honest party $\bar{X}$.

Functionality 7: $\mathcal{F}_{\text{Alg}}^{\text{off}}$. Generic ideal functionality for precomputations in an algorithm “Alg”

Simulator for AP-secure declassification with corrupted Phone. During initialization, the simulator does not receive anything. During declassification, it receives $(\text{input}, \text{sid}, \langle v \rangle_S)$ from the ideal functionality, and sends $(\text{msg1}, \text{sid}, \langle v \rangle_S)$ to the adversary. The simulator expects to receive $(\text{msg2}, \text{sid}, \langle v \rangle_P)$ from the adversary. The simulator sends $(\text{msg2}, \text{sid}, \langle v \rangle_P)$ to the ideal functionality.

We see that the MAC checks happen the same way in the real and simulated executions. In a real execution, machine $\mathcal{M}_S$ verifies that $\langle v \rangle_P \cdot \Delta_S = \text{ms}\langle v \rangle_P + \text{ms}\langle v \rangle_S$. In a simulated execution, the ideal functionality performs the same check. The output to S is the same in both cases.

Simulator for AP-secure declassification with corrupted Server. During initialization, the simulator receives Δ_S . During declassification, it receives $(\text{msg1}, \text{sid}, \langle v \rangle_S)$ from $\mathcal{A}$. The simulator makes up an arbitrary $\text{ms}\langle v \rangle_S$ and sends $(\text{declassify}, \text{sid}, \langle v \rangle_S)$ to the ideal functionality. The ideal functionality responds with $(\text{result}, \text{sid}, v, \text{ms}\langle v \rangle_P)$. The simulator sends $(\text{msg2}, \text{sid}, \langle v \rangle_P)$ to $\mathcal{A}$. In this way, the messages that $\mathcal{A}$ receives from $\mathcal{M}_P$ in the real execution are the same as the messages it receives from the simulator in the ideal execution. Also, the output that P receives from the system is the same.

B.2 Defining precomputations

Certain algorithms in Sec. 4 (Alg. 6, Alg. 8, Alg. 9, Alg. 12, and Alg. 15) require the pre-distribution of values coming from a certain (joint) probability distribution. In the UC framework, we model the generation of such values as the availability of a certain ideal functionality, given in Func. 7. We see that it is initialized by the MAC keys. Indeed, the correlated random values may need MACs on them. The only protocol-specific part in this functionality is the actual sampling of random values.

B.3 Protocols from Algorithms

In Sec. 4, we have described our signing protocol and its subprotocols in the form of pseudocode typically used to present algorithms on top of MPC protocol

- On input $(\text{init}, \text{sid}, \Delta_X)$ from X to machine $\mathcal{M}_X$, store Δ_X and send $(\text{init}, \text{sid}, \Delta_X)$ to all ideal components as coming from X . Call $(\text{init}, \text{sid}, \Delta_X)$ for the ideal functionalities in Π_{Alg} . If Π_{Alg} is AP-secure, then there is no Δ_P .
- On input $(\text{compute}, \text{sid}, \text{inputs}_X)$ from X to $\mathcal{M}_X$, it proceeds as follows:
 1. If Alg uses correlated randomness, then send $(\text{creq}, \text{sid})$ to $\mathcal{F}_{\text{Alg}}^{\text{off}}$. Get back $\mathbf{R}_X$.
 2. Execute steps of the algorithm Alg on inputs_X and $\mathbf{R}_X$, adding to the intermediate values interm_X . In particular:
 - In order to invoke a subroutine sub : come up with a session identifier sid' in a deterministic manner, e.g. adding the name of the invoked subroutine and the line in Alg containing that invocation to sid . Send $(\text{compute}, \text{sid}', \text{args}_X)$ to $\mathcal{F}_{\text{sub}}$, where args_X contains the public, and shares of the private arguments to sub . Get back $(\text{result}, \text{sid}', \text{res}_X)$ and add it to interm_X .
 - In order to rearrange data (e.g. rotate vectors) or perform local computations: do them.
 - In order to perform linear computations on secret-shared values (including the sharing of constants): perform them on local shares, including the MAC shares.
 - In order to send a message M (only in Alg. 12): send $(\text{msg}, \text{sid}, M)$ to $\mathcal{M}_{\bar{X}}$, where $\{X, \bar{X}\} = \{P, S\}$.
 - In order to receive a message M : receive $(\text{msg}, \text{sid}, M)$ from $\mathcal{M}_{\bar{X}}$.
 3. The execution has computed res_X . Send $(\text{result}, \text{sid}, \text{res}_X)$ back to X .

Protocol 4: Generic two-party protocol from the description of an algorithm “Alg”: Π_{Alg}

sets. Each one of these algorithms could alternatively be represented as a real system in the UC framework, consisting of the machine $\mathcal{M}_P$, machine $\mathcal{M}_S$, and subprotocols $\text{IDEAL}_{\mathcal{F}}$ for zero or more ideal components $\mathcal{F}$; the description of such a real system would make explicit certain details that are not mentioned in the algorithms. Indeed, Prot. 1 is such a representation for Alg. 14.

A real system executing a protocol Π_{Alg} (given in Prot. 4) for an algorithm Alg contains $\mathcal{M}_P$, $\mathcal{M}_S$, the functionalities $\mathcal{F}_{\text{sub}}$ for all subroutines that Alg invokes (except for subroutines that involve only local computations), and $\mathcal{F}_{\text{Alg}}^{\text{off}}$ if Alg uses its own correlated randomness. Note that declassification is also realized by a functionality; protocols with AA security include $\mathcal{F}_{\text{declassify}}^{\text{AA}}$ while protocols with AP security include $\mathcal{F}_{\text{declassify}}^{\text{AP}}$.

B.4 Existing functionalities

In Sec. 4.2, we stated that we make use of certain two-party computation protocols from the literature, the description of which we do not give in this paper. We still need to give the ideal functionalities corresponding to these protocols, in order to use the UC framework to give the security proofs of the protocols that we do describe. These protocols were the following:

- Generation of a public random value modulo q : `pubrand()`. The functionality $\mathcal{F}_{\text{pubrand}}$ is given in Func. 5.
- Generation of a random bit modulo q : `randbit()`. The functionality $\mathcal{F}_{\text{randbit}}$ is an instance of Func. 3, where the function f takes no inputs and randomly returns either 0 or 1. The secret-sharing is done modulo q .
- Bit conversion from mod-2 to mod- q : `bitconv( $\langle\langle b \rangle\rangle^{(2)}$ )`. The functionality $\mathcal{F}_{\text{bitconv}}$ is an instance of Func. 3, where the function f is the identity function, inputs are shared modulo 2 and outputs are shared modulo q .
- If-then-else. The functionality $\mathcal{F}_{\text{ite}}$ is an instance of Func. 3, where the function f takes three arguments: a bit b and two values (or equal-length vectors of values) x and y , all secret-shared with the same modulus. The function f returns $b \cdot (x - y) + y$.

B.5 Randomness generation

Alg. 6, parametrized with a small prime Q , gives an algorithm to generate a secret-shared random value between 0 and $Q - 1$. It corresponds to the protocol $\Pi_{\text{gen_small}}$, which is an instance of Prot. 4. The protocol securely realizes the ideal functionality $\mathcal{F}_{\text{gen_small}}$ that is an instance of Func. 3, where the function f takes no arguments and returns a random value between 0 and $Q - 1$; the output is secret-shared modulo q . The protocol $\Pi_{\text{gen_small}}$ is realized by a system consisting of $\mathcal{M}_P$, $\mathcal{M}_S$, $\mathcal{F}_{\text{pubrand}}$, and $\mathcal{F}_{\text{gen_small}}^{\text{offl}}$ (an instance of Func. 7).

In order to show that $\Pi_{\text{gen_small}}$ is at least secure as $\mathcal{F}_{\text{gen_small}}$, we have to construct a suitable simulator. As always, we present a construction for corrupted P and another one for corrupted S.

Simulator for corrupted phone During initialization, the simulator receives the initialization message from $\mathcal{A}$, and learns Phone’s MAC keys Δ_P from it.

During computation, the simulator will be activated for a run of the protocol when $\mathcal{A}$ sends the command (`creq`, sid) to the simulator, intended for $\mathcal{F}_{\text{gen_small}}^{\text{offl}}$. The simulator reacts to this message by sending (`compute`, sid) to $\mathcal{F}_{\text{gen_small}}$. The simulator receives (`proceed?`, sid , $\langle\langle v \rangle\rangle_P$) from $\mathcal{F}_{\text{gen_small}}$.

The simulator generates a random value $r \in \mathbb{Z}_Q$ and a random length- Q vector $\langle\langle \mathbf{t} \rangle\rangle_P$, satisfying the condition $\sum_{i=0}^{Q-1} i \cdot \langle\langle t_{(i+r) \bmod Q} \rangle\rangle_P = \langle\langle v \rangle\rangle_P$. The simulator sends (`cr`, sid , $\langle\langle \mathbf{t} \rangle\rangle_P$) to $\mathcal{A}$.

The simulator receives the command (`generate`, sid) from $\mathcal{A}$, intended to $\mathcal{F}_{\text{pubrand}}$. The simulator responds that r is the generated public random value. The simulator sends (`proceed!`, sid) to $\mathcal{F}_{\text{gen_small}}$.

Simulator for corrupted server The protocol is symmetric wrt. the roles of Phone and Server. The simulator will be the same, with P and S swapped.

B.6 Key generation

Alg. 7 gives the algorithm for ML-DSA private key generation, such that the result is secret-shared. The algorithm realizes the ideal functionality $\mathcal{F}_{\text{keygen}}$ described in Sec. 5. We define the protocol Π_{keygen} an instance of Prot. 4, corresponding to Alg. 7.

The real system realizing Π_{keygen} consists of the machines $\mathcal{M}_P$ and $\mathcal{M}_S$, and the ideal functionality $\mathcal{F}_{\text{gen_small}}$. Let us construct simulators for corrupted P and for corrupted S.

Simulator for corrupted phone During initialization, the simulator receives the initialization message from $\mathcal{A}$, and learns Phone's MAC keys Δ_P from it.

During computation, the simulator receives $(\text{compute}, \text{sid}, \mathbf{A})$ from $\mathcal{A}$, indicating the start of the computation. The simulator calls $\mathcal{F}_{\text{keygen}}$ with $(\text{compute}, \text{sid}, \mathbf{A})$ and gets back $(\text{proceed?}, \text{sid}, \langle\langle \mathbf{s}_1 \rangle\rangle_P, \langle\langle \mathbf{s}_2 \rangle\rangle_P)$.

The simulator turns $\langle\langle \mathbf{s}_1 \rangle\rangle_P$ and $\langle\langle \mathbf{s}_2 \rangle\rangle_P$ into the shares of the random values that are presumably generated by $\mathcal{F}_{\text{gen_small}}$. This computation is done as follows. Let $\langle\langle r_{i,t} \rangle\rangle_P$ be one of the coefficients of one of the polynomials in either $\langle\langle \mathbf{s}_1 \rangle\rangle_P$ or $\langle\langle \mathbf{s}_2 \rangle\rangle_P$. If $\eta = 2$, then define $\langle\langle r'_{i,t} \rangle\rangle_P \leftarrow \langle\langle r_{i,t} \rangle\rangle_P + \langle\langle 2 \rangle\rangle_P$, where $\langle\langle 2 \rangle\rangle$ denotes the classification of the constant 2. If $\eta = 4$ then let $\langle\langle r'_{i,t} \rangle\rangle_P$ be a random element of $\mathbb{Z}_q$ and define $\langle\langle r''_{i,t} \rangle\rangle_P \leftarrow \langle\langle r_{i,t} \rangle\rangle_P - 3 \cdot \langle\langle r'_{i,t} \rangle\rangle_P + \langle\langle 4 \rangle\rangle_P$.

The simulator waits for $\mathcal{A}$'s commands $(\text{compute}, \text{sid}')$ to $\mathcal{F}_{\text{gen_small}}$, where the session identifier sid' determines, which coefficient of which element of $\mathbf{s}_1$ or $\mathbf{s}_2$ this invocation corresponds to (and in case of $\eta = 4$: whether it corresponds to upper or lower half of the private value). The simulator sends back $(\text{proceed?}, \text{sid}', \langle\langle r'_{i,t} \rangle\rangle_P)$ or $(\text{proceed?}, \text{sid}', \langle\langle r''_{i,t} \rangle\rangle_P)$, according to the position that $\mathcal{A}$ chose. The adversary $\mathcal{A}$ has to send the activations for all positions in $\mathbf{s}_1$ and $\mathbf{s}_2$.

Once the generations of all small random values have been done, the simulator sends $(\text{proceed!}, \text{sid})$ back to $\mathcal{F}_{\text{keygen}}$.

Simulator for corrupted server The protocol is symmetric wrt. the roles of Phone and Server. The simulator is the same as the one for corrupted Phone, with P and S swapped.

B.7 Zero equality checking

Alg. 8 gives the algorithm for checking whether a private value $\langle\langle v \rangle\rangle$ is equal to 0, with the precondition $v < B$, where B is a rather small bound. It corresponds to a protocol $\Pi_{\text{zero_check}}$, which is an instance of Prot. 4. The protocol realizes the ideal functionality $\mathcal{F}_{\text{zero_check}}$, which is an instance of Func. 3, where the function f returns 1 iff $v = 0$ (and returns 0 otherwise). The predicate $\mathbf{B}$ returns 1 if $v < B$.

The protocol $\Pi_{\text{zero_check}}$ is realized by a system consisting of machines $\mathcal{M}_P$ and $\mathcal{M}_S$, as well as the functionalities $\mathcal{F}_{\text{declassify}}^{\text{AA}}$ and $\mathcal{F}_{\text{zero_check}}^{\text{off}}$. The latter is an

instance of Func. 7. Let us present the simulators to show that $\Pi_{\text{zero_check}}$ is at least as secure as $\mathcal{F}_{\text{zero_check}}$.

Simulator for corrupted phone During initialization, the simulator receives the initialization message from $\mathcal{A}$, and learns Phone’s MAC keys Δ_P from it.

A computation starts with the simulator receiving $(\text{compute}, \text{sid}, \langle\langle v \rangle\rangle_P)$ from $\mathcal{A}$. The simulator forwards that command to $\mathcal{F}_{\text{zero_check}}$ and gets back $(\text{proceed?}, \text{sid}, \langle\langle b \rangle\rangle_P)$, where b is the result of doing the zero check. Alternatively, the simulator may receive the value v , if $v \geq B$, where B is the public bound. Let us first consider the case $v < B$.

The simulator generates a random $u \in \mathbb{Z}_q$, and random values $\langle\langle m \rangle\rangle_P$ and $\langle\langle t \rangle\rangle_P$, except that the element $\langle\langle t_{\lfloor u/(\lfloor q/B \rfloor)} \rangle\rangle_P$ is taken to be equal to $\langle\langle b \rangle\rangle_P$. The simulator expects to receive the command $(\text{creq}, \text{sid})$ from $\mathcal{A}$, intended for $\mathcal{F}_{\text{zero_check}}^{\text{off}}$, and sends back $(\text{proceed?}, \text{sid}, \langle\langle m \rangle\rangle_P, \text{actsht}_P)$.

The simulator waits $\mathcal{A}$ to send $(\text{declassify}, \text{sid})$, wanting to declassify the value $\langle\langle d \rangle\rangle_P = \langle\langle m \rangle\rangle_P + \lfloor q/B \rfloor \cdot \langle\langle v \rangle\rangle_P$. The simulator already knows $\langle\langle d \rangle\rangle_P$, because it is able to compute it itself. The simulator computes

$$\begin{aligned} - \text{ }_v \langle\langle d \rangle\rangle_S &\leftarrow u - \text{ }_v \langle\langle d \rangle\rangle_P, \text{ and} \\ - \text{ }_{\text{mp}} \langle\langle d \rangle\rangle_S &\leftarrow \text{ }_v \langle\langle d \rangle\rangle_S \cdot \Delta_P - \text{ }_{\text{mp}} \langle\langle d \rangle\rangle_P, \end{aligned}$$

and sends $(\text{input}, \text{sid}, \text{ }_v \langle\langle d \rangle\rangle_S, \text{ }_{\text{mp}} \langle\langle d \rangle\rangle_S)$ to the adversary (as if coming from $\mathcal{F}_{\text{declassify}}^{\text{AA}}$). The adversary $\mathcal{A}$ sends back $(\text{data}, \text{sid}, \langle\langle d \rangle\rangle_P)$. If received value is not equal to the value of $\langle\langle d \rangle\rangle_P$ already known to the simulator (at least for the v - and ms -parts), then the simulator tells $\mathcal{F}_{\text{zero_check}}$ to stop.

At this point, the to-be-declassified value is fixed as u , and the adversary is able to find the output share $\langle\langle b \rangle\rangle_P$.

If the simulator also received v at the time the computation was started, then it first generates a random $m \in \mathbb{Z}_q$, and computes $u \leftarrow (m + \lfloor q/B \rfloor \cdot v) \bmod q$. It continues as before, secret-sharing m and generating $\langle\langle t \rangle\rangle_P$ so that $\langle\langle t_{\lfloor u/(\lfloor q/B \rfloor)} \rangle\rangle_P = \langle\langle b \rangle\rangle_P$.

Simulator for corrupted server This is very similar to the simulator for corrupted phone. The only difference arises from the asymmetry of declassification.

The initialization of the components (during which the simulator learns Δ_S) works the same way, as does the receiving of the share $\langle\langle v \rangle\rangle_S$, the generation of the shares of correlated randomness, and the computation of $\langle\langle d \rangle\rangle_S$. At this point, the simulator (as $\mathcal{F}_{\text{declassify}}^{\text{AA}}$) receives $(\text{declassify}, \text{sid}, \langle\langle d \rangle\rangle_S)$ from $\mathcal{A}$. If $\langle\langle d \rangle\rangle_S$ coming from the adversary is not equal to the value that the simulator expects it to be (considering the “value” and “P’s MAC” components), then the simulator sends back $(\text{result}, \text{sid}, \perp)$. Otherwise the simulator computes

$$\begin{aligned} - \text{ }_v \langle\langle d \rangle\rangle_P &\leftarrow u - \text{ }_v \langle\langle d \rangle\rangle_S, \text{ and} \\ - \text{ }_{\text{ms}} \langle\langle d \rangle\rangle_P &\leftarrow \text{ }_v \langle\langle d \rangle\rangle_P \cdot \Delta_S - \text{ }_{\text{ms}} \langle\langle d \rangle\rangle_S \end{aligned}$$

and sends $(\text{result}, \text{sid}, u, \text{ }_{\text{ms}} \langle\langle d \rangle\rangle_P)$ to $\mathcal{A}$. This concludes the simulation, as the adversary now knows u and can find $\langle\langle b \rangle\rangle_S$.

B.8 Symmetric reshare to parts

Alg. 9 gives the algorithm for “splitting a private value into digits”, parametrized with the basis $\mathbf{r} = (r_0, \dots, r_{n-1})$. It corresponds to a protocol $\Pi_{\text{symm_rp}}$, an instance of Prot. 4, which is at least as secure as the ideal functionality $\mathcal{F}_{\text{symm_rp}}$, which is an instance of Func. 3, where

- the randomness $\hat{r}$ is a random element of $\{0, \dots, q-1\}$;
- the function $f(v, \hat{r})$ checks whether $v \geq \hat{r}$. If so, then f splits v into digits. If not, then f splits $v+q$ into digits. In either case, f returns the characteristic vectors of these digits.

The protocol $\Pi_{\text{symm_rp}}$ is realized by a system consisting of machines $\mathcal{M}_{\mathbf{P}}$ and $\mathcal{M}_{\mathbf{S}}$, as well as the ideal components $\mathcal{F}_{\text{declassify}}^{\text{AA}}$, $\mathcal{F}_{\text{ite}}$, $\mathcal{F}_{\text{convbit}}$ and $\mathcal{F}_{\text{symm_rp}}^{\text{off}}$. The latter is an instance of Func. 7, generating a random $s \in \mathbb{Z}_q$ and the characteristic vectors $\mathbf{d}_0, \dots, \mathbf{d}_{n-1}$ of its digits. Let us present the simulators showing that $\Pi_{\text{symm_rp}}$ indeed securely realizes $\mathcal{F}_{\text{symm_rp}}$.

Simulator for corrupted phone During initialization, the simulator receives the initialization message from $\mathcal{A}$, and learns Phone’s MAC keys $\Delta_{\mathbf{P}}$ from it.

During computation, the simulator receives the input $(\text{compute}, \text{sid}, \langle\langle w \rangle\rangle_{\mathbf{P}})$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{symm_rp}}$. The functionality responds with output shares $\langle\langle \mathbf{b}_0 \rangle\rangle_{\mathbf{P}}, \dots, \langle\langle \mathbf{b}_{n-1} \rangle\rangle_{\mathbf{P}}$ inside the (proceed?)-message.

The simulator is able to generate the values and random shares according to the probability distribution that $\mathcal{M}_{\mathbf{P}}$ would see them distributed. This is straightforward:

- $\langle\langle s \rangle\rangle_{\mathbf{P}}$ is a triple $(\langle\langle s \rangle\rangle_{\mathbf{P}}, \langle\langle s \rangle\rangle_{\mathbf{ms}}, \langle\langle s \rangle\rangle_{\mathbf{mp}})$ of random elements of $\mathbb{Z}_q$ (for the MAC shares, a vector of random elements);
- similarly, each element of the vectors $\langle\langle \mathbf{d}_i \rangle\rangle_{\mathbf{P}}$ is a triple of a random value and two vectors of random values over $\mathbb{Z}_2$;
- the additional elements of extended vectors $\langle\langle \mathbf{d}_i \rangle\rangle_{\mathbf{P}}$ are $\langle\langle 0 \rangle\rangle_{\mathbf{P}}$;
- the argument to declassification is $\langle\langle w \rangle\rangle_{\mathbf{P}} - \langle\langle s \rangle\rangle_{\mathbf{P}}$;
- y is a random element of $\mathbb{Z}_q$;
- $(z_0, \dots, z_{n-1})$ are the digits of y ;
- $\langle\langle \mathbf{p}'_i \rangle\rangle_{\mathbf{P}}$ and $\langle\langle \mathbf{p}''_i \rangle\rangle_{\mathbf{P}}$ are rotations of $\langle\langle \mathbf{d}_i \rangle\rangle_{\mathbf{P}}$ by z_i and $(z_i + 1)$ positions, respectively;
- $\langle\langle \mathbf{p}_0 \rangle\rangle_{\mathbf{P}} = \langle\langle \mathbf{p}'_0 \rangle\rangle_{\mathbf{P}}$;
- each element of the vectors $\langle\langle \mathbf{p}_i \rangle\rangle_{\mathbf{P}}$ ($i \geq 1$) is a triple of a random value and two vectors of random values over $\mathbb{Z}_2$;
- $\langle\langle c_i \rangle\rangle_{\mathbf{P}}$ ($i \geq 1$) is the sum (i.e. XOR, computed point-wise) of certain elements of $\langle\langle \mathbf{p}_i \rangle\rangle_{\mathbf{P}}$;
- arguments to convbit are sums (i.e. XORs) of certain bits in the vectors $\langle\langle \mathbf{p}_i \rangle\rangle_{\mathbf{P}}$.

The simulator does not generate these values ahead of time, but only during the simulated execution of the protocol together with the adversary $\mathcal{A}$. It starts with

wait for the $\mathcal{A}$ to ask for correlated randomness, and sending $\langle\langle s \rangle\rangle_P$ and $\langle\langle d_i \rangle\rangle_P$ to the adversary, as if coming from $\mathcal{F}_{\text{sym_rp}}^{\text{off}}$.

The adversary will invoke $\mathcal{F}_{\text{declassify}}^{\text{AA}}$ in order to obtain y . The simulator simulates declassification identically to App. B.7: there is again the share $\langle\langle y \rangle\rangle_P$ computed by the simulator, and the simulator can use it to compute server's share of the value and P's MAC, such that the declassification result would be y . The adversary sends back its own version of $\langle\langle y \rangle\rangle_P$; if it differs from simulator's version (the value and S's MAC components), then the execution is stopped.

The adversary will invoke the computation of $\mathcal{F}_{\text{ite}}$ for n times in a row. The simulator will give $\langle\langle p_i \rangle\rangle_P$ as the output shares of these invocations to $\mathcal{A}$.

The adversary will invoke $\mathcal{F}_{\text{convbit}}$ for $\sum \mathbf{r}$ times. The simulator will give it the components of $\langle\langle b_0 \rangle\rangle_P, \dots, \langle\langle b_{n-1} \rangle\rangle_P$ as results.

Simulator for corrupted server The protocol is (almost) symmetric. The simulator for corrupted server works almost the same as the simulator for corrupted phone; the changes in handling asymmetric $\mathcal{F}_{\text{declassify}}^{\text{AA}}$ are the same as in App. B.7.

B.9 Finding the high bits

Alg. 10 gives the algorithm for finding the high bits of a value $\langle\langle w \rangle\rangle$. It corresponds to a protocol $\Pi_{\text{high_bits}}$, an instance of Prot. 4, which is at least as secure as the ideal functionality $\mathcal{F}_{\text{high_bits}}$, which is an instance of Func. 3, where the function f computes the high bits of its argument. The real system implementing $\Pi_{\text{high_bits}}$ consists of machines $\mathcal{M}_P$ and $\mathcal{M}_S$, as well as ideal components $\mathcal{F}_{\text{sym_rp}}$ and $\mathcal{F}_{\text{zero_check}}$. Let us construct the simulators showing that $\Pi_{\text{high_bits}}$ is at least as secure as $\mathcal{F}_{\text{high_bits}}$.

Simulator for corrupted phone During initialization, the simulator receives the initialization message from $\mathcal{A}$, and learns Phone's MAC keys Δ_P from it.

During computation, the simulator receives the input $\langle\langle w \rangle\rangle_P$ from $\mathcal{A}$ (as the argument to the (compute)-command) and forwards it to $\mathcal{F}_{\text{high_bits}}$. The functionality $\mathcal{F}_{\text{high_bits}}$ proceeds, and tells the simulator the output share $\langle\langle v \rangle\rangle_P$.

The simulator waits for $\mathcal{A}$ to start the computation of $\mathcal{F}_{\text{sym_rp}}$ with the input share $\langle\langle w' \rangle\rangle_P$, where $\langle\langle w' \rangle\rangle_P = \langle\langle w \rangle\rangle_P + \langle\langle \alpha/2 - 1 \rangle\rangle_P$. The simulator knows what $\langle\langle w' \rangle\rangle_P$ should be. If the adversary changes either the v -component or the ms -component of $\langle\langle w' \rangle\rangle_P$, then the simulator tells $\mathcal{F}_{\text{high_bits}}$ to stop (i.e. does not tell it to proceed).

The simulator generates random vectors $\langle\langle b_0 \rangle\rangle_P, \dots, \langle\langle b_4 \rangle\rangle_P$ and sends these to the adversary as the output shares from $\mathcal{F}_{\text{sym_rp}}$. It also computes $\langle\langle g \rangle\rangle_P \leftarrow \langle\langle 5 \rangle\rangle_P - \sum_{i=0}^3 \langle\langle b_{i,0} \rangle\rangle_P - \sum_{i=s+1}^{2s} \langle\langle b_{4,i} \rangle\rangle_P$.

The simulator waits for $\mathcal{A}$ to start the computation of $\mathcal{F}_{\text{zero_check}}$ with the input share $\langle\langle g \rangle\rangle_P$. The simulator already knows the expected value of $\langle\langle g \rangle\rangle_P$. If its v - or ms -components of adversary's value differ from the values that the simulator has previously computed, then the simulator tells $\mathcal{F}_{\text{high_bits}}$ to stop.

The simulator computes $\langle\langle c \rangle\rangle_P \leftarrow \sum_{i=0}^{s-1} i \cdot \langle\langle b_{4,i} \rangle\rangle_P + \sum_{i=s}^{2s} (i-s) \cdot \langle\langle b_{4,i} \rangle\rangle_P - \langle\langle v \rangle\rangle_P$. The simulator sends $\langle\langle c \rangle\rangle_P$ to the adversary as the output share from $\mathcal{F}_{\text{zero_check}}$. In this way, the output shares of the subcomputations seen by the adversary match with the output share $\langle\langle v \rangle\rangle_P$ of the full computation of high bits.

Simulator for corrupted server The protocol is symmetric wrt. the roles of Phone and Server. The simulator is the same as the one for corrupted Phone, with P and S swapped.

B.10 Commitment computation

Alg. 11 gives the algorithm for creating the secret $\langle\langle \mathbf{y} \rangle\rangle$ and the corresponding FSwA commitment $\langle\langle \mathbf{w}^H \rangle\rangle$. We consider it as a template for the protocol Π_{gcom} (an instance of Prot. 4), parametrized with the bit $\mathbf{m}$. The inputs and outputs of the protocol Π_{gcom} are the same as those of the ideal functionality $\mathcal{F}_{\text{gcom}}$ introduced in Sec. 5.

The protocol Π_{gcom} is realized by a system consisting of the machines $\mathcal{M}_P$ and $\mathcal{M}_S$, as well as the ideal functionalities $\mathcal{F}_{\text{high_bits}}$ and $\mathcal{F}_{\text{randbit}}$. Let us present the simulators showing that Π_{gcom} is at least as secure as $\mathcal{F}_{\text{gcom}}$.

Simulator for corrupted phone During initialization, the simulator receives the initialization message from $\mathcal{A}$, and learns Phone's MAC keys Δ_P from it.

During computation, the simulator receives the input $\mathbf{A}$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{gcom}}$. The functionality $\mathcal{F}_{\text{gcom}}$ performs the computations and gives to simulator the output shares $\langle\langle \mathbf{y} \rangle\rangle_P$ and $\langle\langle \mathbf{w}^H \rangle\rangle_P$.

The simulator waits for $\mathcal{A}$ to start the computations of the many instances of $\mathcal{F}_{\text{randbit}}$. The simulator comes up with random shares $\langle\langle b_{i,t,u} \rangle\rangle_P$ for $i \in [\ell]$, $t \in [256]$, and $u \in [(\log \gamma_1) + 1]$, with the condition that the relevant linear combinations of these shares were equal to the coefficients of the elements of $\langle\langle \mathbf{y} \rangle\rangle_P$. Concretely, for each $i \in [\ell]$ and $t \in [256]$, the simulator

- randomly generates $\langle\langle b_{i,t,u} \rangle\rangle_P$ for $u \in \{1, \dots, \log \gamma_1\}$,
- defines $\langle\langle b_{i,t,0} \rangle\rangle_P \leftarrow \text{cff}_t(\langle\langle y_i \rangle\rangle_P) - \sum_{u=1}^{\log \gamma_1} 2^u \cdot \langle\langle b_{i,t,u} \rangle\rangle_P$,

where $\text{cff}_t(p)$ denotes the coefficient of X^t in the polynomial p .

The simulator computes $\langle\langle \mathbf{w} \rangle\rangle_P \leftarrow \mathbf{A} \cdot \langle\langle \mathbf{y} \rangle\rangle_P$. Also, the simulator sends all $\langle\langle b_{i,t,u} \rangle\rangle_P$ to $\mathcal{A}$, as output shares from the computations performed by $\mathcal{F}_{\text{randbit}}$.

The simulator waits for $\mathcal{A}$ to ask $\mathcal{M}_P$ to start the computations of $\mathcal{F}_{\text{high_bits}}$ for each coefficient of each element of $\langle\langle \mathbf{w} \rangle\rangle_P$. The simulator already knows the value $\langle\langle \mathbf{w} \rangle\rangle_P$. If the adversary's submission is different from the value that the simulator knows (for v- and mp-components), then the simulator tells $\mathcal{F}_{\text{gcom}}$ to stop.

The simulator will send $\langle\langle \mathbf{w}^H \rangle\rangle_P$ to the adversary as the output shares of computations of $\mathcal{F}_{\text{high_bits}}$. It will also tell $\mathcal{F}_{\text{gcom}}$ to proceed.

For modified ML-DSA, the protocol Π_{gcom} also includes the ideal functionality $\mathcal{F}_{\text{gcom}}^{\text{offl}}$. Together with the values $\langle\langle b_{i,t,u} \rangle\rangle_P$, the simulator also generates the

- On input $(\text{init}, \text{sid})$ from $\mathcal{P}$ and $(\text{init}, \text{sid}, \Delta_S)$ from $\mathcal{S}$: Store Δ_S .
- On input $(\text{compute}, \text{sid}, \langle v \rangle_P)$ from $\mathcal{P}$ and $(\text{compute}, \text{sid}, \langle v \rangle_S)$ from $\mathcal{S}$, do the following:
 - Recover v from the shares; check the MAC. If $\mathcal{S}$ is honest, but the MACs does not verify, then stop.
 - Pick random $v_P \in \mathbb{Z}_q$ and put $v_S \leftarrow (v - v_P) \bmod q$.
 - Let $\mathbf{x}_P \leftarrow \text{split}(v_P)$ and $\mathbf{x}_S \leftarrow \text{split}(v_S)$. Let $\mathbf{r} \leftarrow \mathbf{x}_P \cdot \Delta_S^{(Q)}$. Secret-share $\mathbf{r}$ as $[\mathbf{r}]^{(Q)}$.
 - Define $\langle \mathbf{p} \rangle_P \leftarrow (\mathbf{x}_P, [\mathbf{r}]_P)$ and $\langle \mathbf{p} \rangle_S \leftarrow (\mathbf{x}_S, [\mathbf{r}]_S)$.
 - If $\mathcal{X}$ is corrupted, then send $\langle \mathbf{p} \rangle_X$ to $\mathcal{S}$. Wait for the adversary's permission to proceed.
 - Send $(\text{result}, \text{sid}, \langle \mathbf{p} \rangle_{\bar{X}})$ to the honest $\bar{X}$.

Functionality 8: $\mathcal{F}_{\text{asymm_rp}}$: Ideal functionality for asymmetric reshare to parts

value $\langle \mathbf{e}_w \rangle_P$ and uses it in the computation of $\langle \langle b_{i,t,0} \rangle \rangle_P$. The adversary can send $(\text{crrreq}, \text{sid})$ to the simulator (adversary presumes $\mathcal{F}_{\text{gcom}}^{\text{offl}}$ to receive this) and get back $(\text{proceed?}, \text{sid}, \langle \mathbf{e}_w \rangle_P)$.

Simulator for corrupted server The protocol is symmetric wrt. the roles of Phone and Server.

The simulator is the same as the one for corrupted Phone, with $\mathcal{P}$ and $\mathcal{S}$ swapped.

B.11 Asymmetric reshare to parts

Alg. 12 gives the algorithm for splitting Phone's share of a value into digits and adding MACs to these digits. Alg. 12 is informally expected to have AP-security, i.e. it offers security against a malicious Phone, but only privacy against a malicious Server. We can define the protocol $\Pi_{\text{asymm_rp}}$, an instance of Prot. 4, on the basis of Alg. 12. The protocol is realized by a system consisting of the machines $\mathcal{M}_P$ and $\mathcal{M}_S$, as well as the ideal component $\mathcal{F}_{\text{asymm_rp}}^{\text{offl}}$ for correlated randomness, which is an instance of Func. 7. Note that $\mathcal{F}_{\text{asymm_rp}}^{\text{offl}}$ is an AP-secure functionality.

The ideal functionality $\mathcal{F}_{\text{asymm_rp}}$ securely realized by $\Pi_{\text{asymm_rp}}$ is given in Func. 8. We see that its output are the digits $\mathbf{x}_P$ and $\mathbf{x}_S$ of the shares of a random resharing of its input v ; there are also Server's MACs on the digits held by the Phone. A notable omission in Func. 8 is the corrupted Server's ability to define the algorithm $\mathfrak{A}$ running on the input. Indeed, as the server is actually passive in Alg. 12, it does not get a chance to change the output so precisely; it can only change its own output share.

In order to show that $\Pi_{\text{asymm_rp}}$ is at least as secure as $\mathcal{F}_{\text{asymm_rp}}$, let us present the simulators.

Simulator for corrupted phone During the initialization, the simulator does not get any information, because there is no Δ_P . At the beginning of the computation, the simulator receives $\langle v \rangle_P = (\llbracket v \rrbracket_P, \llbracket V \rrbracket_P)$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{asymm_rp}}$ (as part of the (compute)-commands). The ideal functionality continues and sends the output shares $\langle \mathbf{p} \rangle_P = (\mathbf{x}_P, \llbracket \mathbf{r} \rrbracket_P)$ to the simulator.

The simulator computes $m \leftarrow \text{split}^{-1}(\mathbf{x}_P)$. It generates a random $\llbracket M \rrbracket_P^{(q)}$. It sends m , $\llbracket M \rrbracket_P$ and $\llbracket \mathbf{r} \rrbracket_P$ to $\mathcal{A}$, after the latter has asked for the correlated randomness from $\mathcal{F}_{\text{asymm_rp}}^{\text{off}}$. The simulator waits for the adversary to tell $\mathcal{M}_P$ that it should send the message (w, M_w) to $\mathcal{M}_S$. The simulator verifies that $w = \llbracket v \rrbracket_P - m$ and $M_w = \llbracket V \rrbracket_P - \llbracket M \rrbracket_P$. If not, then the simulator tells $\mathcal{F}_{\text{asymm_rp}}$ to stop. Otherwise, the simulator tells $\mathcal{F}_{\text{asymm_rp}}$ to proceed.

Simulator for corrupted server During the initialization, the simulator learns Δ_S from $\mathcal{A}$. At the beginning of the computation, the simulator receives $\langle v \rangle_S = (\llbracket v \rrbracket_S, \llbracket V \rrbracket_S)$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{asymm_rp}}$. The ideal functionality continues and sends the output shares $\langle \mathbf{p} \rangle_S = (\mathbf{x}_S, \llbracket \mathbf{r} \rrbracket_S)$ to the simulator. The simulator computes $v_S = \text{split}^{-1}(\mathbf{x}_S)$ and $w \leftarrow v_S - \llbracket v \rrbracket_S$. The simulator generates a random $\llbracket M \rrbracket_S \in \mathbb{Z}_q$ and sends it and $\llbracket \mathbf{r} \rrbracket_S$ to $\mathcal{A}$, when the latter asks for them from $\mathcal{F}_{\text{asymm_rp}}^{\text{off}}$. The simulator tells $\mathcal{A}$ that $\mathcal{M}_S$ received the message w, M_w from $\mathcal{M}_P$, where $M_w = w \cdot \Delta_S - (\llbracket V \rrbracket_S - \llbracket M \rrbracket_S)$. The simulator also tells $\mathcal{F}_{\text{asymm_rp}}$ to proceed.

B.12 Characteristic vector

Alg. 15 gives the algorithm for computing the characteristic vector of a value shared modulo Q , where the resulting bits are shared modulo N . We can define an AP-secure protocol $\Pi_{\text{ch_vec}}$ as an instance of Prot. 4, implementing Alg. 15. The protocol $\Pi_{\text{ch_vec}}$ is realized by a system consisting of machines $\mathcal{M}_P$ and $\mathcal{M}_S$, as well as the ideal components $\mathcal{F}_{\text{declassify}}^{\text{AP}}$ and $\mathcal{F}_{\text{ch_vec}}^{\text{off}}$, where the latter is an instance of Func. 7. Let us give the simulators showing that $\Pi_{\text{ch_vec}}$ is a secure implementation of $\mathcal{F}_{\text{ch_vec}}$, where the latter is an instance of Func. 4, where the function f computes the characteristic vector of its input, the inputs to the functionality are secret-shared modulo Q , and the outputs are shared modulo N .

Simulator for corrupted phone During the initialization, the $\mathcal{A}$ notifies simulator of the initialization. At the beginning of the computation, the simulator receives $\langle v \rangle_P^{(Q)}$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{ch_vec}}$. The ideal functionality continues and gives the output $\langle \mathbf{b} \rangle_P^{(N)}$ to the simulator.

The simulator generates a random $\langle r \rangle_P^{(Q)}$. It also generates a random $f \in \mathbb{Z}_Q$ and defines $\langle d_i \rangle_P \leftarrow \langle b_{(i+f) \bmod Q} \rangle_P$ for all $i \in \mathbb{Z}_Q$. The simulator sends $\langle r \rangle_P^{(Q)}$ and $\langle \mathbf{d} \rangle_P$ to $\mathcal{A}$, after the latter has queried $\mathcal{F}_{\text{ch_vec}}^{\text{off}}$ for those.

The simulator computes $\langle f \rangle_P \leftarrow \langle v \rangle_P - \langle r \rangle_P$ and $\llbracket f \rrbracket_S \leftarrow f - \llbracket f \rrbracket_P$, and it expects $\mathcal{A}$ to command $\mathcal{F}_{\text{declassify}}^{\text{AP}}$ to declassify $\langle f \rangle_P$. If the value submitted by

the adversary is different from the value computed by the simulator then the simulator tells $\mathcal{F}_{\text{ch_vec}}$ to stop. The simulator tells $\mathcal{F}_{\text{ch_vec}}$ to proceed.

Simulator for corrupted server. During the initialization, the simulator receives Δ_S from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{ch_vec}}$. At the start of the execution with session identifier sid , the simulator receives $\langle v \rangle_S$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{ch_vec}}$. The ideal functionality continues to run and the simulator receives the output share $\langle \mathbf{b} \rangle_S$ from it.

The simulator picks a random $f \in \mathbb{Z}_Q$. It generates a random $r \in \mathbb{Z}_Q$ and secret-shares it as $\langle r \rangle$ (uses Δ_S to compute the MAC). Let $\mathbf{d}$ be the characteristic vector of r . The simulator secret-shares $\mathbf{d}$ as $\langle \mathbf{d} \rangle$ with the additional constraint that $\langle d_i \rangle_S = \langle b_{(i+f) \bmod Q} \rangle_S$ for each $i \in \mathbb{Z}_Q$. The simulator sends $\langle r \rangle_S$ and $\langle \mathbf{d} \rangle_S$ to the adversary, as the response of the latter asking for them from $\mathcal{F}_{\text{ch_vec}}^{\text{off}}$.

The simulator waits for $\mathcal{A}$ to instruct $\mathcal{F}_{\text{declassify}}^{\text{AP}}$ to declassify $\langle f \rangle_S$. The simulator knows what the expected value of $\langle f \rangle_S$ is, but the adversary may send a different $\llbracket f' \rrbracket_S$. Let $\varepsilon = \llbracket f \rrbracket_S - \llbracket f' \rrbracket_S$ be the difference. The simulator defines $f' \leftarrow f + \varepsilon$ and secret-shares f' into $\langle f' \rangle$, with the constraint that $\langle f' \rangle_S$ has already been given by the adversary. This “secret-sharing under a constraint” actually means that the simulator computes

$$\begin{aligned} & - \llbracket f' \rrbracket_P \leftarrow f' - \llbracket f' \rrbracket_S, \text{ and} \\ & - {}_{\text{ms}}\langle f' \rangle_P \leftarrow \llbracket f' \rrbracket_P \cdot \Delta_S - {}_{\text{ms}}\langle f' \rangle_S. \end{aligned}$$

Here all computations are modulo Q . Previously, $\langle \mathbf{d} \rangle^{(N)}$ had been computed in the same manner.

The simulator sends $(\text{result?}, sid, f', {}_{\text{ms}}\langle f' \rangle_P)$ to $\mathcal{A}$, as if coming from $\mathcal{F}_{\text{declassify}}^{\text{AP}}$. The simulator now creates the following description of an algorithm $\mathfrak{A}$ that would take $\langle v \rangle_P$ as an argument:

- Recover v from $\llbracket v \rrbracket_P$ and $\llbracket v \rrbracket_S$.
- Return the characteristic vector of $v + \varepsilon$.

The simulator sends the description of $\mathfrak{A}$ to the ideal functionality and tells it to proceed.

B.13 Short overflow

Alg. 16 gives the algorithm for computing the bits that indicate whether the value v , secret-shared over a small modulus Q , is equal to a constant $r - 1$, or is greater than or equal to r . The returned bits may be shared over a different modulus N . We can define an AP-secure protocol $\Pi_{\text{short_of}}$ as an instance of Prot. 4, implementing Alg. 16. This protocol is realized by a system consisting of the machines $\mathcal{M}_P$ and $\mathcal{M}_S$, as well as the ideal component $\mathcal{F}_{\text{ch_vec}}$. The protocol securely implements the ideal functionality $\mathcal{F}_{\text{short_of}}$, which is an instance of Func. 4, where the inputs are shared modulo Q , the outputs are shared modulo N , and the function f computes the previously mentioned two bits. Let us present the simulators showing that $\Pi_{\text{short_of}}$ indeed is at least as secure as $\mathcal{F}_{\text{short_of}}$.

Simulator for corrupted phone During the initialization, $\mathcal{A}$ notifies the simulator of the initialization. At the beginning of the computation, the simulator receives $\langle v \rangle_{\mathcal{P}}^{(Q)}$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{short_of}}$. The ideal functionality $\mathcal{F}_{\text{short_of}}$ continues and gives the output shares $\langle b \rangle_{\mathcal{P}}^{(N)}$ and $\langle c \rangle_{\mathcal{P}}^{(N)}$ to the simulator.

The simulator waits for $\mathcal{A}$ to invoke the computation of $\mathcal{F}_{\text{ch_vec}}$ with the argument $\langle v \rangle_{\mathcal{P}}$. If the argument received from $\mathcal{A}$ is the different from what was received from $\mathcal{A}$ and forwarded to $\mathcal{F}_{\text{short_of}}$ earlier, then the simulator tells $\mathcal{F}_{\text{short_of}}$ to stop. The simulator generates a random vector $\langle \mathbf{d} \rangle_{\mathcal{P}}^{(N)}$ of length Q , with constraints $\langle d_{r-1} \rangle_{\mathcal{P}} = \langle b \rangle_{\mathcal{P}}$ and $\sum_{i=r}^{Q-1} \langle d_i \rangle_{\mathcal{P}} = \langle c \rangle_{\mathcal{P}}$. The simulator sends $\langle \mathbf{d} \rangle_{\mathcal{P}}$ to $\mathcal{A}$ as output from $\mathcal{F}_{\text{ch_vec}}$. The simulator tells $\mathcal{F}_{\text{short_of}}$ to proceed.

Simulator for corrupted server During the initialization, the simulator receives $\Delta_{\mathcal{S}}$ from $\mathcal{A}$ and forwards it to the ideal functionality. At the beginning of the computation, the simulator receives $\langle v \rangle_{\mathcal{S}}$ from $\mathcal{A}$ and forwards it to the ideal functionality. The ideal functionality sends the shares $\langle b \rangle_{\mathcal{S}}^{(N)}$ and $\langle c \rangle_{\mathcal{S}}^{(N)}$ of the output to the simulator.

The simulator waits for $\mathcal{A}$ to invoke the computation of $\mathcal{F}_{\text{ch_vec}}$ with the argument $\langle v \rangle_{\mathcal{S}}$. As $\mathcal{F}_{\text{ch_vec}}$ is an instance of Func. 4, the adversary now expects to receive its share of the output from this functionality. The simulator thus generates a random $\langle \mathbf{d} \rangle_{\mathcal{S}}^{(N)}$ satisfying the constraints $\langle d_{r-1} \rangle_{\mathcal{S}} = \langle b \rangle_{\mathcal{S}}$ and $\sum_{i=r}^{Q-1} \langle d_i \rangle_{\mathcal{S}} = \langle c \rangle_{\mathcal{S}}$, and sends it to the adversary. The adversary now sends the description of the algorithm $\mathfrak{A}$ that it expects to be executed by $\mathcal{F}_{\text{ch_vec}}$. The algorithm $\mathfrak{A}$ expects to get $\langle v \rangle_{\mathcal{P}}$ as the input, and it will return a vector of booleans, which may be the characteristic vector of v .

The simulator will now create the description of the following algorithm $\mathcal{B}$ that takes $\langle v \rangle_{\mathcal{P}}$ as the input:

1. compute $\mathbf{d} \leftarrow \mathfrak{A}(\langle v \rangle_{\mathcal{P}})$, where $\mathbf{d} \in \mathbb{Z}_N^Q$;
2. let $b \leftarrow d_{r-1}$ and $c \leftarrow (\sum_{i=r}^{Q-1} d_i) \bmod N$;
3. return b and c .

The simulator sends the description of $\mathfrak{B}$ to $\mathcal{F}_{\text{short_of}}$, and also instructs it to proceed.

B.14 Long overflow

Alg. 17 gives the algorithm for computing the bit indicating whether the length- d vector of values $\mathbf{p}$, secret-shared over a small modulus Q , is larger than r^d , when we interpret the elements of $\mathbf{p}$ as digits in basis r (and the digits themselves are allowed to be larger than $r - 1$). The returned bit may be shared over a different modulus M , and there is a “middle” modulus N for the bits indicating the overflow of each single digit. We can define an AP-secure protocol $\Pi_{\text{long_of}}$ as an instance of Prot. 4, implementing Alg. 17. This protocol is realized by a system consisting of the machines $\mathcal{M}_{\mathcal{P}}$ and $\mathcal{M}_{\mathcal{S}}$, as well as ideal components

$\mathcal{F}_{\text{short_of}}$ (with input modulus Q and output modulus N) and $\mathcal{F}_{\text{ch_vec}}$ (with input modulus N and output modulus M). The protocol securely implements the ideal functionality $\mathcal{F}_{\text{long_of}}$, which is an instance of Func. 4, where the inputs are shared modulo Q , the outputs are shared modulo M , and the function f , applied to $(p_0, \dots, p_{d-1})$, outputs whether $\sum_{i=0}^{d-1} r^i p_i$ is greater or equal to r^d . Let us present the simulators showing that $\Pi_{\text{long_of}}$ indeed is at least as secure as $\mathcal{F}_{\text{long_of}}$.

Simulator for corrupted phone During the initialization, $\mathcal{A}$ notifies the simulator of the initialization. At the beginning of the computation, the simulator receives $\langle \mathbf{p} \rangle_{\mathbf{P}}^{(Q)}$ from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{long_of}}$. The functionality $\mathcal{F}_{\text{long_of}}$ continues and gives the output share $\langle z \rangle_{\mathbf{P}}^{(M)}$ to the simulator. The simulator generates random $\langle b_i \rangle_{\mathbf{P}}^{(N)}$ and $\langle c_i \rangle_{\mathbf{P}}^{(N)}$ for $i \in [d]$.

The simulator waits for $\mathcal{A}$ to invoke the computation of $\mathcal{F}_{\text{short_of}}$ with each $\langle p_i \rangle_{\mathbf{P}}$. The simulator simulates $\mathcal{F}_{\text{short_of}}$ running and responds that the shares of the output are $\langle b_i \rangle_{\mathbf{P}}^{(N)}$ and $\langle c_i \rangle_{\mathbf{P}}^{(N)}$. If the inputs the adversary gives to $\mathcal{F}_{\text{short_of}}$ are different from the inputs the adversary gave to the computation of long overflow, then the simulator tells $\mathcal{F}_{\text{long_of}}$ to stop. Indeed, the ideal component $\mathcal{F}_{\text{short_of}}$ of the real system would catch such changes, because it checks the MACs.

The simulator computes $\langle m \rangle_{\mathbf{P}} \leftarrow \sum_{i=1}^{d-1} 2^{i-1} \cdot \langle b_i \rangle_{\mathbf{P}} + \sum_{i=0}^{d-1} 2^i \cdot \langle c_i \rangle_{\mathbf{P}}$. It waits for $\mathcal{A}$ to invoke the computation of $\mathcal{F}_{\text{ch_vec}}$ with the argument that should be equal to $\langle m \rangle_{\mathbf{P}}$. If it is not, then the simulator tells $\mathcal{F}_{\text{long_of}}$ to stop, because $\mathcal{F}_{\text{ch_vec}}$ would have caught a wrong MAC.

The simulator generates a random $\langle \mathbf{k} \rangle_{\mathbf{P}}^{(M)}$ of length N , with the constraint $\sum_{i=2^d-1}^{N-1} \langle k_i \rangle_{\mathbf{P}} = \langle z \rangle_{\mathbf{P}}$. It tells $\mathcal{A}$ that the output of $\mathcal{F}_{\text{ch_vec}}$ is $\langle \mathbf{k} \rangle_{\mathbf{P}}$. The simulator also tells $\mathcal{F}_{\text{long_of}}$ to proceed.

Simulator for corrupted server During the initialization, the simulator receives $\Delta_{\mathbf{S}}$ from $\mathcal{A}$ and forwards it to the ideal functionality. At the beginning of the computation, the simulator receives $\langle \mathbf{p} \rangle_{\mathbf{S}}$ from $\mathcal{A}$ and forwards it to the ideal functionality. The ideal functionality continues its execution and gives the share $\langle z \rangle_{\mathbf{S}}^{(M)}$ of the output to the simulator. Indeed, $\mathcal{F}_{\text{long_of}}$ is an instance of Func. 4.

The simulator waits for $\mathcal{A}$ to invoke the computation of $\mathcal{F}_{\text{short_of}}$ with each $\langle p_i \rangle_{\mathbf{S}}$. The adversary expects to get back the shares of the output; the simulator makes up random $\langle b_i \rangle_{\mathbf{S}}$ and $\langle c_i \rangle_{\mathbf{S}}$ and sends them to $\mathcal{A}$. The adversary then sends the descriptions of algorithms $\mathfrak{A}_0, \dots, \mathfrak{A}_{d-1}$ to the simulator (these interactions for all the d invocations may be arbitrarily interleaved).

The simulator waits for $\mathcal{A}$ to invoke the computation of $\mathcal{F}_{\text{ch_vec}}$ with $\langle m \rangle_{\mathbf{S}}^{(N)}$. If the adversary does not try to affect the computations then the equality $\langle m \rangle_{\mathbf{S}} = \sum_{i=1}^{d-1} 2^{i-1} \cdot \langle b_i \rangle_{\mathbf{S}} + \sum_{i=0}^{d-1} 2^i \cdot \langle c_i \rangle_{\mathbf{S}}$ would hold. However, the adversary may be performing an active attack, and the value of its submitted $\langle m \rangle_{\mathbf{S}}^{(N)}$ may be something else. The simulator now creates a random $\langle \mathbf{k} \rangle_{\mathbf{S}}^{(M)}$ of length N ,

with the constraint $\sum_{i=2^{d-1}}^{N-1} \langle k_i \rangle_S = \langle z \rangle_S$. The simulator sends $\langle \mathbf{k} \rangle_S^{(M)}$ to the adversary, as if it being the output share of $\mathcal{F}_{\text{ch_vec}}$. The adversary sends the description of an algorithm $\mathfrak{B}$ to the simulator.

The simulator now constructs the following algorithm $\mathfrak{C}$ that takes $\langle \mathbf{p} \rangle_P$ as an argument:

1. For each $i \in [d]$, let $(b_i, c_i) \leftarrow \mathfrak{A}_i(\langle p_i \rangle_P)$. The values b_i and c_i are elements of $\mathbb{Z}_N$.
2. For each $i \in [d]$, compute $\langle b_i \rangle_P$ from b_i and $\langle b_i \rangle_S$, using Δ_S to compute the MAC.
3. Similarly, compute $\langle c_i \rangle_P$ for each $i \in [d]$.
4. Let $\langle m \rangle_P \leftarrow \sum_{i=1}^{d-1} 2^{i-1} \cdot \langle b_i \rangle_P + \sum_{i=0}^{d-1} 2^i \cdot \langle c_i \rangle_P$.
5. Let $\mathbf{k} \leftarrow \mathfrak{B}(\langle m \rangle_P)$. The elements of the vector $\mathbf{k}$ belong to $\mathbb{Z}_M$.
6. Return $\sum_{i=2^{d-1}}^{N-1} k_i$.

The simulator sends the description of $\mathfrak{C}$ to $\mathcal{F}_{\text{long_of}}$, and also tells it to proceed.

B.15 Inequality check

Alg. 18 gives the algorithm to compute the bit indicating whether a secret-shared value is less than a constant. The input is shared over the modulus q , while the output may be shared over some different modulus M , and there are also moduli Q and N (all prime), radix r and length d parameterizing the algorithm. We can define an AP-secure protocol Π_{ineq} as an instance of Prot. 4, implementing Alg. 18. This protocol is realized by a system consisting of the machines $\mathcal{M}_P$ and $\mathcal{M}_S$, as well as ideal components $\mathcal{F}_{\text{asymm_rp}}$ (with output modulus Q) and $\mathcal{F}_{\text{long_of}}$ (with input modulus Q , “middle” modulus N , and output modulus M). The protocol securely implements the ideal functionality $\mathcal{F}_{\text{ineq}}$, which is an instance of Func. 4, where the inputs are shared modulo q , the output bit is shared modulo M , and the function f , applied to private v and public C , outputs whether $v < C$. Let us present the simulators showing that Π_{ineq} indeed is at least as secure as $\mathcal{F}_{\text{ineq}}$.

Simulator for corrupted phone During the initialization, the $\mathcal{A}$ notifies simulator of the initialization. At the beginning of the computation, the simulator receives $\langle v \rangle_P$ and C from $\mathcal{A}$ and forwards them to the ideal functionality. The functionality $\mathcal{F}_{\text{ineq}}$ continues and gives the output share $\langle z \rangle_P^{(M)}$ to the simulator. The simulator waits for $\mathcal{A}$ to invoke $\mathcal{F}_{\text{asymm_rp}}$ with the input $\langle v \rangle_P$; if the value submitted to $\mathcal{F}_{\text{asymm_rp}}$ differs from the input to the computation then the simulator instructs $\mathcal{F}_{\text{ineq}}$ to stop. Otherwise the simulator generates a random vector $\langle \mathbf{p} \rangle_P^{(Q)}$ and sends it to the adversary, as if coming from $\mathcal{F}_{\text{asymm_rp}}$.

The simulator waits for $\mathcal{A}$ ’s commands invoke $\mathcal{F}_{\text{long_of}}$ with the input $\langle \mathbf{p} \rangle_P$ twice. If the adversary tries to change $\langle \mathbf{p} \rangle_P$ then the simulator tells $\mathcal{F}_{\text{ineq}}$ to stop. The simulator generates random $\langle x \rangle_P^{(M)}$ and $\langle y \rangle_P^{(M)}$ satisfying the constraint $\langle z \rangle_P = \langle 1 \rangle_P - \langle y \rangle_P + \langle x \rangle_P - \langle 0 \rangle_P$. The simulator sends $\langle x \rangle_P$ and $\langle y \rangle_P$ to $\mathcal{A}$, as if coming from the two computations done by $\mathcal{F}_{\text{long_of}}$. The simulator also tells $\mathcal{F}_{\text{ineq}}$ to proceed.

Simulator for corrupted server During the initialization, the simulator receives Δ_S from $\mathcal{A}$ and forwards it to the ideal functionality. At the beginning of the computation, the simulator receives $\langle v \rangle_S$ from $\mathcal{A}$ and forwards it to the ideal functionality. The simulator also receives from $\mathcal{F}_{\text{ineq}}$ the share $\langle z \rangle_S^{(M)}$ of the output, because $\mathcal{F}_{\text{ineq}}$ is an instance of Func. 4.

The simulator waits for $\mathcal{A}$ to submit $\langle v \rangle_S$ to $\mathcal{F}_{\text{asymm_rp}}$. Let $\varepsilon \in \mathbb{Z}_q$ be the difference between the value $\llbracket v \rrbracket_S$ submitted to $\mathcal{F}_{\text{asymm_rp}}$ and the original value $\llbracket v \rrbracket_S$ that the simulator forwarded to $\mathcal{F}_{\text{ineq}}$. Generate a random $\langle \mathbf{p} \rangle_S^{(Q)}$ and send it to $\mathcal{A}$ as if coming from $\mathcal{F}_{\text{asymm_rp}}$. Let $w_S \leftarrow \text{split}^{-1}(\llbracket \mathbf{p} \rrbracket_S)$. Let $\langle c \rangle_S^{(M)} \leftarrow \langle 0 \rangle_S$; update $\llbracket c \rrbracket_S$ to 1 if $w_S \geq C$.

Wait for $\mathcal{A}$ to instruct $\mathcal{S}$ to submit $\langle \mathbf{a} \rangle_S^{(Q)}$ and $\langle \mathbf{b} \rangle_S^{(Q)}$ to $\mathcal{F}_{\text{long_of}}$ for computation. The adversary expects to get the output shares $\langle x \rangle_S^{(M)}$ and $\langle y \rangle_S^{(M)}$. The simulator randomly generates them, such that the constraint $\langle z \rangle_S = \langle 1 \rangle_S + \langle x \rangle_S - \langle y \rangle_S - \langle c \rangle_S$ is satisfied, and sends them to $\mathcal{A}$. The adversary sends the descriptions of algorithms $\mathfrak{A}_1$ and $\mathfrak{A}_2$ meant for the two computations of $\mathcal{F}_{\text{long_of}}$. These algorithms expect $\langle \mathbf{p} \rangle_P$ as an input. The simulator creates the description of the following algorithm $\mathcal{B}$, expecting $\langle v \rangle_P$ as an input:

1. Recover v from $\llbracket v \rrbracket_P$ and $\llbracket v \rrbracket_S$.
2. Let $\llbracket \mathbf{p} \rrbracket_P \leftarrow \text{split}(v + \varepsilon) - \llbracket \mathbf{p} \rrbracket_S$. I.e. $\llbracket \mathbf{p} \rrbracket_P$ is computed so that the condition for the output of Alg. 12 holds, if we put $(v + \varepsilon) \bmod q$ in the position of v there.
3. Extend $\llbracket \mathbf{p} \rrbracket_P$ to $\langle \mathbf{p} \rangle_P$ by computing P 's MAC share from $\llbracket \mathbf{p} \rrbracket_P$, Δ_S and S 's MAC share.
4. Let $x \leftarrow \mathfrak{A}_1(\langle \mathbf{p} \rangle_P)$ and $y \leftarrow \mathfrak{A}_2(\langle \mathbf{p} \rangle_P)$. Here $x, y \in \mathbb{Z}_M$.
5. Output $(1 + x - y - c) \bmod M$.

The simulator sends the description of $\mathfrak{B}$ to $\mathcal{F}_{\text{ineq}}$, and also tells it to proceed.

B.16 Rejection check

Alg. 13 gives the algorithm for the “second half” of a signing attempt: the rejection check. It takes a number of secret-shared values and returns 1 if all these values are inside bounds. We can define an AP-secure protocol $\Pi_{\text{rej_check}}$ as an instance of Prot. 4, implementing Alg. 13. The protocol $\Pi_{\text{rej_check}}$ is implemented by a system consisting of machines $\mathcal{M}_P$ and $\mathcal{M}_S$, and the ideal components $\mathcal{F}_{\text{ineq}}$ and $\mathcal{F}_{\text{ch_vec}}$ (with parameters named in Alg. 13). The protocol $\Pi_{\text{rej_check}}$ is at least as secure as the ideal functionality $\mathcal{F}_{\text{rej_check}}$ described in Sec. 5. Let us present the simulators that attest to this.

Simulator for corrupted phone During the initialization, $\mathcal{A}$ notifies the simulator of the initialization. At the beginning of the computation, the simulator receives $\langle \mathbf{z} \rangle_P$ and $\langle \mathbf{x} \rangle_P$ from the $\mathcal{A}$ and forwards them to the ideal functionality. The ideal functionality $\mathcal{F}_{\text{rej_check}}$ continues to execute and gives the output share $\langle b_0 \rangle_P^{(2)}$ to the simulator.

The simulator waits for $\mathcal{A}$ to request the start of the computation of $\mathcal{F}_{\text{ineq}}$ for each coefficient of each element of $\langle \mathbf{z} \rangle_{\mathbf{P}}$ and $\langle \mathbf{x} \rangle_{\mathbf{P}}$ (shifted by appropriate amount). If the adversary changes these inputs in any other way than shifting them by the given amount, then the simulator tells $\mathcal{F}_{\text{rej_check}}$ to stop. The simulator generates a random vector $\langle \mathbf{v} \rangle_{\mathbf{P}}$ (using the modulus M) of the correct length $(256 \cdot (k + \ell))$, and uses the elements of this vector as $\mathbf{P}$'s output shares from all the computations of $\mathcal{F}_{\text{ineq}}$. The simulator sends the vector $\langle \mathbf{v} \rangle_{\mathbf{P}}^{(M)}$ to $\mathcal{A}$, element by element.

The simulator computes $\langle s \rangle_{\mathbf{P}} \leftarrow \sum_{i=0}^{256(k+\ell)-1} (\langle 1 \rangle_{\mathbf{P}} - \langle v_i \rangle_{\mathbf{P}})$ and expects $\mathcal{A}$ to request the start of the computation of $\mathcal{F}_{\text{ch_vec}}$ with the input $\langle s \rangle_{\mathbf{P}}$. If the input is anything else then the simulator tells $\mathcal{F}_{\text{rej_check}}$ to stop. Otherwise, the simulator generates a random vector $\langle \mathbf{b} \rangle_{\mathbf{P}}^{(2)}$, where $\langle b_0 \rangle_{\mathbf{P}}$ has already been defined. The simulator sends $\langle \mathbf{b} \rangle_{\mathbf{P}}$ to the adversary and tells $\mathcal{F}_{\text{rej_check}}$ to proceed.

Simulator for corrupted server During the initialization, the simulator receives $\Delta_{\mathbf{S}}$ from $\mathcal{A}$ and forwards in to the ideal functionality. At the beginning of the computation, the simulator receives $\langle \mathbf{z} \rangle_{\mathbf{S}}$ and $\langle \mathbf{x} \rangle_{\mathbf{S}}$ from $\mathcal{A}$ and forwards them to the ideal functionality. The simulator also receives the output share $\langle b_0 \rangle_{\mathbf{S}}$ from $\mathcal{F}_{\text{rej_check}}$.

The simulator waits for $\mathcal{A}$ to start the computation of $\mathcal{F}_{\text{ineq}}$ for each coefficient of each element of $\langle \mathbf{z} \rangle_{\mathbf{S}}$ and $\langle \mathbf{x} \rangle_{\mathbf{S}}$ (shifted by appropriate amount). The adversary expects to get the output shares of the results of each comparison. Hence the simulator generates a random $\langle \mathbf{v} \rangle_{\mathbf{S}}$ (of the correct length) and sends it to $\mathcal{A}$, element by element. The adversary responds by sending the algorithm descriptions of the algorithms $\mathfrak{A}_0, \dots, \mathfrak{A}_{256(k+\ell)-1}$, each of them expecting a share $\langle z_i \rangle_{\mathbf{P}}$ or $\langle x_i \rangle_{\mathbf{P}}$ as input and producing an element of $\mathbb{Z}_M$ as output, to the simulator.

The simulator computes $\langle s \rangle_{\mathbf{S}} \leftarrow \sum_{i=0}^{256(k+\ell)-1} (\langle 1 \rangle_{\mathbf{S}} - \langle v_i \rangle_{\mathbf{S}})$ and expects $\mathcal{A}$ to start the computation of $\mathcal{F}_{\text{ch_vec}}$ for $\langle s \rangle_{\mathbf{S}}$. The adversary expects to get the output share of this computation; the simulator generates a random $\langle \mathbf{b} \rangle_{\mathbf{S}}^{(2)}$, where the element $\langle b_0 \rangle_{\mathbf{S}}$ has already been received from $\mathcal{F}_{\text{rej_check}}$, and sends $\langle \mathbf{b} \rangle_{\mathbf{S}}$ to $\mathcal{A}$. The adversary responds by sending to the simulator the description of the algorithm $\mathfrak{B}$ that takes a share $\langle s \rangle_t^{(M)}$ *extsfP* as input and returns a vector of bits.

The simulator creates the description of the following algorithm $\mathfrak{C}$ that takes $\langle \mathbf{z} \rangle_{\mathbf{P}}$ and $\langle \mathbf{x} \rangle_{\mathbf{P}}$ as input and outputs a bit:

1. Apply each of $\mathfrak{A}_i$ to one of the elements of $\langle \mathbf{z} \rangle_{\mathbf{P}}$ or $\langle \mathbf{x} \rangle_{\mathbf{P}}$, shifted by the required amount (either by $\gamma_1 - \beta - 1$ or by $\gamma_2 - \beta - 1$). Let $\mathbf{v} \in \mathbb{Z}_M^{256(k+\ell)}$ be the vector of outputs from these algorithms $\mathfrak{A}_i$.
2. Let $s \leftarrow \sum_i v_i$. Compute $\langle s \rangle_{\mathbf{P}}$ so, that $(\langle s \rangle_{\mathbf{P}}, \langle s \rangle_{\mathbf{S}})$ will be a secret-sharing of s . This computation uses $\Delta_{\mathbf{S}}$.
3. Let $\mathbf{b} \leftarrow \mathfrak{B}(\langle s \rangle_{\mathbf{P}})$.
4. Return b_0 .

The simulator sends the description of $\mathfrak{C}$ to $\mathcal{F}_{\text{rej_check}}$ and also tells it to proceed.

C Correctness of protocols

C.1 Zero equality checking protocol

Alg. 8 gives a two-party protocol for checking whether a secret-shared value $\langle\langle v \rangle\rangle$ is zero. Let us show that if $v < B$ then it indeed produces the correct result. We have denoted $a = \lfloor q/B \rfloor$. The vector $\mathbf{t}$ is constructed so that the bit $t_{\lfloor m/a \rfloor}$ is equal to 1, while all other bits are equal to 0. There are two cases.

Case 1: $v = 0$. Then $d = m$. The protocol outputs the secret-shared bit $t_{\lfloor m/a \rfloor}$. By the construction of the vector $\mathbf{t}$, this bit is equal to 1.

Case 2: $v \neq 0$. The protocol outputs the secret-shared bit $t_{\lfloor d/a \rfloor}$. If we consider all values as integers (i.e. elements of $\mathbb{Z}$, not elements of $\mathbb{Z}_q$), then we can write $\lfloor \frac{d}{a} \rfloor = \lfloor \frac{(m+av) \bmod q}{a} \rfloor$. We have $m < q$ and $av \leq \frac{q}{B} \cdot v < \frac{q}{B} \cdot B = q$. Hence $0 \leq m + av < 2q$. Let us consider two cases.

Case 2a: $m + av < q$. Then $\lfloor \frac{d}{a} \rfloor = \lfloor \frac{m+av}{a} \rfloor = \lfloor \frac{m}{a} \rfloor + v \neq \lfloor \frac{m}{a} \rfloor$. Hence by the construction of the vector $\mathbf{t}$, the bit $t_{\lfloor d/a \rfloor}$ is equal to 0.

Case 2b: $q \leq m + av < 2q$. Then $d = m + av - q \leq m + \frac{q}{B} \cdot (B - 1) - q \leq m + (q - a) - q = m - a$. Hence $\lfloor \frac{d}{a} \rfloor \leq \lfloor \frac{m-a}{a} \rfloor = \lfloor \frac{m}{a} \rfloor - 1$. We obtain $\lfloor d/a \rfloor \neq \lfloor m/a \rfloor$, hence the bit $t_{\lfloor d/a \rfloor}$ is equal to 0.

C.2 Long Overflow

The input to the long overflow protocol (Alg. 17) is a multi-digit number $\mathbf{p}$, where the least significant digit is indexed as 0 and the most significant digit as $d - 1$. With the help of the short overflow protocol, we find for each digit p_i whether it overflows (indicated by the bit c_i) or “almost overflows” (indicated by the bit b_i). Here “almost overflowing” means that the overflowing is propagated: if the next lowest digit overflows then this one will overflow as well. The whole number overflows if one of the following cases occurs:

- $c_{d-1} = 1$, i.e. the most significant digit overflows;
- $b_{d-1} = c_{d-2} = 1$, i.e. the second most significant digit overflows, and this propagates through the most significant digit;
- $b_{d-1} = b_{d-2} = c_{d-3} = 1$, i.e. the third most significant digit overflows, and this propagates through the two most significant digits;
- etc.
- $b_{d-1} = b_{d-2} = \dots = b_1 = c_0 = 1$, i.e. the least significant digit overflows, and this propagates through all digits.

Note also that all these cases are mutually exclusive, because each pair of them contains some position i , where one case has c_i and the other one has b_i . Both b_i and c_i cannot be 1 at the same time.

Alg. 17 computes the weighted sum m of all b_i and c_i , such that this sum will be at least 2^{d-1} iff one of the above cases is true. We see that for each case given above, the weights given to the bits involved in this case sum up to exactly 2^{d-1} . If some other bits are also set, then the sum can be greater than that.

It is also not difficult to see that if none of the cases hold, then the sum will be less than 2^{d-1} . Indeed, suppose that s is the greatest index where $c_s = 1$, and there exists some $t \in \{s+1, \dots, d-1\}$, such that $b_t = 0$. Then the weighted sum can be at most

$$\sum_{i=0}^s 2^i + \sum_{i=s+1}^{d-1} 2^{i-1} - 2^{t-1} = 2^{s+1} - 1 + 2^{d-1} - 2^s - 2^{t-1} = 2^{d-1} - 1 + 2^s - 2^{t-1},$$

which is less than 2^{d-1} , because $t-1 \geq s$.

C.3 Inequality check

Alg. 18 gives the protocol for finding out whether a secret-shared value is less than a fixed number. Its correctness follows through the same arguments as the correctness of Attrapadung et al.'s inequality protocol [4]. Let us give a reshare of these algorithms.

We have the number v shared as $(\llbracket v \rrbracket_P + \llbracket v \rrbracket_S) \bmod q$, and the constant C . Alg. 18 applies the asymmetric reshare to parts to it; if we ignore the plurality of digits, then we can think of this operation as *resharing* $\llbracket v \rrbracket$: we change $\llbracket v \rrbracket_P$ and $\llbracket v \rrbracket_S$ (now denoted w_S in Alg. 18), such that they are still the shares of v . Let us write

- $v_1 := \llbracket v \rrbracket_P$;
- $v_2 := \llbracket v \rrbracket_S$;
- $v'_2 := (\llbracket v \rrbracket_S - C) \bmod q$.

If we ignore the shift $r^d - q$ (the necessity of which we have argued in the main body of the paper), and the plurality of digits, then we can think that the shares of $\llbracket \mathbf{a} \rrbracket$ are v_1 and v_2 , while the shares of $\llbracket \mathbf{b} \rrbracket$ are v_1 and v'_2 . When computing the overflows, we figure out the bit x indicating whether $v_1 + v_2 \geq q$. Similarly, the bit y indicates whether $v_1 + v'_2 \geq q$. We also have the bit c indicating whether $v_2 \geq C$. It turns out that knowing the values of these three bits is sufficient to figure out whether $v < C$. Let us consider all eight possibilities for the values of the bits x, y, c , and figure out the output bit of the inequality check in each case.

- 000** $v_1 + v_2 < q$, $v_1 + v'_2 < q$, $v_2 < C$. In this case, $v'_2 = v_2 + q - C$. We have $C = v_2 - v'_2 + q > v_2 - v'_2 + v_1 + v'_2 = v_1 + v_2$, i.e. the result should be 1.
- 001** $v_1 + v_2 < q$, $v_1 + v'_2 < q$, $v_2 \geq C$. In this case, $v'_2 = v_2 - C$. We have $0 \leq v_1 + v_2 - C < q$. We have $C \leq (v_1 + v_2) \bmod q$, i.e. the result should be 0.
- 010** $v_1 + v_2 < q$, $v_1 + v'_2 \geq q$, $v_2 < C$. In this case, $v'_2 = v_2 + q - C$. We have $C = v_2 - v'_2 + q \leq v_2 - v'_2 + v_1 + v'_2 = v_1 + v_2$, i.e. the result should be 0.
- 011** $v_1 + v_2 < q$, $v_1 + v'_2 \geq q$, $v_2 \geq C$. In this case, $v'_2 = v_2 - C$. This case is impossible, because subtraction means that $v'_2 \leq v_2$, but the first two inequalities mean that $v'_2 > v_2$.

- 100** $v_1 + v_2 \geq q$, $v_1 + v'_2 < q$, $v_2 < C$. In this case, $v'_2 = v_2 + q - C > v_2$. This case is impossible.
- 101** $v_1 + v_2 \geq q$, $v_1 + v'_2 < q$, $v_2 \geq C$. In this case, $v'_2 = v_2 - C$. We have $C = v_2 - v'_2 > v_2 - q + v_1$, i.e. the result should be 1.
- 110** $v_1 + v_2 \geq q$, $v_1 + v'_2 \geq q$, $v_2 < C$. In this case, $v'_2 = v_2 + q - C$. We have $C = v_2 - v'_2 + q > v_2 - q + v_1$ (because $v'_2 < q$ and $q > v_1$), i.e. the result should be 1.
- 111** $v_1 + v_2 \geq q$, $v_1 + v'_2 \geq q$, $v_2 \geq C$. In this case, $v'_2 = v_2 - C$. We have $C = v_2 - v'_2 \leq v_2 - q + v_1$, i.e. the result should be 0.

We see that in all six cases that are actually possible, the result is $1 + x - y - c$.

D Additional details on the Zero-Knowledge of Dilithium identification protocol

D.1 Existing discussion

Existing proposals for privacy-preserving implementations of Dilithium (mostly masked implementations in hardware) must have faced the issue of computing $H(\mu, \mathbf{w}^H)$ (line 2 of Alg. 2). One can try to do it either under the protection of privacy-preserving techniques, or open $\mathbf{w}^H$ and compute that hash in the clear. When Cozzo and Smart [32] made their complexity estimates of implementing the signing algorithms of NIST candidate PQ signature schemes, they stated that one needs a non-standard hardness assumption (similarly to [7]) to claim the security of opening $\mathbf{w}^H$, and thus their estimations include the privacy-preserving computation of H .

Barthe et al. [7] propose mechanisms to mask the GLP lattice-based signature scheme [56], the construction of which is similar to Dilithium, but the commitment is decoupled from the secret $\mathbf{y}$ by adding a small error instead of taking the high bits. They indeed introduce a new hardness assumption, defining the *Rejected-Decisional Compact Knapsack Problem* and assuming that it is hard for certain parameter values. The presumably hard problem basically requires one to distinguish rejected transcripts of the GLP scheme from random values.

Migliore et al. [78] ignore the issue, basically implying that the no-abort HVZK property of the Dilithium identification protocol is enough for the security of making $\mathbf{w}^H$ public. Azouaoui et al. [5] acknowledge the problem and the necessity of a LWR-type assumption, but conjecture that as the error introduced by rounding $\mathbf{w}$ is significantly higher than the error $\mathbf{s}_2$ in the public key (that is protected by assuming the hardness of MLWE), the publication of $\mathbf{w}^H$ even in rejected runs will be safe.

Coron et al. [31] state that the results of Devevey et al. [41] allow $\mathbf{w}^H$ to be revealed in all runs. In [30] they retract this statement and propose to treat the security of opening $\mathbf{w}^H$ (which they do in their construction) as an open problem.

Input: $pk = (\mathbf{A}, \mathbf{t})$, $sk = (\mathbf{s}_1, \mathbf{s}_2)$
Output: A sample from the distribution of transcripts

```

1  $c \xleftarrow{\$} B_\tau$ 
2  $\mathbf{y} \xleftarrow{\$} \tilde{S}_{\gamma_1}^\ell$ 
3  $\mathbf{z} \leftarrow \mathbf{y} + c \cdot \mathbf{s}_1$ 
4 if  $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$  then
5    $\mathbf{r}_0 \leftarrow \text{LowBits}(\mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t})$ 
6   if  $\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$  then
7      $\mathbf{x}_0 \leftarrow \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$ 
8      $\mathbf{w} \leftarrow \mathbf{x}_0 + c \cdot \mathbf{s}_2$ 
9     return  $(\mathbf{w}^H, c, \mathbf{z})$ 
10  else
11     $\mathbf{x}_0 \leftarrow \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$ 
12     $\mathbf{w} \leftarrow \mathbf{x}_0 + c \cdot \mathbf{s}_2$ 
13    return  $(\mathbf{w}^H, c, \perp)$ 
14 else
15    $\mathbf{w} \leftarrow \mathbf{A} \cdot \mathbf{y}$ 
16   return  $(\mathbf{w}^H, c, \perp)$ 

```

Algorithm 19: Generating transcripts of Dilithium identification protocol:
1st hybrid

D.2 Transforming the transcript generator

Alg. 3 shows the generation of the transcripts for the Dilithium identification protocol. Let us rewrite the control flow of this algorithm, emphasizing the rejection check in line 3. We also rewrite the definition of $\mathbf{r}_0$ as the low bits of $\mathbf{x}_0 = \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$ and move it to down to this branch where it is used. Recall also that $\mathbf{x}_0 = \mathbf{w} - c \cdot \mathbf{s}_2$.

In line 4 of Alg. 19, the probability (taken over the choice of $\mathbf{y}$) of passing the check does not depend on the value of $c \cdot \mathbf{s}_1$. Hence we can randomly pick the branch to go into and generate the values $\mathbf{y}$ and/or $\mathbf{z}$ in each branch, subject to the conditions that must be satisfied in this branch. We also take into account that in the first branch, where $\mathbf{z}$ is returned, we have $\mathbf{w}^H = \mathbf{x}_0^H$. Let $p_{\mathbf{z}}$ be the probability of passing the check. This change will give us the transcript generator in Alg. 20.

In Alg. 20, we can think of the value $\mathbf{z}$ generated in line 3 as only determining, into which branch we go in line 5. We will then regenerate $\mathbf{z}$ in each branch, subject to the condition in line 5 holding or not. This gives us Alg. 4. Let us discuss the **else**-branch a bit more. We have to (re)generate $\mathbf{z}$, subject to the following conditions:

- $\mathbf{z} \in S_{\gamma_1 - \beta - 1}^\ell$;
- $\|\text{LowBits}(\mathbf{x}_0)\|_\infty \geq \gamma_2 - \beta$, where $\mathbf{x}_0 = \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$.

We have $\mathbf{x}_0 = \mathbf{w} - c \cdot \mathbf{s}_2 = \mathbf{A} \cdot \mathbf{y} - c \cdot \mathbf{s}_2$. We also have $\mathbf{y} = \mathbf{z} - c \cdot \mathbf{s}_1$. After these substitutions, we get the sampling in line 4 of Alg. 4.

Input: $pk = (\mathbf{A}, \mathbf{t})$, $sk = (\mathbf{s}_1, \mathbf{s}_2)$
Output: A sample from the distribution of transcripts

```

1  $c \xleftarrow{\$} B_\tau$ 
2 if Bernoulli( $p_z$ ) then
3    $\mathbf{z} \xleftarrow{\$} S_{\gamma_1 - \beta - 1}^\ell$ 
4    $\mathbf{r}_0 \leftarrow \text{LowBits}(\mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t})$ 
5   if  $\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$  then
6      $\mathbf{x}_0 \leftarrow \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$ 
7     return  $(\mathbf{x}_0^H, c, \mathbf{z})$ 
8   else
9      $\mathbf{x}_0 \leftarrow \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$ 
10     $\mathbf{w} \leftarrow \mathbf{x}_0 + c \cdot \mathbf{s}_2$ 
11    return  $(\mathbf{w}^H, c, \perp)$ 
12 else
13    $\mathbf{y} \xleftarrow{\$} \{\mathbf{y}' \in \bar{S}_{\gamma_1}^\ell \mid \|c \cdot \mathbf{s}_1 + \mathbf{y}'\|_\infty \geq \gamma_1 + \beta\}$ 
14    $\mathbf{w} \leftarrow \mathbf{A} \cdot \mathbf{y}$ 
15   return  $(\mathbf{w}^H, c, \perp)$ 

```

Algorithm 20: Generating transcripts of Dilithium identification protocol:
 2nd hybrid

Input: $pk = (\mathbf{A}, \mathbf{t})$
Output: A sample from the distribution of transcripts

```

1  $c \xleftarrow{\$} B_\tau$ 
2 if Bernoulli( $p_z$ ) then
3    $\bar{\mathbf{z}} \xleftarrow{\$} S_{\gamma_1 - \beta - 1}^\ell$ 
4   if  $\|\text{LowBits}(\mathbf{A} \cdot \bar{\mathbf{z}} - c \cdot \mathbf{t})\|_\infty < \gamma_2 - \beta$  then
5      $\mathbf{z} \xleftarrow{\$} \{\mathbf{z}' \in S_{\gamma_1 - \beta - 1}^\ell \mid \|\text{LowBits}(\mathbf{A} \cdot \mathbf{z}' - c \cdot \mathbf{t})\|_\infty < \gamma_2 - \beta\}$ 
6     return  $(\text{HighBits}(\mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}), c, \mathbf{z})$ 
7   else
8      $\mathbf{b} \xleftarrow{\$} R_q^k$ 
9     return  $(\mathbf{b}^H, c, \perp)$ 
10 else
11    $\mathbf{b} \xleftarrow{\$} R_q^k$ 
12   return  $(\mathbf{b}^H, c, \perp)$ 

```

Algorithm 21: Generating transcripts of Dilithium identification protocol
 in zero-knowledge

Finally, the MLWR assumptions allow us to replace the values returned in the last two branches of Alg. 4, giving us Alg. 21.

E Approximate counts of communicated bits and communication rounds

The goal of this appendix is to compare the communication costs of our protocol with the costs of [11]. For comparisons that are as fair as possible, we have to consider similar settings for these protocols. Our protocol is a two-party protocol (with the additional CRP), where one of these parties may be malicious. The protocol of [11] is designed for an arbitrary number of parties with honest majority. But the subprotocols of the protocol of [11] are generic, and make sense also in two-party setting (with one malicious party, and with the additional CRP). Hence the comparisons will be done in this setting.

We count the number of bits in the values that the two parties have to exchange, and we also count the number of communication rounds. We ignore the MAC checks, because these will be amortized over a large number of parallel operations, adding a minuscule overhead to each communication round. Let us start with the following basic counts:

- A multiplication of two (private) k -bit values requires each party to send two k -bit values [39]. It may be more efficient in settings where several multiplications with linearly dependent arguments are being computed. Dilithium does not have such settings.
- The conversion of a k -bit value into a characteristic vector requires each party to send one k -bit value.
- The conversion of a bit into a value shared over a larger modulus requires each party to send one bit.

All these operations require one round of communication.

E.1 Rabbit

The Rabbit comparison protocol [76] is used by [11] in both the high bits computation and in rejection checking. Hence let us estimate its communication costs separately as well. The following operations in [76] are used in [11]:

- Comparing a bitwise shared value (with m bits) and a public value. Its cost is the same as a **PrefixOR** over m bits. A work-efficient algorithm requires $2\lceil \log m \rceil - 1$ rounds and $2m - \log m$ one-bit multiplications. A round-efficient algorithm requires $\lceil \log m \rceil$ rounds and $\approx m \log m$ one-bit multiplications.
- Comparing a value shared over $\mathbb{Z}_q$ with a public value. It requires two comparisons of a bitwise shared value (with $m = \lceil \log q \rceil$ bits) with a public value; these can be done in parallel. Before that, it uses an **edaBit** [50] to convert from $\mathbb{Z}_q$ to $\mathbb{Z}_2^m$; this takes one round and m bits of communication per party. Note that the result of the comparison is shared over $\mathbb{Z}_2$.

If q is the ML-DSA modulus, then the comparison of a private value with a public constant (the result is in $\mathbb{Z}_2$; it would take one more bit and one more round to get it back to $\mathbb{Z}_q$) requires:

- Using work-efficient **PrefixOR**: 10 rounds, and the following number of bits sent per party:

$$23 + 2 \cdot 2 \cdot (2 \cdot 23 - 5) = 187 \text{ .}$$

- Using round-efficient **PrefixOR**: 6 rounds, and the following number of bits sent per party:

$$23 + 2 \cdot 2 \cdot (23 \cdot 5) = 483 \text{ .}$$

E.2 High bits

We consider the necessary number of rounds and sent bits for a protocol that takes as input a vector of private values (shared over $\mathbb{Z}_q$), and returns a vector of public values: the high bits of the inputs. The protocols in both this paper and in [11] process each value separately, hence the total communication cost is $256 \cdot k$ times the cost of the high bits computation and declassification for a single private input, while the total number of rounds is the same as finding the high bits of a single value. The following estimates the costs for finding the high bits of a single value shared over $\mathbb{Z}_q$.

The cost of our protocol Our high-bits computation consists of a symmetric reshare, followed by a zero check, and a declassification. The declassification takes one round and 23 bits of communication from each party. The zero check is also a declassification, it also takes one round and 23 bits of communication from each party.

The symmetric reshare to parts protocol begins with a declassification (one round, 23 bits sent by each party) and finishes by converting bits (shared over $\mathbb{Z}_2$) back to sharing over a bigger modulus (one round, 1 bit sent by each party per bit being converted, the number of converted bits is either 176 (for ML-DSA-44) or 145 (for ML-DSA-65 and ML-DSA-87)).

The central part of the symmetric reshare to parts protocol is essentially an addition circuit. It takes four rounds (which we have reduced to two by using binomial tuples). At each round, it multiplies a bit with a vector of bits. Hence, at each round, the number of communicated bits (per party) is one larger than the length of the vector. The lengths of the vectors are (62, 48, 32, 32) (for ML-DSA-44) or (62, 66, 64, 32) (for ML-DSA-65 and ML-DSA-87). The number of communicated bits in the central phase is four plus the sum of the lengths, i.e. 178 or 228.

In total:

- Symmetric reshare to parts requires four rounds, and either 377 (for ML-DSA-44) or 396 (for ML-DSA-65 and ML-DSA-87) bits communicated by each party.
- The high bits computation requires six rounds, and either 423 (for ML-DSA-44) or 442 (for ML-DSA-65 or ML-DSA-87) bits communicated by each party.

The cost of DecomposeMod A significant step in DecomposeMod is the bit-wise addition of (private) m -bit values; in our setting, $m = 23$. Let us assume that an addition circuit with carry-propagation is used for this, otherwise the number of rounds will be too high ($m - 1$). This circuit performs ca. $2m$ one-bit multiplications and converts to a protocol with $\lceil \log m \rceil$ rounds.

DecomposeMod in [11] consists of a bit decomposition, followed by the conversion of these bits into $\mathbb{Z}_q$, and declassification of the high bits. The latter two steps take two rounds and 27 bits sent by either party.

Bit decomposition consists of the following steps:

- declassification. One round, 23 bits sent by either party.
- Comparing a bitwise shared value (with 23 bits) and a public value. If we use work-efficient protocol for this, then it takes 9 rounds and $2 \cdot (2 \cdot 23 - 5) = 82$ bits of communication from each of the parties. If we use round-efficient protocol for this, then it takes 5 rounds and ca. 230 bits of communication from each of the parties.
- Adding together two 23-bit bitwise shared values. This takes five rounds, and ca. $2 \cdot 2 \cdot 23 = 92$ bits of communication from each of the parties.

Putting it together, we need 17 or 13 rounds (depending on the used PrefixOR) algorithm and 224 or 372 bits communicated per party. This is for ML-DSA-65 and ML-DSA-87; more is necessary for ML-DSA-44 and [11] does not spell it out clearly.

E.3 Rejection check

We consider the necessary number of rounds and sent bits that will take as input a vector of private values (shared over $\mathbb{Z}_q$) and a vector of public bounds, and return a vector of private bits (shared over $\mathbb{Z}_2$ or over some other smallish modulus) indicating whether these values were below these bounds. This vector of private bits has to be post-processed; there are a number of ways to do it, with [11, Protocol 2] presenting a particularly efficient way to do it.

The cost of our protocol With our comparison protocol (which is an adaptation of [4]), we do $256 \cdot (k + \ell)$ comparisons of a private value against a public bound. Our protocols (single comparison) have the following costs:

- Asymmetric reshare to parts requires P to send $\lceil \log q \rceil$ bits to S.
- The entire comparison protocol consists of one asymmetric reshare, and two long overflows.
 - A long overflow consists of one characteristic vector for values shared over modulus $N = 67$, and $d = 6$ short overflows. To compute the characteristic vector, each party has to send 7 bits.
 - * The short overflow consists of a characteristic vector from the modulus $Q = 29$. In order to compute it, each party has to send 5 bits. The short overflow takes a single round of communication.

Hence the short overflows require 30 bits to be sent by each party. A long overflow requires 37 bits to be sent by each party in two rounds.

The comparison protocol thus takes three rounds. It requires the server to send 74 bits, while the client has to send 97 bits.

Thus the number of rounds for all the comparisons is 3 (there is one extra round at the end of Alg. 13, where the count is turned to a characteristic vector). The number of bits sent by **P** and **S** respectively is 198656 and 151552 (for ML-DSA-44), or 273152 and 208384 (for ML-DSA-65), or 372480 and 284160 (for ML-DSA-87).

The cost of using Rabbit Bienstock et al. [11] use Rabbit for the comparisons that make up the rejection checks; they also do $256 \cdot (k + \ell)$ comparisons of a private value against a public bound. The number of rounds (10 or 5, depending on the used PrefixOR algorithm) is equal to that of a single comparison. The number of bits sent by each party is 382796 or 989184 (for ML-DSA-44), 526592 or 1360128 (for ML-DSA-65), or 718080 or 1854720 (for ML-DSA-87).

F About malicious CRP

The discussions in Sec. 7 mention the “cheap additional checks” that are necessary in the case where the CRP learns the produced signatures. Indeed, the CRP can affect the randomness used in the key generation and in signing. Even though Phone and Server add their own randomness, the resulting distribution may still be different from the desired one. Consider the vector $\mathbf{y}$ that is used to mask the value $c \cdot \mathbf{s}_1$ in $\mathbf{z}$. If the coefficients of the elements of $\mathbf{y}$ are correctly distributed (uniformly over $\{-\gamma_1, \dots, \gamma_1 - 1\}$), then $\mathbf{z}$ does not depend on $c \cdot \mathbf{s}_1$ after the rejection sampling. If the distribution of $\mathbf{y}$ is not correct, then, for a fixed $c \cdot \mathbf{s}_1$, the distribution of a coefficient of an element of $\mathbf{z}$ no longer has to be uniform over a certain segment that includes the entire $\{-\gamma_1 + \beta + 1, \dots, \gamma_1 - \beta - 1\}$, and the rejection check does not necessarily remove the dependency any more.

The CRP contributes the correlated randomness also to other steps of the signing, but this no longer has to be verified to be correct. Indeed, once $\mathbf{y}$ has been correctly generated:

- CRP cannot affect the value of $\mathbf{w}$, because no additional correlated randomness is used.
- CRP can affect the value of $\mathbf{w}^H$. This can only cause the challenge c to be different. It will be verified at the end of the signing protocol and become known to CRP only if a valid signature is created.
- After $\mathbf{y}$ and $\mathbf{w}^H$ have been created, the CRP has no further influence to the values of $\mathbf{z}$ and $\mathbf{x}$. The CRP can affect whether the rejection checks of these values pass, but similarly to a malicious server, the best strategy of the CRP is to not tamper with the rejection checks.

- For the modified ML-DSA [11], the CRP selects the additional error $\mathbf{e}_w$. The effects of sampling it wrongly are not larger than what the CRP can achieve while tampering with the computation of the high bits $\mathbf{w}^H$. Also, the value $\mathbf{e}_w$ is not sensitive if the rejection checks pass [11, Thm. 1].

These discussions do not cover the case, where the *values* in the correlated randomness are correct, but the *MACs* are wrong. If MACs are wrong, then selective failures of the signing protocol are possible, we discuss this in Sec. 7. Indeed, the failures can be selective due to Phone and Server not checking *all* MACs in the correlated randomness, e.g. the MACs of the unused elements of characteristic vectors.

How can P and S check in privacy-preserving manner that a random bit $\llbracket b \rrbracket$ received from the CRP and additively shared over $\mathbb{Z}_q$, is actually a bit, i.e. $b \in \{0, 1\}$? They could compute $\llbracket b \rrbracket^2 - \llbracket b \rrbracket$, open the result and verify that it is 0. This would require the CRP to give them a multiplication triple; actually, a squaring doublet would suffice, and this doublet would not have to be verified for correctness. Also, P and S would both have to send an element of $\mathbb{Z}_q$ to each other in order to use the squaring doublet. As the number of bits to check is $\ell \cdot 256 \cdot ((\log \gamma_1) + 1)$, this is a pretty significant overhead (51.75 KiB for ML-DSA-44, 100.6 KiB for ML-DSA-87; compare this with Table 2).

Fortunately, the checking of these bits can be batched [9]. Let $B \in \mathbb{N}$ and let $\omega_1, \dots, \omega_{2B} \in \mathbb{Z}_q$ be points over which we're going to interpolate polynomials. As $\mathbb{Z}_q$ is NTT-friendly, it makes sense to select B as a power of two and the points as roots of unity. Let $\llbracket b_1 \rrbracket, \dots, \llbracket b_B \rrbracket$ be the private values that P and S want to check (that they are bits). Consider the polynomial $f(X) \in \mathbb{Z}_q[X]$ of degree at most $B - 1$ satisfying $f(\omega_i) = b_i$ for all $i \in \{1, \dots, B\}$; let P and S compute the shares of the coefficients of f . Let $g(X) = f(X) \cdot (f(X) - 1)$. Let the additively shared values $\llbracket c_{B+1} \rrbracket, \dots, \llbracket c_{2B} \rrbracket$ be included in the correlated randomness, where $c_i = g(\omega_i)$. Let P and S compute also the shares of the coefficients of g , assuming that $g(\omega_1) = \dots = g(\omega_B) = 0$. P and S now pick a random value $r \in \mathbb{Z}_q$ and verify that $\llbracket g(r) \rrbracket = \llbracket f(r) \rrbracket \cdot (\llbracket f(r) \rrbracket - 1)$; this requires a single squaring doublet. If some of the values $b_1, \dots, b_B$ were not bits, then the polynomial equality $g(X) = f(X) \cdot (f(X) - 1)$ cannot hold and the check discovers this with probability at least $(1 - \frac{2B}{q})$. The check can be repeated with other values r to reduce the soundness error.

If we take $B = 2^{12}$ (considering that $|\mathbb{Z}_q^*|$ is divisible by 2^{13}), then the soundness error of a single check is ca. 2^{-10} , meaning that we need 13 of them for the soundness of ca. 2^{-128} . The number of bits to check is 18432 (for ML-DSA-44) or 25600 (for ML-DSA-65) or 35840 (for ML-DSA-87), meaning that we need 5 or 7 or 9 batches. For each batch we need 13 squarings, each of which requires 23 bits to be sent by each party. The number of bits a party sends is thus 1495 or 2093 or 2691, which increases the communication reported in Table 2 by less than 2%. Recall also that these checks can be done in parallel with the rest of the protocol, and the performance of the whole protocol depends more on the latency than the available bandwidth.